Technical and Operational Manual

My Document Pile

(MyDocpile)

Table of Contents

Abstract 6
1. Introduction 8
2. The Login Process 9
2.1 Technical Description 9
2.2 The Workflow 10
2.3 Usability 12
3. The Backend 14
3.1 Technical Description 14
3.2 The Workflow 15
3.3 Usability 16
4. The Main User Interface 18
4.1 Technical Description 18
4.2 The Workflow 20
4.3 Usability 21
5. The Settings Dialog 23
5.1 Technical Description 23
5.2 The Workflow 25
5.3 Usability 26
6. The Multilingual Interface 28
6.1 Technical Description 28
6.2 The Workflow 29
6.3 Usability 29
7. The End-to-End Encryption 31
7.1 Technical Description 31
7.2 The Workflow 32
7.3 Usability 33
8. The Grid View 35
8.1 Technical Description 35
8.2 The Workflow 36
8.3 Usability 37
9. The Gallery View 38
9.1 Technical Description 38
9.2 The Workflow 39
9.3 Usability 40
10. The Office View 42

10.1 Technical Description 42
10.2 The Workflow 44
10.3 Usability 44
11. The Preview 46
11.1 Technical Description 46
11.2 The Workflow 48
11.3 Usability 50
12. Online Office (OnlyOffice Integration) 52
12.1 Technical Description 52
12.2 The Workflow 53
12.3 Usability 54
13. The Text and Code Editor 55
13.1 Technical Description 55
13.2 The Workflow 56
13.3 Usability 57
14. Search 59
14.1 Technical Description 59
14.2 The Workflow 61
14.3 Usability 62
15. Multi Rename Tool 64
15.1 Technical Description 64
15.2 The Workflow 65
15.3 Usability 66
16. Public Sharing 68
16.1 Technical Description 68
16.2 The Workflow 70
16.3 Usability 73
17. The SFTP Mode (Server Admin Mode) 75
17.1 Technical Description 75
17.2 The Workflow 79
17.3 Usability 81
18. The Help and Ticketing System 83
18.1 Technical Description 83
18.2 The Workflow 85
18.3 Usability 86
19. The Administrative UI and Configuration Management 88

19.1 Technical Description 88
19.2 The Workflow 90
19.3 Usability 92
20. Cronjob Housekeeping 94
20.1 Technical Description 94
20.2 The Workflow 97
20.3 Usability 98
21. Core File Operations and Transfer Logic 100
21.1 Technical Description 100
21.2 The Workflow 101
21.3 Usability 103
22. User Identity and Access Management 104
22.1 Technical Description 104
22.2 The Workflow 105
22.3 Usability 106
23. System Logging, Telemetry, and Diagnostics 108
23.1 Technical Description 108
23.2 The Workflow 109
23.3 Usability 110
24. Security Architecture and Web Application Firewall 111
24.1 Technical Description 111
24.2 The Workflow 112
24.3 Usability 113
Appendix A: Emergency Decryption of E2E Encrypted Files 114
A.1 Technical Description 114
A.2 The Workflow 115
A.3 Usability 117
Appendix B: The Heartbeat Mechanism 119
B.1 Technical Description 119
B.2 The Workflow 120
B.3 Usability and Security Considerations 121
Appendix C: Multilanguage Files 123
C.1 Technical Description 123
Appendix D: Help File Layout 124
D.1 Technical Description 124
Appendix E: Server Data File Structure 126

E.1 Technical Overview of the Flat-File Architecture....126
E.2 User Profiles and Interface State (user_profiles/)....126
E.3 Security, Authorization, and Locking Ledgers....128
E.4 Application State and Sharing Databases (data/)....129
E.5 The Inter-Process Communication (IPC) Matrix (data/ssh_ipc/)....130
E.6 Cryptographic and Structural In-Tree Data....131
Appendix F: Relevant config.php Variables....133
F.1. Technical Description....133
F.2. The Workflow....136
F.3. Usability....137
Appendix G: Application Programming Interface (API) and Stateless Endpoints....138
G.1 Technical Description....138
G.2 The Workflow....139
G.3 Usability....140
Appendix H: Installation, Deployment, and Server Requirements....141
H.1 Technical Description....141
H.2 The Workflow....142
H.3 Usability....143

Abstract

This manual presents a comprehensive description of *MyDocpile*, a highly advanced, web-based file management and cloud storage platform designed to replicate the responsiveness and usability of native desktop file explorers while operating entirely within a standard web browser.

The system is architected around ***a security-first, zero-knowledge philosophy***, combining a ***hardened PHP backend*** with a dynamic, state-driven ***JavaScript single-page application frontend***. All core data, configuration, and user files are ***stored outside the public web root***, thereby minimizing the attack surface and enforcing strict cryptographic and filesystem boundaries.

A central theme of the platform is its rigorous ***security architecture***. *MyDocpile* integrates enterprise-grade authentication mechanisms, including *Argon2id* password hashing with automated legacy hash migration, subnet-level brute-force mitigation, k-Anonymity breach detection via *Have I Been Pwned*, and enforced multi-factor authentication under elevated risk conditions. Stateless and stateful authentication flows—such as "Remember Me" tokens and *PWA App Mode*—are implemented using selector–validator patterns, *HMAC* binding, and session hardening techniques to prevent hijacking, fixation, and replay attacks. A localized *Web Application Firewall*, strict *Content Security Policies*, and pervasive *CSRF* protection further harden the platform against injection, XSS, and automated attacks.

The ***backend*** operates as an asynchronous API gateway that separates presentation logic from data processing. It enforces strict path-resolution jails, role-based access control matrices, and concurrency optimizations through deliberate session lock release for heavy I/O operations. This design allows users to perform large uploads, archive operations, and batch filesystem manipulations without blocking interface responsiveness. Integration with external document services, such as ONLYOFFICE, is achieved through tightly controlled stateless JWT-based bridges.

On the ***client side***, the user interface is a highly modular, device-adaptive SPA supporting list, grid, gallery, dual-pane "Commander Mode," and split-pane Office workflows. Advanced interaction paradigms—including keyboard-centric navigation, command palettes, drag-and-drop batch operations, and lazy-loaded thumbnail pipelines—provide a native-like user experience even for directories containing thousands of files. The system is

fully internationalized, supporting dynamic language switching, right-to-left layouts, and localized UI metadata.

A defining capability of *MyDocpile* is its **integrated End-to-End Encryption** system. Using WebCrypto APIs, all encryption and decryption occur client-side, ensuring the server retains zero knowledge of file contents or filenames. A KEK/DEK envelope model, aggressive key-derivation parameters, session-scoped memory isolation, and seamless cross-vault transfers balance cryptographic rigor with day-to-day usability.

1. Introduction

MyDocpile is an advanced, web-based file management and cloud storage platform architected to deliver a native desktop-like experience within a standard web browser. All files, code and config as well as data, are stored outside of www-root and just referred to by a small index.php stub. The system is built upon a hybrid architecture utilizing a robust PHP backend for secure file operations, stateful session management, and cryptographic boundary enforcement, paired with a highly dynamic, vanilla JavaScript frontend. This frontend relies heavily on asynchronous API communication to prevent page reloads, ensuring a fluid, uninterrupted user experience while minimizing server overhead.

The application is designed with a zero-trust philosophy regarding both user inputs and client states. It incorporates strict Content Security Policies (CSP) to mitigate cross-site scripting (XSS) and injection attacks, sophisticated anti-brute-force mechanisms operating at the subnet level, and stateless bridge endpoints to integrate safely with external services, specifically ONLYOFFICE document servers. A core operational principle of the platform is the seamless integration of End-to-End Encryption (E2EE). By utilizing WebCrypto APIs, the system performs AES-256-GCM encryption directly within the client's browser. This ensures that the server retains zero knowledge of the encrypted file contents or their original filenames, establishing a true zero-knowledge privacy architecture.

Furthermore, the platform's user interface is exceptionally modular and data-driven. It features a responsive grid and list system, a dual-pane "Commander Mode" for complex batch operations, and a highly customizable settings engine that adapts layout rules based on device heuristics (differentiating between desktop environments, tablets, and mobile devices). The environment is fully internationalized, dynamically serving translations and layout directions—including Right-to-Left (RTL) alignments—based on user preference profiles or browser headers. The following chapters detail the technical specifications, workflows, and usability paradigms of the core modules driving this ecosystem.

2. The Login Process

2.1 Technical Description

The authentication and authorization module, primarily governed by main.php and main_login.php, serves as the perimeter defense for the application. It establishes strict environmental controls and cryptographic verifications before any core business logic is permitted to execute.

Session Hardening and State Management The application aggressively overrides default PHP session behaviors to prevent session hijacking and fixation. It forces session.use_strict_mode to reject uninitialized session IDs provided by a client, disables session.use_trans_sid to prevent IDs from leaking into URLs, and increases the session.sid_length to 64 bytes with 6 bits per character to maximize entropy. Cookies are bound strictly using session_set_cookie_params, enforcing HttpOnly, Secure (when over HTTPS), and SameSite => Lax policies. Furthermore, upon a successful login or 2FA verification, session_regenerate_id(true) is invoked to issue a completely new session identifier, neutralizing any potential pre-login session fixation vectors.

Cryptographic Credential Verification and Migration User credentials are verified against stored hashes. To maintain forward secrecy and resist offline cracking, the system utilizes the Argon2id hashing algorithm [1]. The login logic includes an automated migration routine: if a user successfully authenticates against a legacy SHA-256 hash, the system immediately recalculates the password using password_hash($password, PASSWORD_ARGON2ID) and updates the user database file ($user_db) via a secure flock() stream replacement using regular expressions (preg_replace). To mitigate timing attacks—where an attacker might measure server response times to enumerate valid usernames—the system verifies the provided password against a hardcoded dummy Bcrypt hash whenever a non-existent username is submitted, ensuring CPU consumption remains identical regardless of user validity.

Breach Detection (k-Anonymity via HIBP) Post-authentication, the system calculates the SHA-1 hash of the plaintext password. The first five characters of this hash are transmitted to the HaveIBeenPwned (HIBP) API [2]. The API returns a list of breached hash suffixes matching that prefix. The system searches this response for the remainder of the user's hash. If a match is found, the system flags the $hibp_breached variable to true, which strictly

overrides any auto-login or "Remember Me" bypasses, forcing the user into a mandatory Two-Factor Authentication (2FA) flow.

Brute-Force and Rate Limiting Topologies The ClientSecurity WAF class handles initial flood control, geo-IP validation, and ASN checks. Beyond this, the main_login.php script implements an exponential backoff brute-force tracker. It evaluates incoming requests based on their subnet mask (e.g., /24 for IPv4 or /64 for IPv6) rather than individual IPs to thwart distributed attacks from local networks. If the failure count exceeds $login_failures, the $dynamic_block_time is calculated using $login_block_seconds * pow($brute_force_factor, $bf_level). This state is persisted in a locked JSON ledger ($login_bruteforce_file).

Stateful "Remember Me" Tokens and App Mode Persistent logins utilize a selector-validator pattern to mitigate database theft vulnerabilities. A 16-byte random hex string acts as the selector, and a 32-byte string acts as the validator. Only the password_hash() of the validator is stored on the server (stateful_tokens.json). To prevent privilege escalation via token tampering, the stored token payload includes an HMAC (hash_hmac('sha256', ...)) binding the username and expiration date to the server's private $api_key. Additionally, an "App Mode" mechanism allows Progressive Web Apps (PWAs) to bypass the login screen natively. This requires an incoming GET request containing a cryptographic signature ($app_mode_sig) calculated using the token selector, the string 'app_mode', the client IP, and the User-Agent.

2.2 The Workflow

The login sequence operates as a rigid state machine, progressing through the following stages:

1. **Pre-Flight Security Execution:**
    - Global headers (CSP, Referrer-Policy, X-Content-Type-Options) are injected.
    - ClientSecurity WAF inspects the incoming request for malicious payloads, banned IPs, or flood behaviors. If triggered, execution halts with an HTTP 403/404 or redirects to a blocking sinkhole.
    - Subnet block ledger is evaluated. If the IP falls within an active exponential backoff window, the request is discarded.
2. **Token & Environment Verification:**

- The system checks for the presence of the auth_cookie.
- If present, the selector is matched in the database, the validator is verified against the stored hash, and the HMAC signature is authenticated.
- If App Mode is requested (via PWA headers) and the app_mode_confirmed signature matches, authentication succeeds without UI rendering (skipping directly to Step 7), unless the user is an Administrator (Admins are denied App Mode bypasses).

3. **Credential Submission (POST):**
   - CSRF token is validated. If mismatched or missing, the process stalls with an artificial delay (usleep) and rejects the request.
   - Username is aggressively sanitized using filter_var and preg_replace to strip control characters and prevent injection.
   - Password verification occurs (with simultaneous dummy-hash checking for timing attack mitigation).
4. **Post-Authentication Analysis:**
   - If credentials are valid, the automatic Argon2id migration hook fires if necessary.
   - The SHA-1 prefix is dispatched to the HIBP API.
   - The user's role and privileges are extracted from $user_details.
5. **Two-Factor Authentication Routing:**
   - If the user bypassed 2FA previously via a valid device cookie, and no HIBP breach was detected, they proceed to Step 7.
   - Otherwise, a secure 8-character code and a 64-byte URL token are generated.
   - The pending 2FA state is written to $verify_store_file and bound to the current session_id().
   - An email containing both the code and the direct magic link is dispatched via mail().
6. **2FA Verification:**

- The system waits for either a POST request containing the 8-character code (validated via hash_equals) or a GET request hitting the magic link matching the 64-byte token.
- Five failed 2FA attempts will instantly destroy the session.

7. **Session Establishment:**
   - session_regenerate_id(true) is called.
   - The brute-force ledger for the user's subnet is reset.
   - A session fingerprint is generated by hashing the User-Agent.
   - If the user opted to be remembered, a new selector-validator cookie pair is generated, HMAC-signed, and dispatched.
   - The user is redirected to processing.php with a time-bound, HMAC-signed payload for final entry into the cloud interface.

2.3 Usability

From the perspective of an end-user, the login process is designed to feel entirely frictionless until a security threshold is crossed. When arriving at the portal, the user is presented with a minimal, distraction-free interface featuring the application logo, a username field, a password field (with a toggle to reveal the typed characters), and a "Remember on this device" checkbox. The interface automatically adapts its language based on previous visits or browser settings.

If the user enters an incorrect password, the system intentionally pauses for a few seconds before displaying a generic "Invalid username/password" message. This delay is imperceptible to a human trying to remember a password, but it drastically slows down automated bots attempting to guess passwords. If the user makes too many mistakes in a row, they are locked out, and the login screen is replaced by a hard block page.

The real complexity is hidden behind the scenes. If a user logs in and the system detects that their specific password was recently leaked in a global data breach (like a LinkedIn or Yahoo hack), the interface immediately halts. A prominent, red-bordered security warning modal appears over the login screen. It clearly explains that while their account here hasn't been hacked, their password is no longer mathematically safe to use.

Because of this risk, the system forces them into a Two-Factor Authentication loop. The user is instructed to check their email. Upon opening the email, they have two distinct, highly convenient choices: they can manually type the provided 8-character code into the waiting browser window, or, if they are checking their email on the same device, they can simply tap the large "Approve Login" button. Tapping the button securely verifies their identity in the background and automatically redirects their original, waiting login tab directly into the cloud file manager, ensuring high security without the traditional frustration of copying and pasting OTP codes.

3. The Backend

3.1 Technical Description

The backend architecture, primarily encapsulated within the controller.server.php module, operates as a high-performance, asynchronous API gateway. It is constructed around an object-oriented paradigm utilizing the MyCloudServer class, which serves as the central router for all client-side XMLHttpRequest (XHR) and Fetch API calls. The backend strictly communicates via JavaScript Object Notation (JSON) payloads, completely divorcing the data processing layer from the frontend HyperText Markup Language (HTML) rendering layer.

A foundational element of the backend is the resolve() method, which functions as an impenetrable cryptographic and filesystem jail. When the frontend requests an operation on a file, it transmits a relative path. The resolve() method sanitizes this input by stripping null bytes and directory traversal characters (e.g., ../). It then utilizes PHP's native realpath() function to map the logical path to the physical disk. Crucially, the engine performs a strict string-position comparison to ensure the resulting absolute path resides entirely within the user's mathematically defined root directory or the designated .recycle_bin. If an operation attempts to breach this boundary, the method yields a boolean false, and the controller immediately terminates the execution thread. The resolver also possesses specialized logic to virtualize ZIP archives, allowing the system to securely validate paths that terminate inside a compressed container without extracting it to disk.

Complementing the path resolver is the sanitizeAndValidateName() method, a rigorous input filter applied to all incoming file and folder creations, renames, and uploads. This mechanism actively strips characters that are illegal across various operating systems to ensure cross-platform database integrity. Furthermore, it implements a hardcoded blocklist to prevent the instantiation of system-critical files (such as .htaccess, .env, or .ssh/id_rsa). To thwart Remote Code Execution (RCE) vulnerabilities, the sanitizer rejects files with executable extensions (e.g., .php, .sh, .cgi, .jsp) and employs regular expressions to detect and neutralize double-extension bypass attempts (e.g., payload.php.jpg).

The controller also integrates a granular Role-Based Access Control (RBAC) engine. Before evaluating any filesystem Input/Output (I/O) request, the requested action is cross-referenced against the

$MYCLOUD_RIGHTS_MATRIX. If the session's assigned role dictates that a specific operation (e.g., delete or batch_rename) is restricted, the backend aborts the process at the memory level, bypassing the disk entirely.

To manage high-concurrency environments, the backend employs a deliberate session-release protocol. Standard PHP environments lock the session file during script execution, which forces subsequent requests from the same user to queue sequentially. The MyCloudServer router explicitly invokes session_write_close() immediately after authentication and CSRF validation for heavy I/O operations such as large file uploads, recursive directory deletions, or archive extractions. This architectural decision permits the web server to process multiple heavy requests from a single client in parallel.

Finally, the backend implements specialized stateless bridge endpoints to communicate with external microservices, specifically ONLYOFFICE document servers. Because these external servers do not share the user's browser context or cookies, the backend generates mathematically signed JSON Web Tokens (JWT). The controller includes strict encoding (jwtEncode) and decoding (jwtDecode) algorithms utilizing the HS256 hashing standard, verifying the integrity of inbound webhooks and file conversion payloads before committing any externally modified data back into the user's filesystem jail.

3.2 The Workflow

The backend request lifecycle follows a strict, sequential validation and execution chain:

- **Initialization and Interception:**
    - The script verifies it is not being accessed directly via the browser address bar.
    - Stateless external requests (e.g., /myCloudOfficeFetch/ or /myCloudOfficeCallback) are intercepted immediately, authenticated via JWT signatures, processed, and exited without initializing the standard user session state.
    - Session states, global configurations, and User Profile variables are mapped to the active execution thread.
- **Security Gatekeeping:**

    - The checkRateLimit() method is invoked. It reads an isolated, user-specific JSON ledger in the temporary directory to calculate request frequencies across a sliding time window, returning an HTTP 429 status if thresholds are exceeded.
    - The incoming myCloud_token is strictly compared against the myCloud_csrf_token stored in the session utilizing a timing-safe string comparison.
    - The requested myCloud_action is validated against the user's specific RBAC clearance matrix.
- **Path Resolution and Concurrency Control:**
    - Target source and destination paths are passed through the resolve() jail.
    - For actions flagged as heavy I/O operations (e.g., upload, zip, delete), the PHP session lock is explicitly released.
- **Action Execution:**
    - The router switches the execution flow to the designated private method (e.g., actionUpload()).
    - If applicable, anti-malware hooks (ClamAV) are executed on temporary files before they are committed to the primary storage volume.
    - Filesystem modifications are committed to the physical disk.
- **Response Formulation:**
    - Active output buffers are purged to guarantee no extraneous characters corrupt the response.
    - The execution result, along with any updated structural data, is serialized into a JSON string and transmitted back to the client with the appropriate application/json headers.

3.3 Usability

While the backend operates entirely out of sight, its architectural design directly defines the physical responsiveness and reliability experienced by the user. Because the backend explicitly releases session locks during heavy file operations, a user can highlight a folder containing gigabytes of data,

instruct the system to compress it into a ZIP archive, and immediately open a new tab or navigate to a different directory to continue working. The interface remains highly fluid and responsive, entirely avoiding the "frozen browser" syndrome typical of synchronous web applications.

When a user attempts an action that is not permitted—such as attempting to upload an executable script file, or trying to delete a file in a folder where they only possess read-only viewing rights—the backend intercepts the failure instantaneously. Instead of crashing, timing out, or serving a raw HTTP 500 error page, the backend gracefully catches the violation and transmits a structured error code. The frontend translates this into a polite, localized popup message explaining precisely why the action was halted. This creates a secure, predictable environment where users feel guided by the system's guardrails rather than punished by ambiguous technical failures. Furthermore, the intelligent path resolver ensures that even if a network interruption occurs during a complex batch move operation, the system will never accidentally overwrite data in an unintended directory, preserving absolute data integrity regardless of the user's connection stability.

4. The Main User Interface

4.1 Technical Description

The Main User Interface module is a highly complex, state-driven Single Page Application (SPA) rendered entirely via client-side JavaScript (core.ui.main_explorer_ui.php, core.ui.toolbar_menues.php, and assets.js.core_engine.php). It operates independently of page reloads, relying on a central state object (myCloudState) that acts as the single source of truth for the current directory, selected files, view modes, and user preferences.

Dynamic Rendering Engine and View Modes The UI engine dynamically flushes and rebuilds the Document Object Model (DOM) based on the active state. It supports three distinct structural paradigms: a highly detailed tabular List view, a spatial Symbol/Grid view, and a media-centric Gallery interface. The myCloudRenderUI() function evaluates the current dataset and constructs the requisite HTML strings in memory before injecting them into the DOM container. To maintain high performance when rendering directories containing thousands of files, the engine implements an IntersectionObserver. This API monitors the viewport and defers the generation, fetching, and decryption of image thumbnails until the specific file row physically scrolls onto the user's screen, drastically reducing memory consumption and initial bandwidth overhead.

Context-Aware Toolbar and Ribbon Menus The primary navigation toolbar is fundamentally context-aware. Driven by the myCloudUpdateToolbarState() function, the ribbon continuously recalculates the availability of buttons based on the exact state of the selection array. For instance, if a user selects a single PDF file, the "Preview," "Rename," and "PDF Toolkit" buttons illuminate. If the user selects multiple files, the "Rename" and "Preview" buttons are dynamically disabled (or hidden, depending on user settings), while the "Stack PDFs" and "Batch Download" buttons become active. The toolbar supports two rendering modes: a flat, linear layout for smaller screens, and a categorized Ribbon layout that groups actions logically (e.g., "View", "Edit", "Selection", "Actions") via fly-out menus calculated utilizing getBoundingClientRect() to prevent off-screen clipping.

Spatial Navigation and Command Palette The interface features comprehensive keyboard accessibility designed to mimic native operating system environments. In the list view, the up and down arrow keys increment the

visualCursorIndex sequentially. However, in the grid/symbol view, the engine dynamically calculates the number of visible columns by dividing the container width by the individual item width. It then maps the up and down arrow keys to jump the index mathematically by the column count, resulting in true two-dimensional spatial navigation.

For advanced execution, the UI features a Command Palette initialized via the Ctrl+P shortcut. This injects a transient search modal into the DOM. As the user types, the engine performs a fuzzy-search algorithm against both a dictionary of system commands (e.g., "New Folder", "Empty Recycle Bin") and the cached array of loaded files, allowing users to execute actions or jump to files entirely without a mouse.

Interactive Marquee and Drag-and-Drop (DND) The UI implements a custom rubber-band selection tool (Marquee). When the user clicks and drags in the empty space of the file container, the myCloudInitMarquee() function generates a translucent absolute-positioned div. During the mousemove event, the engine calculates the Axis-Aligned Bounding Box (AABB) intersection between the marquee coordinates and every rendered file row's bounding rectangle. Rows that intersect are instantly pushed into the selectedFiles state array.

The Drag-and-Drop system utilizes the HTML5 DND API. The UI assigns draggable="true" to file rows. When a drag event initiates, the engine intercepts it, generates a custom floating avatar indicating the number of selected files (myCloudGetDragImage()), and serializes the selected file paths into the dataTransfer object. Drop zones are calculated dynamically, allowing users to drag files into subfolders, up the breadcrumb hierarchy, or across split panes.

Commander Mode (Dual-Pane Operation) For power users, the UI can mutate into "Commander Mode." The myCloudToggleCommander() function destroys the standard view and replaces it with two independent flexbox containers (commanderLeft and commanderRight) separated by a resizable DOM handle. The state object splits, maintaining isolated active directories, selections, and view modes for each pane. A middle toolbar is injected containing dedicated cross-pane operation buttons (Copy to Target, Move to Target, Sync), allowing for rapid, complex data migrations between disparate directory trees without traversing complex hierarchical menus.

4.2 The Workflow

The translation of user intent into visual UI changes follows a strict, unidirectional data flow:

- **Event Triggering:**
  - The user interacts with a DOM element (e.g., double-clicking a folder row, clicking a breadcrumb, or pressing an arrow key).
  - The specific event listener intercepts the action and calls a router function (e.g., myCloudHandleEnter()).
- **State Mutation:**
  - The router function updates the central myCloudState object. It modifies the currentDir, resets the selectedFiles array to empty, and resets the visualCursorIndex to zero.
- **Data Fetching (Asynchronous):**
  - The system invokes myCloudFetchDirectory().
  - A loading skeleton or spinner is injected into the UI.
  - An AJAX POST request is dispatched to the backend. If a previous request is still pending, an AbortController terminates it to prevent race conditions.
- **Data Reconciliation:**
  - The backend JSON response is received and parsed.
  - The global allItems memory array is purged of any stale references to the target directory and repopulated with the fresh metadata.
- **DOM Re-Rendering:**
  - myCloudRenderUI() is executed. The existing HTML container is wiped clean.
  - The rendering engine iterates through the filtered allItems array, building a new table or grid structure.
  - Tag components, encryption badges, and contextual hover actions are appended to each row.
- **Post-Render Hooks:**

- The IntersectionObserver is attached to the newly generated thumbnail placeholders.
- myCloudUpdateToolbarState() analyzes the new empty selection state and disables irrelevant action buttons.
- Focus is programmatically returned to the primary container to maintain uninterrupted keyboard navigation.

4.3 Usability

The user interface is meticulously crafted to eliminate the boundaries between a local desktop environment and the web browser. When a user logs in, they are not presented with a static web page, but a highly reactive workspace. If they want to select multiple files, they don't have to click individual checkboxes; they can simply click and drag their mouse across the empty space to draw a selection box over dozens of items, watching them instantly highlight. Holding the 'Shift' key while clicking files behaves exactly as it does on a native operating system, selecting a contiguous block of documents in one fluid motion.

The context-awareness of the interface ensures the user is never overwhelmed by irrelevant options. If they select a single PDF, the top ribbon intelligently brings forward tools specifically for PDFs—allowing them to split pages, merge documents, or extract text natively. If they select a JPEG image, the interface offers different options. Furthermore, the inclusion of organizational color tags allows users to categorize their workflow visually. A user can assign a custom label to the red tag, such as "Needs Signature," apply it to specific files, and then instantly filter the entire visible directory to show only items bearing that red tag via a simple dropdown menu.

For users undertaking heavy administrative tasks, standard folder navigation can become tedious. By clicking the "Commander" icon, the interface splits gracefully down the middle, transforming into a high-efficiency, dual-pane workstation. A user can open their "Archives" folder on the left pane and their "Active Projects" on the right. Using either the mouse or purely keyboard shortcuts, they can highlight files on the left and strike a single button in the middle to seamlessly migrate gigabytes of data across the structural hierarchy. Every aspect of the UI, from the instantaneous loading of directories to the tactile feedback of the drag-and-drop avatars, is engineered to provide absolute control and absolute clarity over the user's data landscape.

5. The Settings Dialog

5.1 Technical Description

The Settings Dialog module, primarily encapsulated within ui.modules.settings.php and supported by core engine scripts, represents a highly sophisticated state-management and configuration interface. It deviates from traditional monolithic settings pages by implementing a device-contextual configuration matrix, allowing the application state to mutate based on the physical hardware accessing it.

Device-Specific Configuration Matrix The core configuration is stored in a master JSON object, initiated on the client-side as myCloudDefaultSettings. This object is structurally partitioned into three distinct device archetypes: desktop, tablet, and phone. Each archetype contains an identical schema of configuration keys (e.g., treeOpen, darkMode, fontSize, singleClick, stackedToolbar, showCheckboxes, clickToPreview, symbolSize, commanderSplit), but with differing default boolean or integer values optimized for that specific form factor.

Upon initialization, the system executes the myCloudGetCurrentDeviceKey() function. This routine analyzes the navigator.userAgent, physical screen dimensions (window.innerWidth, window.innerHeight), and hardware capabilities (navigator.maxTouchPoints, CSS media queries for pointer coarseness) to classify the current hardware into one of the three archetypes. When the settings dialog is rendered, it binds specifically to the archetype currently in use, though the user can navigate to the other tabs to pre-configure the environment for alternative devices.

Dynamic DOM Generation and Tab Measurement The settings modal eschews static HTML in favor of dynamic DOM injection via the _cloudExRenderSettingsContent() function. Because the settings dialog contains tabs of vastly different vertical heights (e.g., the "Admin" tab versus the "Tags" tab), the engine performs a pre-computation sequence upon invocation. It temporarily renders the content of the three primary tabs in the background, extracts their respective offsetHeight metrics using the browser's layout engine, and stores the maximum value. It then applies this maximum height to the active wrapper, ensuring that switching between tabs utilizes a smooth CSS cubic-bezier transition without causing the modal's outer dimensions to jitter or snap violently.

Tag Management and Drag-and-Drop Reordering The "Tags" configuration tab implements a complex visual management system for color-coded organizational markers. It relies on three synchronized state arrays: tagNames (a dictionary mapping hex color codes to user-defined strings), visibleTags (an array of hex codes currently enabled for display), and tagOrder (the master sequence array). The UI allows users to dynamically reorder these tags utilizing the HTML5 Drag-and-Drop API.

When a tag row is dragged (ondragstart), the system stores its hex code in memory. As it is dropped (ondrop) onto a target row, the engine calculates the source index and destination index within the tagOrder array, performs an atomic splice() operation to extract and re-insert the color code, and then immediately forces a sort on the visibleTags array using masterOrder.indexOf(a) - masterOrder.indexOf(b) to guarantee structural integrity.

Cryptographic Password Management Interface The "General" tab features a password modification interface that enforces strict client-side security policies before communicating with the server. The ce_validate_my_pwd() function executes a series of regular expression evaluations, mandating a minimum length of 8 characters, the inclusion of specific special characters ([-_.:,;()/+#!§%&]), the inclusion of integers, and the strict exclusion of string-breaking characters ($, ", '').

Furthermore, the module integrates an asynchronous k-Anonymity breach detection sequence (ce_is_my_pwd_pwned). It calculates the SHA-1 hash of the proposed plaintext password using crypto.subtle.digest, extracts the first five hex characters, and transmits this prefix to the HaveIBeenPwned API via a highly concurrent fetch request. It then iterates through the returned hash suffixes to detect a collision. If a match occurs, the UI actively blocks the password change and throws a security exception.

State Synchronization and Persistence All interactions within the settings dialog—whether toggling a checkbox, dragging a slider, or modifying a text input—trigger an immediate, synchronous update to the myCloudState.settings object in Random Access Memory (RAM). Following the memory mutation, the myCloudApplySettings() function executes, translating the boolean states into physical DOM changes (e.g., applying CSS classes like ce-no-checkboxes or modifying the --font-size-base CSS variable). Simultaneously, a debounced AJAX POST request (myCloudSaveSettings()) serializes the entire JSON object and transmits it to the PHP backend, writing it to the user's localized profile configuration file.

5.2 The Workflow

The operational flow of the Settings Dialog is structured as follows:

- **Invocation and Measurement:**
  - User triggers myCloudShowSettings().
  - The engine resets modal states and invokes _cloudExRender-SettingsContent() sequentially for 'all', 'tags', and the active device key.
  - Maximum content height is calculated and applied to the wrapper's CSS inline styles to prevent layout shifts.
- **Rendering the Interface:**
  - The tab navigation row is constructed.
  - The active tab's configuration object is read from myCloud-State.settings.
  - HTML strings are concatenated using utility functions (render-Toggle, renderHeader), injecting specific data-key attributes bound to the JSON schema.
  - The final HTML payload is injected into innerHTML.
- **Event Binding:**
  - onchange listeners are attached to all checkboxes and select dropdowns.
  - oninput listeners are attached to range sliders (e.g., Font Size) for real-time visual feedback.
  - HTML5 Drag-and-Drop listeners (ondragstart, ondragover, ondrop) are bound to the Tag Management rows.
- **Interaction and Mutation:**
  - A user alters a setting (e.g., enabling "Single Click Mode").
  - The event listener captures the data-key and overwrites the specific value in the JavaScript state object.
  - If conditional logic applies (e.g., enabling "Single Click" forces "Show Checkboxes" to true), dependent variables are mutated synchronously.

- **Application and Synchronization:**
    - myCloudApplySettings() fires, injecting or removing CSS classes across the body and myCloudContainer nodes.
    - A timeout debounce ensures that rapid toggling does not flood the server.
    - myCloudSaveSettings() executes a fetch POST request, transmitting the stringified JSON payload to the backend for persistent disk storage.

5.3 Usability

From the user's perspective, the settings dialog acts as a real-time control center that immediately reflects their preferences without ever requiring a disruptive page refresh. Upon opening the dialog, the user is presented with a clean, tabbed layout. The system intelligently detects whether they are using a desktop, tablet, or smartphone, and places a small "Current Device" indicator next to the relevant tab. This allows a user to tailor their desktop experience—enabling dense list views, a persistent navigation tree, and a complex ribbon toolbar—while separately configuring their mobile experience to utilize large, touch-friendly icons and simplified menus.

The immediacy of the interface is its defining characteristic. If a user adjusts the "Font Size" slider, they watch the text across the entire application grow or shrink dynamically in the background while the slider moves. Toggling "Dark Mode" instantly swaps the application's color palette.

In the Tags menu, the user experiences a highly tactile organizational tool. They can click on the default "Red" tag and rename it to "Urgent Documents". If they use this tag frequently, they can simply click the drag-handle icon, physically drag the red tag to the top of the list, and drop it. The interface instantly reorders itself, ensuring that "Urgent Documents" will now appear first in all context menus across the application.

When managing security, the user is guided by strict but informative guardrails. Attempting to change a password invokes an inline validation engine. If the user types a weak password, the system immediately halts the process and lists exactly which security parameters are missing. If they attempt to use a password known to have been compromised in a public data breach, the system performs a split-second check and blocks the action, politely informing the user that their chosen password is unsafe. This creates an

environment where users feel firmly in control of their workspace aesthetics while being seamlessly protected by rigorous security protocols.

6. The Multilingual Interface

6.1 Technical Description

The Internationalization (i18n) module, primarily driven by core.i18n.php, provides a lightweight, high-performance translation matrix that evaluates user preferences, browser environments, and fallback mechanisms to render the application in one of twenty supported languages.

Language Resolution Hierarchy The module operates outside of heavy database queries, relying on a localized file-system approach. The resolution engine follows a strict order of operations to establish the $detectedLang variable. First, it probes the user's saved configuration profile (stored as a JSON blob on the server). If a language key exists and matches an index within the hardcoded $available_languages array, this value is selected.

If no profile exists (e.g., during the initial login phase), the engine interrogates the $_SERVER['HTTP_ACCEPT_LANGUAGE'] header transmitted by the client's browser. Because this header typically contains a complex, comma-separated string with quality factors (e.g., de-DE,de;q=0.9,en;q=0.8), the engine explodes the string, strips the quality weights, and extracts the primary two-letter ISO 639-1 code. It iterates through these codes sequentially until it finds a match within the supported dictionary. If all resolution attempts fail, the system enforces a hardcoded fallback to en (English).

Dictionary Merging and Missing Key Mitigation To prevent application failure or the rendering of blank UI elements due to incomplete translation files, the module employs a dual-load merging strategy. First, it unconditionally loads the English dictionary array ($baseArray). Subsequently, it loads the dictionary array corresponding to the $detectedLang ($targetArray). The engine then executes array_merge($baseArray, $targetArray). Due to the behavior of PHP's array_merge on associative arrays, keys present in the target language overwrite the English keys, but any keys missing from the target translation seamlessly fall back to their English definitions.

Right-to-Left (RTL) Layout Engine The i18n module extends beyond string translation to dictate structural DOM layout. The backend defines an array of RTL language codes (['ar', 'fa', 'he', 'ur']). If the active $language intersects with this array, the PHP engine injects dir="rtl" into the root HTML container. The application's CSS framework utilizes logical properties (margin-inline-start, padding-inline-end) rather than physical properties (margin-left, padding-right). Consequently, the dir="rtl" attribute natively forces the browser's

rendering engine to mirror the entire flexbox hierarchy, moving the navigation tree to the right and reversing the flow of breadcrumbs and toolbars without requiring a separate stylesheet.

6.2 The Workflow

- **Request Interception:** The PHP thread initiates core.i18n.php prior to generating any HTML output.
- **Profile Evaluation:** The system attempts to read the user's localized JSON settings file to extract a saved language preference.
- **Header Fallback:** If the profile is absent, the engine parses HTTP_ACCEPT_LANGUAGE, extracting and validating the primary ISO code against the supported list.
- **Dictionary Instantiation:**
    - The baseline English file (lang/en.php) is loaded into memory.
    - The target language file is loaded.
    - The arrays are merged, guaranteeing 100% key coverage.
- **DOM Integration:** * The unified dictionary array is encoded via json_encode() and injected into the HTML <head> as the window.myCloud_LANG JavaScript object.
    - The dir attribute is calculated and applied to the <body> or root .
- **Client-Side Hot Swapping:** If a user changes the language via the settings dropdown, an AJAX POST triggers actionSwitchLanguage(). The backend recalculates the dictionary and returns the new JSON payload. The frontend instantly overwrites window.myCloud_LANG, updates the dir attribute, and invokes a total UI re-render, translating the active view instantly.

6.3 Usability

The application is designed to be globally accessible with zero configuration required by the end-user. When an international user accesses the login page for the first time, the system silently interrogates their browser settings and immediately presents the interface in their native tongue. If the user speaks Arabic, Hebrew, or Farsi, the system doesn't merely translate the text; it mirrors the entire physical layout of the application. The sidebar shifts

to the right, icons flip their orientation, and text aligns naturally, respecting the fundamental reading mechanics of RTL languages.

If a user wishes to override this automatic detection, they can navigate to the settings dialog and select from a dropdown list of 20 languages—ranging from standard Spanish and Japanese to highly localized dialects like Bairisch or Nigerian Pidgin. Selecting a new language is a frictionless experience; the moment a new option is clicked from the dropdown, the entire application interface translates itself instantly before their eyes, without requiring a page reload or disrupting any active file uploads or background tasks. Furthermore, because of the robust fallback system, if a newly added feature lacks a translation in a specific regional dialect, the user will simply see that specific button in English, ensuring the application remains completely usable and stable rather than throwing an error or displaying blank text.

7. The End-to-End Encryption

7.1 Technical Description

The End-to-End Encryption (E2EE) module (assets.js.crypto_engine.php and corresponding backend hooks) implements a zero-knowledge, mathematically secure cryptographic boundary operating entirely within the client's browser. It utilizes the native WebCrypto API to ensure that plaintext data, encryption keys, and unencrypted filenames never traverse the network.

Key Derivation and Envelope Architecture The architecture is built upon a Key Encryption Key (KEK) and Data Encryption Key (DEK) envelope model. When a user creates a secure vault, they supply a master password. The WebCrypto engine generates a 32-byte cryptographically secure random salt. The deriveKekFromPassword function utilizes the Password-Based Key Derivation Function 2 (PBKDF2) algorithm, combining the password and salt through 600,000 iterations of the SHA-512 hashing algorithm to derive the KEK. This immense iteration count acts as an aggressive countermeasure against offline dictionary and GPU-based brute-force attacks.

Simultaneously, a 256-bit AES-GCM DEK is generated. The KEK is used exclusively to encrypt ("wrap") this DEK. The system packages the salt, the wrapping Initialization Vector (IV), and the wrapped DEK into a JSON payload. This payload is transmitted to the server and stored as a .mycloud_crypto_salt file within the target directory, marking it as a cryptographic root.

Session Master Key and RAM Isolation To maintain usability across an active session without requiring the user to re-enter their password for every file access, the DEK must be kept in memory. However, leaving raw key material in JavaScript variables is vulnerable to XSS extraction. To mitigate this, upon login, the system generates an ephemeral "Session Master Key" via crypto.getRandomValues, storing the raw bytes in the browser's isolated sessionStorage.

When a user unlocks a vault, the DEK is unwrapped using their password-derived KEK. Immediately, the raw DEK is re-wrapped using the Session Master Key and stored in the wrappedDirKeys memory dictionary. During subsequent file operations, the DEK is dynamically unwrapped for the exact millisecond required to perform the AES operation and is instantly destroyed by JavaScript garbage collection, severely minimizing the exposure window of raw key material.

File and Filename Ciphertext Structuring All file encryptions utilize the Advanced Encryption Standard in Galois/Counter Mode (AES-256-GCM), ensuring both data confidentiality and authenticity (tamper evidence). When encrypting a file (encryptFile), a unique 12-byte IV is generated. The resulting binary Blob is constructed with a strict header format: a 16-byte empty padding space (reserved for legacy salt compatibility), followed by the 12-byte IV, followed by the ciphertext and the 16-byte GCM authentication tag.

Filenames are encrypted independently (encryptName) to obscure directory contents from server administrators. The plaintext filename is encrypted via AES-GCM. The unique 12-byte IV is prepended to the ciphertext, the entire byte array is encoded using a URL-safe Base64 transformation, and a .enc extension is appended.

7.2 The Workflow

- **Vault Initialization:**
    - User selects an empty directory and invokes the "Encrypt" action, providing a password.
    - WebCrypto generates the DEK, salt, and KEK. The DEK is wrapped.
    - The frontend transmits the JSON payload to the backend, which saves it as .mycloud_crypto_salt.
    - The myCloudMigrateDirectory function recursively traverses the folder, downloading any existing files, encrypting their contents and names, uploading the encrypted Blobs, and permanently deleting the plaintext originals.
- **Vault Unlocking:**
    - User selects a locked vault. The backend transmits the JSON payload from the salt file.
    - User inputs their password. The KEK is derived.
    - The system attempts to unwrapKey the DEK. If the password is wrong, the GCM authentication tag fails, and the operation throws an exception.

- Upon success, the DEK is wrapped with the Session Master Key and mapped to the directory path in RAM. The UI re-renders, displaying decrypted filenames.

- **File Decryption (Access):**
  - User requests to view/download an encrypted file.
  - The backend streams the raw ciphertext Blob to the client.
  - The client extracts the IV from the Blob's byte array header.
  - The DEK is unwrapped from the session memory.
  - crypto.subtle.decrypt processes the payload, verifying integrity and yielding a plaintext Blob in RAM.
  - The Blob is piped to a local Object URL for previewing or passed to the native File System Access API for direct-to-disk downloading.

- **Cross-Boundary E2E Transfers:**
  - If a user moves a file *between* two different vaults, or from a vault to a plaintext folder, the myCloudE2ETransfer orchestrator intercepts the action.
  - It downloads the source file, decrypts it in RAM, re-encrypts it using the destination vault's specific DEK (or leaves it plaintext), uploads the new Blob, and deletes the source, ensuring data is never decrypted on the server.

7.3 Usability

The implementation of End-to-End Encryption is designed to be mathematically uncompromising yet entirely transparent to the daily workflow of the user. When a user decides that a specific folder contains highly sensitive legal or financial documents, they simply right-click the folder and select "Encrypt." They are prompted to create a strong password. From that moment on, the folder transforms into a secure vault. To the server infrastructure, network administrators, or potential attackers, the contents of that folder become completely opaque—a meaningless collection of random characters with scrambled, unreadable filenames.

When the authorized user logs into the application, they see a "Locked" badge next to their vault. By clicking it and entering their password, the vault

seamlessly unlocks. The scrambled filenames instantly revert to their normal, readable text. The beauty of the system is its frictionlessness post-unlock: the user can double-click a confidential PDF, and it opens in the browser's previewer just as quickly as a normal file. They can drag and drop new photographs into the vault, and the application silently encrypts them in the background before they are uploaded to the cloud.

The system intelligently handles complex operations to protect the user's data integrity. If a user attempts to move an encrypted file out of the vault into a standard, unencrypted folder, the application doesn't simply move the locked file. Instead, it intercepts the action, displays a progress bar indicating "E2E Transfer in Progress," silently decrypts the file in the computer's memory, and uploads a readable copy to the new destination. This ensures that the user doesn't accidentally lock themselves out of their own files, providing the ultimate guarantee of privacy without sacrificing the intuitive, drag-and-drop simplicity expected from modern cloud storage platforms.

8. The Grid View

8.1 Technical Description

The Grid View (internally referred to as the Symbol View) is a spatial, icon-centric rendering mode managed primarily by the myCloudRenderSymbols() function within ui.views.icon_view.php. Unlike the List View, which relies on standard HTML <table> elements, the Grid View constructs a responsive CSS Flexbox/Grid architecture (.myCloud-symbol-grid) that dynamically reflows items based on the viewport width and the user-defined symbolSize configuration (small, medium, large, xlarge).

Asynchronous Thumbnail Queue Management To prevent network starvation when a user navigates into a directory containing thousands of images, the Grid View implements a highly optimized, parallelized request manager: myCloudThumbQueue. Browsers typically limit concurrent connections to a single domain (historically 6 for HTTP/1.1). If the UI blindly requested 500 thumbnails simultaneously, the browser would stall, delaying critical API requests (like folder navigation or chunk uploads). The myCloudThumbQueue strictly limits active thumbnail requests to its maxConcurrent property (set to 6). It operates a First-In-First-Out (FIFO) queue that continuously polls itself via the process() method. As one thumbnail finishes downloading (or fails), the active counter decrements, and the next item in the queue is immediately dispatched.

Intersection Observer Integration The queue is fed exclusively by the symbolThumbObserver, an instance of the native IntersectionObserver API. As the grid is constructed, the engine assigns this observer to every image placeholder (.ce-sym-icon-box). The observer is configured with a rootMargin of "200px", meaning it detects elements just before they enter the physical viewport. When an element intersects, the observer extracts its data attributes, unobserves the element to save CPU cycles, and pushes the path into the myCloudThumbQueue.

Fast-Path vs. Token-Based Resolution When processing a thumbnail, the queue engine attempts a "Fast Path" resolution first via myCloudGetFastThumbUrl(). This generates a URL containing a Base64-encoded file path and the CSRF token. The backend directly intercepts this specific parameter, bypassing session initialization locks to stream the JPEG bytes instantly using readfile(). However, if the file is End-to-End Encrypted (E2EE) or resides inside a virtualized ZIP container, the Fast Path is skipped. Instead,

the engine executes a standard asynchronous fetch to obtain a short-lived download token, downloads the encrypted Blob, decrypts it in RAM utilizing myCloudCrypto.decryptFile, creates an ephemeral Blob URL (URL.createObjectURL), and attaches it to the DOM, ensuring zero-knowledge privacy is maintained even for thumbnail previews.

8.2 The Workflow

The lifecycle of rendering the Grid View proceeds through the following computational steps:

1. **State Preparation and DOM Clearing:**
    - The myCloudRenderSymbols function is invoked.
    - The myCloudThumbQueue is immediately purged (clear()) to abort any pending thumbnail downloads from the previously viewed directory.
    - The primary DOM container is emptied, and the new .myCloud-symbol-grid element is instantiated.
2. **Item Sorting and Filtering:**
    - The central state array (allItems) is filtered to extract only the items belonging to the currentDir.
    - The items are sorted based on the global sort parameters (e.g., alphabetically by name, or numerically by size), placing directories before files.
3. **Tile Generation:**
    - The engine iterates through the sorted array.
    - For each item, a .myCloud-symbol-item is constructed.
    - The system determines the file extension. If it is an image, a blank placeholder is created and passed to the symbolThumbObserver. If it is a document, directory, or unrecognized file, a statically mapped, lightweight SVG icon is injected directly.
    - Cryptographic badges (Lock/Unlock icons) and organizational color tags are calculated and appended via absolute positioning over the tile.

4. **Interaction Binding:**
    - Click, double-click, and context-menu event listeners are bound to the tile.
    - HTML5 Drag-and-Drop event listeners (dragstart, dragover, drop) are attached, serializing the selection state into the dataTransfer payload.
5. **Observation and Execution:**
    - The newly constructed grid is appended to the main document body.
    - The browser calculates the layout. As tiles enter the 200px threshold of the viewport, the IntersectionObserver fires.
    - Tiles are pushed to the myCloudThumbQueue, which dispatches up to 6 concurrent network requests to fetch the actual image data, replacing the SVG placeholders with <img src="..."> elements as the data arrives.

8.3 Usability

When a user switches to the Grid View, the application transforms from a data-heavy ledger into a highly visual, spatial workspace. Folders and files are represented as large, easily identifiable tiles that automatically wrap and resize to fill the available screen real estate perfectly. For users managing photography, design assets, or marketing materials, this view is indispensable.

As the user rapidly scrolls down a folder containing thousands of high-resolution images, the interface remains perfectly smooth and responsive. The application intelligently waits to download the actual image previews until they are just about to appear on the screen. Because of the parallel queue system, the user never experiences the "broken image" phenomenon typical of overloaded web servers; thumbnails pop into existence fluidly.

Interaction within the grid feels inherently native. A user can click a single tile to select it, or hold the 'Ctrl' (or 'Cmd') key to selectively highlight multiple disparate tiles across the grid. The tiles provide immediate visual feedback, changing their background color and illuminating a small checkbox in the corner. If the user wants to reorganize their workspace, they simply click and hold a selected tile, drag it across the screen, and drop it directly onto a

folder tile. The folder tile visually highlights to indicate it is a valid drop target, and upon release, the files are instantly moved into the subfolder, exactly as one would expect when operating a native desktop computer system like Windows Explorer or macOS Finder.

---

9. The Gallery View

9.1 Technical Description

The Gallery View is an augmented derivation of the Grid View, designed specifically for immersive media consumption. It is activated either by a distinct user setting (symbolDarkMode: true) or when the global interface mode is explicitly set to gallery. When active, it fundamentally alters the CSS environment and overrides standard interaction paradigms to prioritize visual content over file management mechanics.

Multiselect State Interception In the standard Grid View, every tile features a persistent checkbox for selection. In the Gallery View, these checkboxes are hidden via CSS to eliminate visual clutter. To allow users to manage files without checkboxes, the view implements an aggressive state-interception engine: toggleSymbolMultiselect(active).

When a user wishes to select multiple files, they trigger this mode (often via a long-press on touch devices). The engine dynamically injects the .multiselect-mode class into the grid container, which unhides the checkboxes. Concurrently, it summons the myCloudSymbolActionToast—a persistent, floating action bar pinned to the bottom of the viewport containing quick-actions (Share, Copy, Move, Delete). To ensure state integrity, a global click listener is attached to the document body. If the engine detects a click outside of the grid or the toast bar, it instantly tears down the multiselect state, hiding the checkboxes and destroying the floating toast to return the user to the immersive viewing mode.

Filmstrip Carousel Architecture A defining technical feature of the Gallery View is the myCloudInitFilmstrip() integration. When a user previews an image, the Gallery View dynamically generates a horizontal carousel of all sibling images residing in the same directory, appending it to the bottom of the preview overlay.

The filmstrip logic scans the in-memory allItems array, isolating files with image extensions. It constructs miniature <img> nodes for each and assigns

them to a dedicated myCloudFilmstripObserver. This secondary IntersectionObserver handles lazy-loading for the carousel independently of the main grid. When the user navigates between images in the primary previewer, the filmstrip's active state updates, and the engine invokes scrollIntoView({ behavior: 'smooth', inline: 'center' }). This forces the browser's composite engine to smoothly translate the carousel horizontally, keeping the currently viewed image perfectly centered in the filmstrip.

EXIF Metadata Extraction The Gallery View features an integrated metadata parser for photography workflows. When a user triggers the info panel, the system executes myCloudShowExif(). For standard files, it issues a POST request to the backend, which utilizes PHP's native exif_read_data() function to parse the JPEG/TIFF headers.

However, if the image is End-to-End Encrypted, the backend cannot read the EXIF data, as the file is opaque ciphertext. In this scenario, the frontend dynamically imports the exif-js library. It locates the decrypted Blob URL currently residing in the myCloudState.previewCache, fetches the raw binary Blob from the browser's memory, and parses the Application Segments (APP1) of the JPEG locally. The engine then formats this data (calculating F-Stops, Exposure Times, and converting GPS coordinates from Degrees/Minutes/Seconds arrays into actionable decimal format) and injects it into a floating .myCloud-exif-modal DOM node.

9.2 The Workflow

- **Activation and Context Shift:**
    - The UI detects the gallery interface flag.
    - Dark mode CSS classes (.symbol-dark-container) are appended to the main container, turning the background deeply dark or black.
    - Toolbar visibility is suppressed to maximize vertical screen space.
- **Interaction Routing:**
    - The standard onclick handlers on tiles are overridden. Instead of highlighting the tile, a single click directly invokes myCloudHandleEnter(), immediately launching the full-screen media previewer.

    - The oncontextmenu (right-click) listener is modified. If the grid is in standard mode, right-clicking a tile selects it and opens the context menu. If in multiselect-mode, right-clicking is suppressed to prioritize batch operations via the floating toast.

- **Media Consumption (Filmstrip):**
    - Upon launching an image, myCloudInitFilmstrip() queries the state array for all adjacent images.
    - The filmstrip DOM container is generated and appended to the preview overlay.
    - The user clicks an image in the filmstrip; the onclick handler fires myCloudDownloadFile(), replacing the primary high-resolution preview image while maintaining the overlay state.
- **Metadata Querying:**
    - The user taps the "Info" icon in the previewer.
    - The engine determines if the file is encrypted.
    - It routes the EXIF extraction task either to the backend API or the local exif-js processor.
    - The parsed data is formatted into an HTML table and displayed over the image.

9.3 Usability

The Gallery View is engineered for environments where visual impact is paramount. When a user enters this mode, the traditional file-manager aesthetic vanishes. The interface turns dark, the toolbars recede, and the screen is dedicated entirely to large, edge-to-edge media tiles. There are no distracting checkboxes or data columns. If a user wants to view a photograph, they do not have to double-click it; a single, effortless click instantly expands the image to fill the entire monitor.

While viewing an expanded image, the user's workflow is vastly accelerated by the Filmstrip. Situated at the bottom of the screen, the filmstrip provides a continuous, scrolling carousel of all other images in that specific folder. The user can rapidly click through the thumbnails in the filmstrip to review a photoshoot or a batch of design mockups, with the main screen instantly updating to show the high-resolution version.

Despite this simplified, media-first approach, the Gallery View retains immense power under the hood. If a user needs to perform bulk operations, they simply right-click an image (or long-press on a touchscreen). Instantly, the interface adapts: selection outlines appear, checkboxes fade into view, and a sleek action bar rises from the bottom of the screen. The user can then rapidly tap multiple images and hit the "Share" or "Download" button on the action bar. Furthermore, photographers can instantly access critical camera data. By clicking the "Info" button while viewing a picture, a sleek overlay appears detailing the exact camera model, lens aperture, ISO, shutter speed, and even a direct, clickable link to Google Maps if the photo contains embedded GPS location data.

10. The Office View

10.1 Technical Description

The Office View module (ui.views.office_view.php) represents a radical paradigm shift within the application, transforming the standard file explorer into a split-pane, high-productivity document management system. It fundamentally relies on dynamic DOM restructuring and tight integration with internal rendering engines and external document servers.

Layout Orchestration and Split-Pane Architecture When activated via myCloudToggleOffice(), the engine modifies the myCloudBody flex container. It injects a .myCloudResizer (a draggable DOM handle) and a .myCloudPreviewPane element adjacent to the standard file list.

The myCloudInitOfficeResizer() function attaches mousedown, mousemove, and mouseup event listeners to the resizer handle. To prevent erratic behavior caused by iframes (which frequently capture and swallow mouse events during drag operations), the engine temporarily applies pointer-events: none to the preview pane while dragging is active. The width is calculated dynamically by subtracting the e.pageX delta from the initial width, clamping the minimum width to 200px and the maximum width to 70% of the viewport. Upon releasing the mouse, the final calculated width is asynchronously saved to the user's device-specific profile (officePreviewWidth) to persist across sessions.

Document Rendering Routing (myCloudUpdateOfficePreview) When a user selects a file in the adjacent list, the system evaluates the file extension. Rather than opening a full-screen modal, it routes the rendering logic directly into the newly created preview pane (myCloudRenderInlineHtml).

- **Images/Media:** Rendered natively using HTML5 <img src="..."\>, <video>, or <audio> tags.
- **XLSX (Excel):** If an Excel file is selected, the system dynamically imports the SheetJS library from a CDN. It fetches the file as an ArrayBuffer, parses the spreadsheet, and utilizes XLSX.utils.sheet_to_html to generate a lightweight, read-only HTML table representation of the data, complete with clickable tabs to switch between worksheets.
- **DOCX (Word):** The system dynamically imports docx-preview.js. It fetches the document Blob and renders it into a scaled container (#docx_zoom_wrapper). The engine calculates the optimal CSS

transform: scale() factor to ensure the document fits the variable width of the split pane perfectly, recalculating on every window resize event.

- **EPUB (eBooks):** Utilizing epub.js, the system mounts a continuous, scrolled reading view. It actively parses the EPUB manifest to build an interactive Table of Contents overlay and injects CSS rules directly into the epub.js rendition themes to support specialized footnote styling and dark-mode compatibility.
- **PDFs:** On desktop browsers, PDFs are loaded into an <iframe> utilizing the browser's native, highly optimized PDF engine. However, on mobile devices (where native PDF iframes often fail or force downloads), the system falls back to a custom implementation of pdf.js, generating high-resolution <canvas> elements for each page and utilizing an IntersectionObserver to lazy-load pages as the user scrolls.

Advanced PDF Toolkit and Orchestration The Office View is deeply integrated with a comprehensive PDF manipulation toolkit, interacting heavily with backend binaries (qpdf, ghostscript, pdftk). The most complex feature is the Multi-Print/Stack Orchestrator (myCloudAction_PdfStackMenu). If a user selects multiple disparate files (e.g., two PDFs, a DOCX, and an XLSX file) and chooses "Stack PDFs," the frontend orchestrates a massive asynchronous batch process. It iterates through the selection, pushing any Office documents to the backend for temporary PDF conversion (office_convert_temp_pdf). Once all files are normalized to PDF format, it transmits the array of temporary and original PDF paths to the backend pdf_stack endpoint, which utilizes qpdf --empty --pages to merge them into a single, cohesive document. The frontend then automatically issues a download token for the resulting merged file or triggers a seamless background print sequence via a hidden iframe.

WYSIWYG PDF Form Filling If a user selects "Fill Form" on a PDF, the system launches myCloudAction_PdfStartFormFill. It utilizes pdf.js to render the PDF pages onto background canvases. It then calls page.getAnnotations() to extract AcroForm widget data. For every detected text field or checkbox, the JavaScript engine calculates the precise X/Y viewport coordinates and absolute dimensions. It dynamically constructs HTML <input> or <textarea> elements, styles them with a translucent blue background, and absolutely positions them precisely over the painted canvas fields. When the user clicks "Save," the system scrapes the values from these HTML

inputs, serializes them into a JSON dictionary, and transmits them to the backend, which generates an XFDF XML file and uses pdftk to permanently burn the data into the PDF structure.

10.2 The Workflow

- **Activation:**
    - User clicks the "Office View" icon in the top toolbar.
    - myCloudToggleOffice fires, injecting the preview pane and resizing the central file tree/list.
- **Inline Previewing:**
    - User single-clicks a file in the list.
    - myCloudUpdateOfficePreview halts any previous rendering tasks and clears the pane.
    - An asynchronous fetch retrieves the secure download token.
    - The file is evaluated. If it is an Office document (DOCX/XLSX), external parsing libraries are lazy-loaded. The raw file buffer is downloaded, parsed locally by the CPU, and injected as raw HTML/Canvas into the pane.
- **PDF Merging (Stacking):**
    - User selects multiple compatible files and clicks "Stack PDFs".
    - myCloudShowPdfMergeDialog renders an HTML5 drag-and-drop sortable list, allowing the user to dictate the exact page order.
    - Upon execution, the frontend tracks conversion progress, displaying a live percentage bar.
    - The backend merges the files and returns a unified token, which the frontend uses to download or preview the final compilation.

10.3 Usability

The Office View is the ultimate productivity mode for professionals who need to process vast amounts of documents rapidly. When activated, the screen splits. On the left, the user retains full access to their folder structures and file lists; on the right, a persistent viewing pane appears. As the user clicks down the list of files on the left, the right pane instantly updates, rendering

the contents of Word documents, Excel spreadsheets, PDFs, and images natively within the browser. The user no longer has to download files, open external applications, and close them just to see what a document contains; they can review dozens of files in seconds simply by pressing the down arrow key.

The true power of this view lies in its PDF management capabilities. If a user needs to combine a contract in PDF format with an invoice in Excel format, they simply select both files in the list and click "Stack PDFs." A dialog appears allowing them to drag and drop the files into the correct order. The system handles all the heavy lifting—silently converting the Excel file into a PDF in the background and fusing it with the contract.

Furthermore, dealing with bureaucratic forms is effortless. If a user receives a fillable PDF form, they click the "Fill Form" tool. The PDF opens in a clean, full-screen interface where all the blank fields are highlighted in a soft blue. The user can click directly into these fields, type their information, check boxes, and click "Save." The system instantly generates a finalized, flattened PDF document ready to be emailed or archived, eliminating the need for expensive third-party PDF editing software. Every tool, from rotating pages to extracting text via Optical Character Recognition (OCR), is embedded directly into the workspace, turning the browser into a fully equipped digital office.

11. The Preview

11.1 Technical Description

The Preview module, fundamentally governed by ui.modules.preview.php and interacting deeply with the core state engine, is a sophisticated, multi-modal rendering environment designed to visualize a vast array of file formats without downloading them to the user's local persistent storage. It operates as a transient, full-viewport DOM overlay (myCloudPreviewOverlay) that intercepts standard navigation logic and replaces it with an immersive, sandboxed viewing context.

The architectural foundation of the Preview module is its dynamic content routing system. When the myCloudOpenPreview() function is invoked, it evaluates the target file's extension against predefined configuration arrays (myCloudConfig.image, video, audio, office, binary). Rather than utilizing a monolithic rendering engine, the application employs a highly modular, lazy-loaded dependency injection model. Core libraries are not loaded during the initial application bootstrap to preserve bandwidth; instead, when a specific filetype is requested, the myCloudLoadJS() helper dynamically appends <script> tags to the document head, resolving Promises only when the external libraries (e.g., SheetJS, epub.js, Leaflet.js, pdf.js) are fully parsed and executable by the browser's JavaScript engine.

For image rendering, the system constructs a complex affine transformation matrix (window.myCloudTransform). This matrix tracks scale, X/Y translation, rotation degrees, and horizontal/vertical flip states. On desktop environments, the myCloudOnWheel event listener intercepts mouse wheel deltas, calculating the exact relative X/Y coordinates of the cursor over the image bounding box. It applies a scaling multiplier and translates the image simultaneously, ensuring the specific pixel under the user's cursor remains stationary while the image magnifies. On touch devices, the myCloudOnTouchStart and myCloudOnTouchMove listeners track multiple touch points. By calculating the hypotenuse between two clientX/clientY coordinates (Math.hypot), the engine derives a pinch-to-zoom scale factor. Furthermore, the touch engine implements a swipe-to-dismiss threshold; if a single-finger downward translation exceeds 15% of the viewport height (window.innerHeight * 0.15), the engine triggers a CSS scale-down animation and destroys the overlay.

To manage high-resolution photography efficiently, the module implements a client-side High-Definition (HD) to Standard-Definition (SD) downsampling routine. When navigating through multi-megabyte images, especially within an End-to-End Encrypted (E2EE) vault where server-side thumbnail generation is mathematically impossible, transferring and decoding full-resolution ciphertext for every image would exhaust browser memory. The myCloudToggleQuality() function intercepts the decrypted RAM Blob, paints it onto an invisible, off-screen HTML5 <canvas> element constrained to a maximum dimension of 1024 pixels, and exports a highly compressed JPEG Blob. This secondary Blob is cached in the myCloudState.previewCache alongside the original, allowing the user to toggle between rapid browsing and pixel-peeping instantly.

The module implements highly specialized rendering pathways for complex filetypes. For instance, EPUB electronic books are processed via epub.js, which parses the internal XML manifest to generate a continuous, scrolled DOM structure. The engine recursively extracts the Table of Contents (ToC) into an interactive sliding drawer and utilizes Canonical Fragment Identifiers (CFI) to map search query highlights directly onto the rendered text nodes. For geographic data (KML/KMZ), the system unpacks the XML coordinate data, dynamically loads the Leaflet.js mapping library, and overlays the vector paths onto an interactive OpenStreetMap tile grid. Electronic mail (EML) files are parsed using the postal-mime library. Because EML files often contain embedded tracking pixels and malicious external CSS, the Preview module constructs a heavily fortified <iframe>. It utilizes the srcdoc attribute combined with a strict Content-Security-Policy (<meta http-equiv="Content-Security-Policy" content="default-src 'none'; style-src 'unsafe-inline'; img-src data: blob: cid:;">) to render the email body natively while absolutely preventing any external network requests or script execution.

PDF rendering highlights the module's device-aware adaptive capabilities. On desktop operating systems, the module relies on the browser's native PDF viewing plugin via a standard <iframe>, appending hash parameters (#view=FitH&toolbar=1&navpanes=0) to force an optimal initial layout. However, mobile operating systems frequently lack robust native iframe PDF support, often forcing the file to download unexpectedly. To circumvent this, the Preview module detects mobile User-Agents and seamlessly diverts the workflow to a local instance of Mozilla's pdf.js. This engine parses the PDF binary in a dedicated Web Worker thread. The main thread calculates the

mobile viewport width, establishes the necessary scaling factor for crisp text rendering given the device's pixel ratio (window.devicePixelRatio), and paints the document page-by-page onto sequential HTML <canvas> elements. To prevent memory exhaustion on massive documents, the engine utilizes an IntersectionObserver to aggressively allocate and deallocate canvas memory based on which pages are currently intersecting the user's visible scroll area.

To eliminate latency during sequential media consumption, the module operates a background prefetching engine (myCloudPrefetchNavTokens). Whenever an item is previewed, the system interrogates the actively sorted allItems array to determine the next and previous contiguous files. It silently dispatches fetch requests to the backend to secure temporary download tokens and, if the directory is encrypted, downloads and decrypts the adjacent Blobs into RAM before the user ever clicks the "Next" button, ensuring near-instantaneous transitions.

11.2 The Workflow

The lifecycle of the Preview module follows this precise operational sequence:

- **Invocation and Overlay Instantiation:**
    - A user interacts with a previewable file (via single-click in Gallery/Office mode, or a specific toolbar action).
    - myCloudOpenPreview() executes, creating the .myCloudPreviewOverlay DOM element if it does not exist, and injecting it at the highest z-index.
    - Touch, mouse, and keyboard listeners (Escape to close, Arrow keys for navigation) are bound to the overlay.
- **Path Resolution and Cache Interrogation:**
    - The engine checks myCloudState.previewCache using the file's absolute path and requested quality tier (_hd or _sd).
    - If cached, the execution skips to the rendering phase.
- **Cryptographic and Network Fetching:**
    - If un-cached, an AJAX POST request secures a short-lived download token from the backend.

    - If the file resides within an E2EE vault, the application halts the UI, displays a "Decrypting..." spinner, fetches the ciphertext Blob, unwraps the Data Encryption Key (DEK) from session memory, decrypts the payload via AES-256-GCM, and generates an ephemeral Object URL.

- **Filetype Evaluation and Dependency Injection:**
    - The file extension is parsed.
    - If external dependencies are required (e.g., SheetJS for XLSX, docx-preview for Word, Leaflet for KML), myCloudLoadJS validates their presence and injects them asynchronously.
- **Content Rendering:**
    - **Images/Video/Audio:** Direct injection into native HTML5 media tags.
    - **Text/Code:** The Blob is converted to a text string, HTML entities are escaped to prevent XSS, and injected into a <pre> block.
    - **Office/Specialty:** The respective JavaScript engine parses the ArrayBuffer and constructs the localized DOM elements within the preview container.
- **Post-Render Hooks:**
    - Navigation arrows are injected if adjacent previewable files exist in the current directory state.
    - The myCloudPrefetchNavTokens function is invoked silently in the background.
    - The IntersectionObserver for the Filmstrip (if active) is updated to center the currently viewed item.
- **Teardown:**
    - Upon closure (via click, Escape key, or swipe-down), CSS transition classes are applied.
    - Event listeners are explicitly detached to prevent memory leaks.
    - Ephemeral Object URLs are revoked to free browser RAM.

11.3 Usability

The Preview module is designed to eliminate the friction between locating a file and actually consuming its contents. In traditional file management systems, viewing a document requires downloading it, locating it in a local downloads folder, and launching a heavy, dedicated third-party application. The Preview module negates this entirely by bringing the application to the file. When a user clicks on an Excel spreadsheet, they don't see a static image; they see a fully rendered, interactive table. They can click through the various sheet tabs at the bottom, scroll through thousands of rows of data, and verify the contents instantly before deciding whether they need to download it or share it.

The experience is highly tailored to the device the user is holding. If a user is reviewing high-resolution photography on a desktop computer, they can use their mouse wheel to smoothly zoom deep into the pixels of the image, click and drag to pan around the photograph, and use the on-screen buttons to rotate or flip the image to inspect it from different angles. If that same user opens the application on their smartphone, the mouse-centric controls are entirely replaced by native-feeling touch gestures. They can pinch two fingers outward to magnify the image, drag a single finger to pan, and when they are finished looking at the file, they simply swipe downwards. The image shrinks and fades away fluidly, returning them exactly to where they left off in their folder list.

For reading long-form content, the usability adapts to maximize comfort. Opening an EPUB eBook doesn't just dump text on the screen; it provides a dedicated reading environment with a clean, distraction-free background. The user can open a sliding drawer to view the book's chapters, click to jump instantly to a specific section, or use the built-in search tool to find specific character names or phrases, which are instantly highlighted in bright yellow across the text. They can even adjust the font size up or down to suit their eyesight, and the system remembers this preference for the next book they open.

The seamlessness of the experience is most apparent when navigating through a large folder of varied files. Because the system preloads the next file in the background, a user can open a PDF invoice, press the right arrow key on their keyboard, and instantly see the next file—perhaps a JPEG receipt—without any loading screens or buffering delays. Even if these files are secured with military-grade encryption, the decryption happens so fast in

the computer's memory that the user remains completely unaware of the complex cryptographic operations facilitating their smooth, uninterrupted workflow.

12. Online Office (OnlyOffice Integration)

12.1 Technical Description

The Online Office module (ui.modules.onlyoffice.php) provides a sophisticated, stateful bridge between the local client environment and an external OnlyOffice Document Server. Because OnlyOffice requires direct HTTP access to document files for collaborative editing and rendering, it presents a fundamental architectural challenge when interacting with End-to-End Encrypted (E2EE) vaults where the server holds only opaque ciphertext. To solve this, the module implements a secure, ephemeral proxy system.

When an encrypted document is targeted for OnlyOffice editing, the client-side WebCrypto engine decrypts the file in Random Access Memory (RAM). The frontend then constructs a uniquely named temporary file (.myCloud_temp_OO_[timestamp]_[filename]) and uploads this plaintext blob to the user's directory. This temporary file acts as the necessary unencrypted bridge for the OnlyOffice server. To mitigate the risks of leaving unencrypted data on the server in the event of a browser crash or closed tab, the frontend attaches a pagehide event listener to the window object. This listener utilizes the navigator.sendBeacon API, passing a multipart/form-data payload to the backend to guarantee the immediate destruction of the temporary file even as the browser process terminates.

The initialization of the OnlyOffice environment is governed by a secure, tokenized handshake. The frontend dispatches an AJAX request (get_office_config) to the backend, which constructs a JSON Web Token (JWT) containing the document key, callback URLs, and user permissions. The frontend then dynamically injects the OnlyOffice API script (api.js) into the Document Object Model (DOM). Upon successful script execution, the DocsAPI.DocEditor object is instantiated within a dedicated container (#onlyoffice_placeholder), applying user-specific configurations such as dark mode toggles and interface scaling based on the active device profile.

Saving and re-encrypting modified documents involves a highly synchronized polling mechanism. When a user closes the editor, the OnlyOffice server requires time to compile the collaborative changes and dispatch the final file back to the PHP backend via the callback webhook. The frontend initiates a recursive polling function (check_office_state) that interrogates the backend every 2000 milliseconds. Once the backend confirms the finalized document is ready, the frontend downloads the updated temporary file

into RAM, re-encrypts it using the directory's Data Encryption Key (DEK), uploads the new ciphertext to overwrite the original .enc file, and permanently deletes the temporary bridge file.

12.2 The Workflow

The OnlyOffice integration lifecycle follows a rigorous, multi-stage synchronization protocol:

- **Pre-Initialization and Security Clearance:**
  - User requests to open a supported Office document (docx, xlsx, etc.).
  - The system evaluates Role-Based Access Control (RBAC); read-only users are denied edit access and routed to the native previewer.
- **E2EE Bridge Construction (If Applicable):**
  - If the file is encrypted, the system evaluates the myCloud_OO_E2E_SkipWarn cookie. If absent, it halts and displays a security warning explaining that the file must be temporarily decrypted on the server.
  - The client downloads the ciphertext, decrypts it in RAM, and uploads it as a hidden temporary plaintext file.
- **Handshake and Instantiation:**
  - The frontend requests the OnlyOffice JWT configuration from the backend.
  - The OnlyOffice api.js script is fetched and executed.
  - The DocsAPI.DocEditor instance is mounted, replacing the standard file explorer UI.
- **Editing and State Tracking:**
  - The user edits the document.
  - The onDocumentStateChange event hook fires, flagging the container's data-is-modified attribute to true.
- **Closure and Re-Encryption:**

- The user clicks the close button; myCloudCloseOnlyOffice() executes.
- The editor instance is destroyed, and the standard UI is restored.
- If the file was encrypted and modified, the frontend begins polling check_office_state.
- Upon server confirmation, the frontend fetches the modified temporary file, encrypts it locally, uploads the final ciphertext, and issues a permanent deletion command for the temporary file.

12.3 Usability

The integration of OnlyOffice transforms the cloud storage platform into a fully-fledged productivity suite without compromising the underlying cryptographic architecture. When a user double-clicks a Word document or an Excel spreadsheet, the file list seamlessly fades away, replaced by a robust, native-feeling word processor or spreadsheet editor. The interface automatically adapts to the user's overarching theme preferences, loading a dark-themed editor if the user operates the cloud in dark mode.

For users managing highly sensitive data within encrypted vaults, the system prioritizes informed consent over silent operational compromises. Before opening an encrypted contract in OnlyOffice, the application explicitly warns the user that the file must be temporarily decrypted on the server to allow the external document engine to read it. The user is given the choice to proceed with the edit or fall back to a completely secure, read-only local preview.

Once the user finishes editing and closes the document, the application transparently handles the complex cryptographic cleanup. A progress indicator appears, assuring the user that the document is being securely re-encrypted and that all temporary server-side traces are being wiped. If the user accidentally closes their browser tab mid-edit, they do not need to panic about unencrypted data lingering on the server; the application utilizes modern browser APIs to fire a silent self-destruct signal in the background, guaranteeing that their privacy boundary remains intact under all circumstances.

13. The Text and Code Editor

13.1 Technical Description

The Editor module (ui.modules.editor.php) provides a localized, high-performance Integrated Development Environment (IDE) tailored for text, configuration, and source code manipulation. It is built upon the open-source Ace Editor framework, wrapped in a custom state-management architecture that supports multi-document tabbing, persistent local drafts, and E2EE compatibility.

The DOM injection for the editor is lazy-loaded via ceInitEditorDOM(). To minimize initial payload size, the complex HTML structures governing the toolbar, search interfaces, status bars, and Ace containers are entirely absent from the document until the user explicitly requests to edit a valid file type. Once triggered, the module dynamically appends the necessary scripts (ace.js, ext-modelist.js, ext-language_tools.js) and instantiates the myCloudEditor_ace object. The syntax highlighting mode is determined intelligently by myCloudEditor_detectMode(), which analyzes both the file extension (ignoring .enc wrappers) and the raw file contents (e.g., sniffing for #!/bin/bash or <?php headers) to apply the correct lexical parser.

A critical security and data-integrity feature is the custom paste interceptor. When a user pastes text into the editor, a specialized event listener sanitizes the clipboard payload before it reaches the Ace document session. It utilizes aggressive regular expressions to normalize various non-breaking spaces (e.g., \xA0, En/Em spaces) into standard 0x20 spaces, and completely strips zero-width joiners, invisible formatting marks, and non-printable ASCII control characters. This prevents hidden characters from corrupting source code execution or causing backend parsing failures.

The module implements a robust, multi-session tab manager (myCloudEditor_files). Each opened file is instantiated as a distinct Ace EditSession object, preserving its own undo/redo history, scroll position, and syntax mode. When switching tabs, the engine simply swaps the active EditSession within the singular Ace rendering container. To prevent data loss, the editor binds a debounced listener to the change event. Every keystroke updates a dirty-state flag and serializes the current document content into the browser's localStorage as a draft (myCloud_draft_[path]). If the application crashes, reopening the file intercepts the initialization routine, detects the orphaned local storage key, and prompts the user to restore their unsaved work.

Advanced visual tools are heavily integrated. The "Minimap" feature instantiates a secondary, synchronized Ace Editor instance with microscopic typography (fontSize: "3px") and stripped UI elements, acting as a high-level scrollable overview of the codebase. The "Diff" mode splits the container, mounting a read-only Ace instance loaded with the f.original unmodified file content alongside the active session, allowing users to visually track their alterations before committing a save. Finally, all saves on encrypted files are handled entirely locally; the editor pulls the plaintext from the Ace session, encrypts it via myCloudCrypto.encryptFile(), and pushes the resulting ciphertext to the server, ensuring the backend never sees the plaintext code.

13.2 The Workflow

The operational pipeline for the Editor module manages data retrieval, session handling, and secure storage:

- **Initialization and Retrieval:**
  - User invokes the edit action on a text-based file.
  - myCloudEditor_open() executes, triggering ceInitEditorDOM() and loading Ace dependencies if absent.
  - If the file is E2EE, it is fetched as a Blob, decrypted locally, and converted to text. If standard, it is fetched via the edit-fetch backend endpoint.
- **Session Instantiation:**
  - The system checks localStorage for pre-existing drafts matching the file path. If found, the user is prompted to restore or discard the draft.
  - myCloudEditor_detectMode() calculates the appropriate syntax highlighting.
  - A new EditSession is created, bound to the myCloudEditor_files array, and set as active.
- **Active Editing and Tracking:**
  - As the user types, the change event fires.
  - The f.isDirty flag evaluates if the current content deviates from f.original.

    - The session.$dirtyLines array tracks modified line numbers, scheduling an asynchronous gutter decoration update to highlight altered lines visually.
    - A debounced timer commits the active content to localStorage.
- **Advanced Tooling (Optional):**
    - Toggling Diff mode splits the view, binding the original text to a secondary read-only Ace instance.
    - Toggling Minimap opens a synchronized, scaled-down overview pane.
- **Commit and Save:**
    - The user triggers a save (via UI button or Ctrl+S).
    - If E2EE, the plaintext is encrypted into a Blob and uploaded to overwrite the source. If standard, the raw text is POSTed to edit-save.
    - Upon success, f.original is updated, the dirty state is cleared, gutter decorations are removed, and the localStorage draft is permanently purged.

13.3 Usability

The built-in Editor module elevates the platform from a simple file repository to a highly capable development and configuration environment. When a user opens a script or a configuration file, it doesn't just display plain text; it launches a fully featured code editor complete with line numbers, syntax highlighting, and auto-completion. The interface supports opening multiple files simultaneously, arranging them in neat tabs at the top of the window so a user can rapidly switch between editing an HTML document and its corresponding CSS stylesheet.

The editor is packed with features designed to protect the user's work and enhance their productivity. If a user is making extensive changes to a complicated configuration file, they can toggle the "Compare with Original" button. The screen splits, showing their actively edited version side-by-side with the unmodified original, making it easy to review changes before saving. A "Minimap" toggle provides a bird's-eye view of long documents, allowing the user to click and instantly jump to different sections of the code. Furthermore, web developers editing CSS files benefit from a built-in color picker;

clicking on any hex color code in the text summons an interactive color wheel, allowing them to adjust shades visually without leaving the editor.

Above all, the editor guarantees peace of mind regarding data loss. Every single character typed is silently backed up to the browser's local storage in real time. If the user accidentally closes the browser tab, experiences a power outage, or loses their internet connection before they manage to save, their work is not lost. The next time they open that specific file, the editor immediately detects the orphaned local draft and politely asks if they would like to restore their unsaved changes, ensuring that hours of careful typing are never wasted by unexpected technical failures.

14. Search

14.1 Technical Description

The Search module, instantiated via ui.modules.search.php, is a highly sophisticated, state-persistent querying interface that bridges the frontend Single Page Application (SPA) with a dual-engine backend search architecture. The module is engineered to handle massive datasets by providing granular filtering controls and supporting both deeply indexed content searches and standard recursive filename traversal.

The architectural foundation of the search module relies on state persistence to prevent data loss during modal dismissal. The myCloudState.searchParams object retains all user inputs in memory—including the active query, temporal bounds, spatial constraints, and tag filters. When the myCloudAction_Search() function is invoked, the system first interrogates the backend via an asynchronous check_index request. This request dictates the operational mode of the search module. If the server confirms the existence of a Recoll database (res.has_index), the frontend exposes the content-search capabilities and retrieves the last_update epoch timestamp to inform the user of the index's freshness. The modal generated is substantially larger than standard application dialogs, utilizing an 80vh flexbox container to accommodate complex data tables without breaking the viewport constraints.

The core complexity of this module lies in the dichotomy between Index Search and Non-Index Search, which the frontend orchestrates via the myCloudSearchContent checkbox and the useIndex state flag.

Index Search (Content and Deep Metadata) When the Recoll index is present and the user engages the index toggle, the search paradigm shifts from a superficial filesystem scan to a deep lexical query. The frontend transmits the content_search: '1' flag to the server. This mode allows the user to query the actual parsed plaintext inside documents (PDFs, Word files, source code). The frontend provides a dedicated syntax helper (myCloudShowSearchHelp), which dynamically calculates its absolute positioning to render a floating tooltip detailing the supported Boolean and proximity operators. In this mode, users can utilize wildcards, exact phrase matching ("exact phrase"), proximity constraints ("tax audit"10), and specific metadata targeting such as author:john or ext:pdf. Because this relies on a pre-compiled Xapian database on the server, the response times are highly

optimized, even when querying terabytes of data, as it bypasses real-time disk I/O.

Non-Index Search (Recursive Fallback) If the Recoll index is missing, or if the user leaves the search input completely blank to solely utilize filters (e.g., finding all files larger than 1GB), the system falls back to a non-index search (content_search: '0'). In this mode, the frontend instructs the backend to perform a live, recursive directory traversal starting from the myCloudState.currentDir. The frontend transmits complex boundary parameters: temporal filters (custom_date_start, custom_date_end), dimensional filters (custom_size_min, custom_size_max), and cryptographic tag filters. The backend evaluates every file against these parameters in real-time. To prevent timeouts, this mode relies heavily on precise filtering parameters to narrow the traversal tree.

Upon receiving the JSON payload from either search engine, the myCloudRenderSearchResultsTable() function takes control. It destroys the previous table body and constructs a new DOM structure. The rendering engine implements a deterministic sorting algorithm that isolates directories from files, forcing directories to the top of the view. It then evaluates the myCloudState.searchSort.col variable to apply ascending or descending lexical or numerical sorts based on the file name, byte size, modification date, or absolute path location.

Furthermore, the module integrates an advanced contextual toolbar. The myCloudUpdateSearchToolbar() function evaluates the current selection within the search results. Based on the selected file's extension and the user's Role-Based Access Control (RBAC) clearance, it dynamically enables or disables action buttons. For instance, if a directory is selected, the download and preview buttons are programmatically disabled. If a valid file is selected, the user can execute operations directly from the search modal—such as launching the file in the OnlyOffice editor, triggering a browser download, or clicking the "Go To" button. The "Go To" function (myCloudSearchAction('goto')) executes a complex state mutation: it strips the search modal from the DOM, parses the absolute path of the selected search result, extracts the parent directory, recursively expands the primary navigation tree (myCloudExpandToPath), fetches the target directory data, and triggers the myCloudSeekAndSelect() polling function to physically scroll the main user interface to the exact location of the discovered file.

14.2 The Workflow

- **Initialization and Pre-Flight Checks:**
  - The user invokes the search command (via toolbar or Ctrl+P shortcut routing).
  - The frontend dispatches the check_index POST request to determine backend capabilities.
  - The myCloudState.searchParams object is read to restore any previous search states.
- **Modal Construction and Parameter Binding:**
  - The DOM is injected with the search overlay, dynamically sizing to 1400px maximum width.
  - Event listeners are attached to the input field, updating the myCloudIndexWanted flag to prevent index queries on empty strings.
  - Dropdown listeners (onchange) evaluate selection states, dynamically unhiding HTML input fields for custom date ranges and byte sizes if the "custom" option is selected.
- **Query Execution:**
  - The user triggers myCloudPerformSearch().
  - The DOM table is replaced with a localized loading spinner.
  - The parameters (query string, date limits, size limits, active tag hexadecimal code, and index flag) are serialized into a URLSearchParams object.
  - The fetch request is transmitted to the backend.
- **Data Parsing and DOM Rendering:**
  - The backend JSON response populates myCloudState.searchResults.
  - myCloudRenderSearchResultsTable() iterates over the array, applying active column sorting algorithms.
  - Table rows are generated with onclick and ondblclick event bindings.

- **Interaction and Action Routing:**
    - The user selects a rendered row. myCloudUpdateSearch-Toolbar() evaluates the file extension and user permissions to enable action buttons.
    - The user clicks an action button (e.g., Preview, Edit, Go To).
    - The myCloudSearchAction() router terminates the search modal and pipes the absolute file path into the corresponding primary UI module function.

14.3 Usability

The search interface is designed to accommodate both casual queries and highly technical forensic data retrieval. When a user opens the search window, they are presented with a sprawling, deeply capable interface that dominates the screen, ensuring that complex data tables can be read without feeling cramped. For a simple task, the user can just type a filename into the main text box and hit enter, instantly receiving a list of matching files.

However, the true power of the module becomes apparent during complex investigations. A user looking for a specific invoice but unable to remember the filename can leverage the deep content index. By checking the "Use Index" box, they can type a specific phrase, such as "tax audit 2023," and the system will dive into the actual paragraphs of PDFs and Word documents to find the match. If the results are too broad, the user can utilize the dropdown menus to ruthlessly narrow the scope. They can instruct the system to only show files modified in the last 24 hours, or select "Custom" to input a specific date range from a calendar popup. They can further restrict the search to files larger than 50 Megabytes, or files that have been explicitly tagged with a specific color marker, such as the red "Urgent" tag.

The interactivity of the search results table mirrors the native desktop experience. Users can click on the column headers—Name, Size, Date, or Location—to instantly reorganize the data. If a user spots the file they need, they are not forced to memorize its location and hunt for it manually in the main interface. They can double-click the file row directly in the search results to open it immediately in the previewer. Alternatively, if they need to see the other files surrounding that specific document, they can highlight the row and click the "Go To" button. The search window vanishes, and the main application elegantly navigates through the folder hierarchy, expanding folders

automatically, until it smoothly scrolls the target file into the dead center of the screen, perfectly selected and ready for operation.

15. Multi Rename Tool

15.1 Technical Description

The Multi Rename Tool (ui.modules.multi_rename.php) provides a sophisticated, client-side string manipulation engine capable of executing complex batch renaming operations utilizing custom syntax tokens and Regular Expressions (RegEx). It isolates the computational heavy lifting to the user's browser, calculating collisions and final output strings in real-time before committing a single batched payload to the backend server.

The module initializes by filtering the global myCloudState.allItems array against the myCloudState.selectedFiles array, extracting the precise metadata required for the operation. It constructs a wide, highly structured modal containing dual input panes: one for syntax-based renaming and one for RegEx-based search and replace. A critical feature of this module is its state memory; it reads myCloudState.settings.renameHistory to populate a custom dropdown, allowing users to rapidly recall their most frequently used naming masks.

The core computational logic resides in the mrParseName() function. This engine evaluates the original filename against the user's input string, performing sequential string replacements based on proprietary tokens. The engine extracts the base name and extension using standard string manipulation (lastIndexOf('.')). It then evaluates static tokens: [N] injects the original base name, [E] injects the original extension, and [C] injects an iterating integer padded by the user-defined mrCountDigits value.

Beyond static tokens, the engine implements advanced dynamic range extraction using JavaScript's replace function combined with a callback. When it detects the [N...] token, it parses the internal arguments. If a comma is detected (e.g., [N-8,5]), it executes a substring extraction (substr), allowing the user to pull specific character lengths from the original string. If a dash is detected (e.g., [N1-5]), it splits the arguments and executes a bounded substring extraction. Furthermore, the engine accesses the file's epoch modification timestamp (fileObj.date), instantiating a Date object to facilitate time-based token replacements ([Y], [M], [D], [h], [m], [s]), alongside tokens for the current execution time ([tY], [tM], etc.), padded via a localized closure function.

Following the token resolution, the engine applies the Search and Replace logic. It reads the mrSearch and mrReplace inputs, evaluates the boolean

states of the mrCase and mrRegex checkboxes, and instantiates a native JavaScript RegExp object. If mrRegex is false, it sanitizes the input string by escaping all regex control characters (replace(/[.*+?^${}()|[\]\\]/g, '\\$&')) to perform a literal string replacement.

As the user modifies any input, the mrUpdatePreview() function fires immediately via oninput event listeners. It iterates through the selected files, running mrParseName() and the RegEx engine. Crucially, it maintains a JavaScript Set (newNamesSet) of the calculated output strings. If the Set detects a duplicate entry during the iteration, it flags the collision variable to true. The DOM rendering loop then highlights the conflicting output strings in bold red and disables standard execution, preventing the user from accidentally instructing the backend to overwrite files with identical newly generated names. Once the user resolves the conflicts and executes the process, the mrOps array is serialized and transmitted to the batch_rename backend endpoint for atomic execution.

15.2 The Workflow

- **Initialization and UI Generation:**
    - The application detects a multi-file selection and the user invokes the rename command.
    - myCloudShowMultiRenameModal() extracts the selected file objects and reads the user's renaming history from the settings state.
    - The DOM constructs the modal, appending event listeners (oninput, onfocus, onblur) to all configuration input fields.
- **Live Evaluation Loop:**
    - The user modifies an input parameter (e.g., typing [N]_[C], or clicking the helper menu to inject a date token).
    - The mrUpdatePreview() function triggers synchronously.
    - The RegEx object is compiled based on the Search and Replace fields and case/regex toggles.
    - The engine iterates through the target array, passing each file to mrParseName().
- **Token Parsing and Transformation:**

- mrParseName() extracts the base name and extension.
- Substring boundaries are calculated for advanced [N...] tokens.
- Epoch timestamps are converted to localized, zero-padded strings and injected.
- The iterating counter is incremented by the defined step value and injected.
- The compiled RegEx object is applied to the resulting string.

- **Collision Detection and DOM Paint:**
  - The final string is pushed to a Set. If it already exists, the collision flag trips.
  - The modal's internal table body is wiped and repainted.
  - Original filenames are rendered on the left, an arrow in the center, and the calculated preview string on the right (colored blue for success, red for collision).
- **Execution and Teardown:**
  - The user clicks "Apply Rename."
  - If a collision flag is active, a final confirmation prompt halts the user.
  - The array of source and destination paths is POSTed to the backend.
  - Upon successful response, the modal is destroyed, and the primary explorer UI is commanded to refresh the active directory.

15.3 Usability

The Multi Rename Tool transforms a tedious, manual administrative chore into a rapid, highly visual process. When a user selects fifty photographs from a digital camera—typically named with unhelpful strings like IMG_001.JPG—and opens this tool, they are presented with a powerful control panel. The interface is split: the top half contains the naming rules, and the bottom half displays a real-time, side-by-side preview of exactly what will happen to every single file.

The user does not need to be a programmer to use the tool, though it offers immense power for those who are. Next to the naming input fields are small

helper buttons (marked with a '+'). Clicking these opens a simple dropdown menu allowing the user to insert placeholders. A user can select [Y]-[M]-[D] ISO Date, type a space, and then select [C] Counter. As they click these helpers, the live preview table at the bottom instantly updates, showing the cryptic IMG_001.JPG transforming beautifully into 2024-10-25 01.jpg. They can adjust the counter settings to start at 100 instead of 1, or tell the counter to step by 10s, watching the preview update with every keystroke.

For users dealing with messy data exports, the Search and Replace functionality is invaluable. They can type "report" into the search box and "Q3_Summary" into the replace box, instantly updating dozens of files. A dedicated "Date Fix" button provides one-click solutions for standardizing messy timestamps, utilizing safe, pre-calculated regular expressions to turn chaotic filenames like 2021.03.20 11.38 into clean, system-friendly 2021-03-20_11-38 formats.

Crucially, the tool protects the user from making catastrophic mistakes. If a user accidentally configures a naming rule that would result in two files sharing the exact same name—which would cause the server to overwrite one of them and permanently destroy data—the live preview table instantly turns the conflicting names bright red, and a stern warning appears at the bottom of the window. This immediate, visual safety net ensures that users can confidently experiment with complex naming patterns without fear of data loss.

16. Public Sharing

16.1 Technical Description

The Public Sharing module, governed primarily by the standalone modules.share_public_ui.php script and supported by specific endpoints within the controller.server.php architecture, represents a critical boundary-crossing mechanism. It allows internal, authenticated users to selectively expose segments of their private, encrypted, or standard file hierarchies to unauthenticated external actors. Because this module deliberately bypasses the primary authentication gateway of the application, it is engineered with an aggressive, multi-layered security posture, functioning as an isolated, heavily fortified sandbox.

The instantiation of a public share is managed by the internal backend routing engine. When an authenticated user executes a share-create action, the system does not generate a sequential or predictable identifier. Instead, it utilizes bin2hex(random_bytes(25)) to generate a 50-character hexadecimal Globally Unique Identifier (GUID). This provides approximately 200 bits of cryptographic entropy, rendering link-guessing or enumeration attacks computationally infeasible. The metadata for this share—including the absolute physical path of the target directory, the expiration timestamp, the maximum permitted download count, the BCrypt-hashed password (if applicable), and the specific permission tier (read, modify, or upload)—is serialized into a central JSON database (shares.json). To ensure data integrity in a highly concurrent web server environment, all read and write operations to this database are protected by strict flock() implementations, preventing race conditions when multiple users create or modify shares simultaneously.

When an external request hits the modules.share_public_ui.php endpoint via a ?cloudshare=GUID GET parameter, the system initiates a rigorous preflight security sequence before any DOM rendering or file I/O is permitted. The first layer of defense is an IP-based rate-limiting engine. The function cxCheckRateLimit() interrogates a dedicated blocklist.json ledger. If the incoming IP address has registered more than the configured $max_login_attempts (defaulting to 5) within a specific timeframe, the connection is instantly severed with a 403 Access Denied response, and the IP remains in a locked state (commonly 900 seconds). This aggressively mitigates localized brute-force attacks against password-protected links.

If the IP passes the initial WAF threshold, the system loads the share database and validates the GUID. It performs atomic checks against the share's lifecycle parameters: it verifies that the current epoch time has not surpassed the expires timestamp, and it validates that the downloads counter has not met or exceeded the max_downloads limit. If the download limit is triggered during a file request, the system fulfills the final request but immediately deletes the share object from the JSON database, ensuring the link is permanently decommissioned. Furthermore, to combat distributed botnet attacks where thousands of different IP addresses might attempt to guess a single share's password, the system implements a second layer of brute-force protection scoped to the GUID itself. The cxIncrementShareAttempts() function tracks global failed attempts against a specific share. If 15 failed attempts occur within an hour, the share is placed into a global locked_until state, denying all access—even with the correct password—for one hour, thereby mathematically neutralizing distributed dictionary attacks.

If the share dictates password protection, the module halts execution and delivers a minimalist, localized HTML login form. This form is fortified with a cryptographically secure Cross-Site Request Forgery (CSRF) token bound to the PHP session. Upon successful password verification via password_verify(), the system issues a session regeneration command (session_regenerate_id(true)) to prevent session fixation. It then establishes a unique tab-isolation token (cx_tab_token). By setting a specific cookie and verifying it against the session state on subsequent requests, the application ensures that if a user opens multiple different password-protected shares in various browser tabs, the authorization states do not bleed into one another, maintaining strict contextual isolation.

Path traversal prevention is absolute and mathematically verified. The module extracts the requested sub-path from the URL, aggressively sanitizes it by stripping trailing slashes, backward slashes, and .. notation. It appends this sanitized relative path to the physical root path of the share and passes the result to PHP's native realpath() function. The system then strictly compares the resulting absolute path against the absolute path of the share root using a strpos($realCurrentPath, $sharedRootPath) === 0 operation. If the requested path resolves to any location outside of the designated share root, the system throws a fatal invalid path error, completely neutralizing directory traversal exploits.

For shares with modify or upload permissions, the module implements a highly restrictive file ingestion pipeline. When handling incoming multi-part/form-data streams, the system first verifies the directory quota using the cxGetDirSize() recursive iterator. If the upload would exceed the configured $quota_str (e.g., 4GB), the upload is aborted. Accepted files are then passed through the cxIsSafeFile() deep-inspection function. This function not only verifies the file extension against a strict allowlist but actively opens the temporary uploaded binary file using fread() to inspect the first 1024 bytes. It actively searches for and rejects malicious magic numbers, including PHP execution tags (<?php), Windows executable headers (MZ), Linux ELF binary headers (\x7FELF), and shell script shebangs (#!). This guarantees that an external actor cannot use a public drop folder to establish a web shell or distribute executable malware.

For read-only operations, the module features dynamic ZIP generation and caching image previewers. When a user requests to download a folder as a ZIP, the system calculates the total recursive byte size of the directory. If it exceeds the $max_zip_size limit (default 300MB), the operation is blocked to prevent CPU and RAM exhaustion (Zip Bomb mitigation). Permitted requests are archived on the fly using the ZipArchive class to a temporary system directory and streamed to the user. For media previews, the module hooks into the exact same caching directory used by the primary application ($cloud_preview_cache). It utilizes Imagick or GD to generate, flatten, and EXIF-correct downscaled JPEG thumbnails, serving them with immutable cache-control headers. This shared caching architecture ensures that if an internal user has already generated thumbnails for a folder, the public share utilizes those exact physical assets, drastically reducing redundant computational overhead on the web server.

16.2 The Workflow

The complete lifecycle of public sharing, from internal instantiation to external consumption, follows this execution path:

- **Phase 1: Share Instantiation (Internal Backend)**
    - The authenticated user selects a directory and transmits share parameters (password, expiration date, download limits, permission tier) to the share-create API endpoint.
    - The backend validates ownership of the directory and generates a secure 50-character hexadecimal GUID.

    - The parameters are serialized and appended to the shares.json database under a strict flock(LOCK_EX) mutual exclusion lock.
    - The backend calculates the absolute URL bridging to the modules.share_public_ui.php script and returns it to the client for distribution.
- **Phase 2: Request Interception and Global Security (External Frontend)**
    - An external actor accesses the URL containing the ?cloudshare=GUID parameter.
    - The PHP script initiates. Session cookie parameters are strictly bound to HttpOnly and SameSite=Strict policies.
    - The client's IP address is extracted and evaluated against blocklist.json. If the IP is blacklisted, execution terminates with an HTTP 403 Access Denied.
- **Phase 3: Share State and Authentication Validation**
    - The script acquires a shared lock (LOCK_SH) on shares.json and retrieves the target share object.
    - The system evaluates the global lock status (locked_until), the expiration timestamp (expires), and the download limit counter (max_downloads). Violations result in immediate termination.
    - If a password hash exists, the system checks for a valid session and matching cx_tab_token cookie.
    - If unauthenticated, a CSRF-protected HTML login form is rendered. Failed attempts trigger cxRegisterFail() (IP penalty) and cxIncrementShareAttempts() (GUID penalty).
    - Upon successful password_verify(), the session is regenerated, tokens are issued, and the user is redirected via a 302 header to the authenticated state.
- **Phase 4: Path Resolution and Capability Assessment**
    - The subpath GET parameter is sanitized and appended to the share's physical root directory.

- realpath() resolves the final destination, and a string-prefix match validates that the destination remains mathematically bounded within the share root.
- The script evaluates the assigned permission tier (read, modify, upload).

- **Phase 5: Action Execution and Routing**
  - **If Action = Blind Upload (upload tier):** The system generates a timestamped, unique sub-folder (e.g., 2024-10-25_14-30_64a9b...) to isolate the external user's drops. The user is presented with a specialized drag-and-drop interface.
  - **If Action = File Download:** The cxRecordDownload() function increments the database counter. Necessary HTTP headers are written, and the file is pushed to the client using chunked fread() loops to preserve server memory.
  - **If Action = Directory ZIP:** The directory is recursively scanned. If the total byte size falls under the safe limit, ZipArchive compiles a temporary file, streams it to the user, and a shutdown function ensures the temporary archive is immediately unlinked from the disk.
  - **If Action = Directory Browsing:** The script scans the target directory via scandir(). It calculates human-readable file sizes, formats modification dates, and evaluates image extensions. Based on the ratio of images to standard files, the script decides whether to default the UI to a List View or a Gallery View.
- **Phase 6: DOM Rendering and Delivery**
  - The output buffer is captured.
  - A responsive HTML5 document is generated, embedding localized strings (e.g., English or German) based on the browser's HTTP_ACCEPT_LANGUAGE header.
  - Inline CSS and JavaScript are injected to handle client-side sorting, lazy-loading IntersectionObserver logic, and asynchronous upload tracking.
  - The final HTML payload is minified via cloudExMinifySafe_Html and delivered to the browser.

16.3 Usability

The public sharing functionality is meticulously designed to offer a frictionless, intuitive experience for two distinct groups: the internal user originating the share, and the external recipient consuming it. From the perspective of the logged-in user, exposing a private directory requires no technical overhead. The user simply right-clicks a folder and selects "Share." A sleek dialog box appears, offering plain-language controls. The user can type a password, define how many days the link should remain active, or state that the link should self-destruct after exactly three downloads. They can also dictate what the recipient is allowed to do. If they select "Read Only," the recipient can only view and download files. If they select "Modify," the recipient effectively gains a mini-cloud drive, capable of deleting files and organizing subfolders. Crucially, if the user selects "Upload Only," they create a "Blind Drop" zone. This is immensely useful for collecting sensitive documents from clients or contractors; the external party can upload files into the folder, but they cannot see the contents of the folder or any files uploaded by other people.

From the viewpoint of the external recipient, the experience is highly polished and professional, entirely devoid of the confusing, ad-filled interfaces common to commercial file-sharing services. When the recipient clicks the link, they are instantly greeted by a clean, minimalist login screen if a password was assigned. The interface automatically adapts its language to match the recipient's operating system, ensuring a German client sees German instructions while a US client sees English.

Upon gaining access, the interface dynamically shapes itself to suit the content. If the sender shared a folder containing spreadsheets and PDF reports, the recipient sees a neat, sortable list view displaying file names, sizes, and modification dates. If the sender shared a folder containing hundreds of high-resolution photographs from a recent event, the interface intelligently detects the media density and automatically shifts into a beautiful, immersive Gallery View. In this mode, the recipient sees a masonry grid of large thumbnails. As they scroll down the page, the images load fluidly and beautifully, utilizing modern lazy-loading techniques so their mobile data plan isn't exhausted instantly.

Interacting with the shared files requires zero training. If the recipient wants a specific file, a single tap or click initiates an instant download. If they want the entire folder, a prominent "Download all as ZIP" button compiles

everything for them automatically. For shares granting upload permissions, the recipient does not need to navigate complex menus; they can simply drag a highlight of files from their computer desktop and drop them directly onto the web browser window. The entire screen highlights with a dashed blue border, confirming the drop zone. A clean progress overlay appears, detailing the upload percentage and confirming exactly which files were successfully transmitted, ensuring the external party feels confident that their data was received securely and accurately.

17. The SFTP Mode (Server Admin Mode)

17.1 Technical Description

The SFTP Mode, internally designated as the "Admin Mode," represents a fundamental paradigm shift within the application's architecture. While standard operational modes virtualize a localized filesystem environment, the SFTP module transforms the web browser into a direct, low-level Remote Server Administration tool. It establishes a secure, persistent bridge between the frontend interface and a remote Linux/Unix filesystem utilizing the SSH-2 protocol. Due to the catastrophic potential of compromised administrative access, this module operates within a highly compartmentalized, zero-trust execution sandbox that bypasses standard application routing to enforce its own cryptographic and session-lifecycle controls.

17.1.1 Cryptographic Isolation and Explicit Access Control

The existence of the Admin Mode is structurally obscured from unauthorized users. During the frontend application bootstrap phase, the PHP pre-processor iterates through the $user_details array. The JavaScript logic required to interact with the Admin Mode is only injected into the Document Object Model (DOM) if the authenticated user explicitly possesses the rights === 'admin_mode' flag for a specific cloud configuration. For users lacking this flag, the entire administrative JavaScript payload is bypassed, minimizing the attack surface by ensuring that the client-side code fundamentally does not exist in their browser memory.

When an authorized administrator accesses an Admin Mode tab, the frontend initializes a sophisticated passive observer engine. This engine monitors user interaction (mousedown, keydown, mousemove) and dispatches an asynchronous admin_heartbeat POST request every 5 minutes to maintain the specialized SSH session state. If the user navigates between a standard cloud tab and an Admin tab, the observer detects the context switch (_ceAdminLastTab !== myCloudState.key) and immediately fires an admin_check request. This request verifies the temporal validity of the SSH session. If the session has expired or the user has not yet authenticated against the remote server, the application suppresses the standard directory fetch and immediately summons the myCloudPromptAdminAuth() modal.

17.1.2 The Authentication Sandbox and Anti-Automation

Connecting to a remote server via SFTP/SSH requires the user to input the server's absolute root or user password. To protect this highly sensitive

credential from client-side interception, the authentication modal deploys a multi-layered defense matrix:

- **Honeypot Architecture:** Modern mobile keyboards, heuristics-based password managers, and browser autofill engines aggressively attempt to capture and inject credentials into recognizable forms. To thwart this, the DOM injects an invisible container (opacity:0; height:0; width:0; overflow:hidden;) containing decoy username and password fields. Automated systems latch onto these decoys.
- **Input Obfuscation:** The genuine password input field employs autocomplete="new-password one-time-code", spellcheck="false", and data-lpignore="true" (to bypass LastPass and similar extensions), ensuring the browser treats the input as a transient, non-recordable keystroke sequence.
- **Backend Exponential Lockout:** Upon submission, the backend evaluates the attempt via the checkLockout() method. Failures are tracked persistently in an admin_lockout.json ledger, keyed by a concatenation of the username and the client's IPv4/IPv6 address. A hard threshold is set at 5 failures. Subsequent failures trigger a mathematically escalating penalty window (pow(2, $fails - 5) * 30 minutes), culminating in a maximum lockout of 525,600 minutes (one year) to entirely neutralize automated credential-stuffing attacks.

Note on Auto-Login: For highly controlled, trusted environments, server administrators can define a $cloud_admin_mode_cloudlist file. The backend hashes the configured SSH target (hash('sha256', 'user@host:port')) and evaluates it against this ledger. If a match is found containing a pre-shared key, the system bypasses the manual password prompt, mapping the credentials directly into the encrypted RAM session.

17.1.3 The Inter-Process Communication (IPC) Daemon

Standard PHP-FPM (FastCGI Process Manager) environments are optimized for rapid, short-lived HTTP requests. Executing prolonged, recursive filesystem modifications (e.g., changing ownership on 500,000 files via SFTP) within a standard PHP thread will inevitably trigger HTTP 504 Gateway Timeouts or max_execution_time fatal errors, resulting in catastrophic, partial filesystem states.

To circumvent this limitation, the Admin Mode implements an ingenious detached background daemon architecture utilizing the SFTPDaemonProxy class.

- **Daemon Instantiation:** When a heavy command is requested, the system calculates a unique, highly specific IPC directory: hash('sha256', $username . $key . session_id()). It checks for an active daemon.pid. If absent or stale, it compiles a configuration payload (containing the SSH host, port, user, and decrypted session password) and writes it to a temporary file locked to chmod 0600.
- **Process Detachment:** The script utilizes pclose(popen(...)) combined with nohup (Linux) or start /B (Windows) to execute a secondary, isolated PHP Command Line Interface (CLI) process. This entirely drops the process handle from the primary web server thread, allowing the daemon to run indefinitely in the background.
- **Atomic Message Passing:** The frontend request is serialized into a JSON payload and written to a .tmp file within the IPC directory. Using an atomic filesystem operation (rename()), the file is swapped to a .json extension. This guarantees that the background daemon never reads an incompletely written file.
- **Execution and Polling:** The primary HTTP thread enters a highly optimized micro-polling loop (usleep(5000)), monitoring the IPC directory for a corresponding .res (response) file. Meanwhile, the background daemon reads the .json request, executes the SFTP operation via phpseclib3, and writes the outcome to the .res file. Once detected, the HTTP thread captures the response, deletes the IPC files, and completes the web request, effectively allowing synchronous-feeling UI interactions powered by asynchronous, timeout-immune background processing.

17.1.4 Interactive Terminal Emulation and Data Streaming

Beyond file management, the Admin Mode offers a fully interactive Secure Shell (SSH) terminal, integrating the Xterm.js frontend library with a custom PHP Server-Sent Events (SSE) streaming engine.

- **The Stream Engine (actionSshStream):** When the user triggers the terminal, the backend establishes a raw SSH-2 connection. It requests a Pseudo-Terminal (PTY) utilizing the xterm-256color environment variable, enabling full color and cursor-positioning support. The PHP

script modifies its HTTP headers to text/event-stream and explicitly disables zlib compression and Nginx/Apache buffering (X-Accel-Buffering: no). It enters an infinite loop, capturing binary output from the SSH stream and flushing it to the browser as Base64-encoded JSON chunks. To prevent the loop from consuming 100% of the CPU, it utilizes the @stream_select function with a 20-millisecond micro-timeout, allowing the process to idle efficiently until the remote server transmits data.

- **The Input Engine (actionSshWrite):** Client keystrokes are captured by Xterm.js, Base64-encoded, and POSTed to a separate backend endpoint. To bridge the gap between the HTTP POST request and the infinitely looping SSE stream, the input is appended to a physical .buf file in the IPC directory using a strict LOCK_EX file lock.
- **Ghost Cursor Mitigation:** The SSE loop continuously monitors the .buf file. If data exists, it acquires an exclusive lock, executes a rewind() operation to snap the file pointer to the 0-byte mark, reads the payload, transmits it into the live SSH socket, and immediately truncates the file (ftruncate($fp, 0)). This highly specific sequence ensures that rapid, overlapping keystrokes from the user are never duplicated or dropped by race conditions between the web server and the filesystem.

17.1.5 Advanced File System Manipulation and Metadata Resolution

The Admin Mode replaces standard cloud file operations with their low-level SFTP equivalents. The actionList method utilizes $sftp->rawlist() to capture deep Unix metadata, including exact octal permission bits (0755, 0644), symbolic link targets, and raw Ownership/Group IDs.

- **UID/GID Caching:** Because raw lists return numerical UIDs and GIDs (e.g., 1000:1000), the resolveUid and resolveGid functions silently fetch /etc/passwd and /etc/group from the remote server upon initialization. They parse the colon-separated structures into associative arrays in RAM, instantly mapping the numerical IDs back to their human-readable strings (e.g., root:www-data) for the frontend UI.
- **Recursive Permissions (chmod/chown):** The module allows users to alter Unix permissions graphically. If the user commands a recursive modification, the backend daemon takes over. It initiates a deep recursive traversal of the target directory tree over the SFTP socket,

executing discrete chmod, chown, and chgrp commands against every child node, protected from timeouts by the detached IPC architecture.

- **Rsync Synchronization (actionAdminSync):** For transferring massive datasets between remote directories, standard SFTP is unacceptably slow. The Admin Mode features a dedicated Directory Sync tool. When invoked, the backend generates an OS-level rsync -avh --info=progress2 command. It pipes this execution through a dedicated SSH stream, utilizing regular expressions (preg_match('/(\d+)%/')) to parse the raw text output of the Rsync binary, extracting the precise completion percentage in real-time and streaming it back to the frontend progress bar.

17.2 The Workflow

The lifecycle of an Admin Mode session from connection to command execution follows this rigorous path:

1. **Context Initialization:**
    - The user navigates to a tab mapped to an Admin Mode configuration.
    - The _ceAdminLastTab observer detects the shift and fires an admin_check XHR request.
    - The backend verifies the presence of an active admin_ssh_pwd session variable. If absent, the server responds with an AUTH_REQUIRED status.
2. **Authentication Sequence:**
    - The frontend triggers myCloudPromptAdminAuth().
    - The user enters the server's SSH password. The payload is POSTed to the admin_auth endpoint.
    - The backend attempts an authentication against the remote SFTP server utilizing phpseclib3.
    - Upon success, the backend generates a 32-byte hexadecimal admin_nonce, caches the password in the PHP session, resets the 12-hour expiration timer, and returns the nonce to the client.
3. **Native API Interception:**

- The frontend overwrites the native window.fetch API.
- For all subsequent POST requests matching the Admin Mode tab key, the interceptor silently injects the current admin_nonce into the FormData or URLSearchParams payload.

4. **Standard SFTP Operation (e.g., List Directory):**
   - The frontend requests a directory list.
   - The backend verifyAuth() method strictly checks the inbound admin_nonce against the session nonce. If they match, a new nonce is generated and returned, establishing a rolling cryptographic challenge-response chain.
   - The backend utilizes $sftp->rawlist() to extract Unix directory parameters, resolves UIDs/GIDs, and formats the JSON response.
5. **Heavy Operation (e.g., Recursive Delete):**
   - The frontend commands the deletion of a large directory.
   - The backend evaluates the request and determines it requires daemonization.
   - The SFTPDaemonProxy class generates the IPC directory, writes the operational parameters to req_UID.json, and spawns the detached CLI process.
   - The primary HTTP thread enters a usleep(5000) polling loop.
   - The detached CLI daemon reads the .json file, executes $sftp->rmdir(), and writes the completion status to req_UID.res.
   - The HTTP thread detects the .res file, consumes the output, cleans the IPC directory, and completes the web request.
6. **Terminal Execution:**
   - The user clicks the Terminal icon.
   - A hidden iframe/fetch request opens a persistent HTTP connection to ssh_stream.
   - The backend allocates a PTY, starts the xterm-256color session, and enters a stream_select loop, flushing output to the browser.

- The user types a command (e.g., htop). The keystrokes are POSTed to ssh_write, written to the IPC .buf file, consumed by the stream loop, and passed into the remote shell. The visual result is streamed back instantly.

17.3 Usability

The Admin Mode is designed to shatter the traditional boundaries between a web browser and a remote server infrastructure. When a system administrator logs into the cloud and switches to a tab configured for Admin Mode, they are immediately greeted by a sleek, secure prompt asking for the server's SSH password. Once authenticated, the familiar, friendly interface of the cloud storage platform transforms into a high-powered, low-level server management utility.

The transition is completely seamless, yet the power available is vastly expanded. The standard file list now displays deep Unix metadata: the user can see exactly who owns a file (e.g., root, www-data), which group it belongs to, and its exact octal permissions. If an administrator needs to fix a broken web directory, they don't need to open a separate SSH client like PuTTY or Terminal. They simply right-click the folder within the browser, select "Permissions," and a highly intuitive dialog box appears. They can visually check boxes for Read, Write, and Execute across Owner, Group, and Others. By checking the "Apply recursively" toggle and hitting Save, the application silently orchestrates a massive background operation, fixing thousands of file permissions on the remote server while the administrator continues to browse freely.

For tasks that require raw command-line power, the Admin Mode delivers an unparalleled feature: the integrated SSH Terminal. With a single click of the Terminal icon in the ribbon, a sleek, black command-line interface slides up from the bottom of the screen. This isn't a simulated, limited prompt; it is a true, fully interactive Pseudo-Terminal. The administrator can run complex, interactive visual applications like htop or vim directly inside their web browser. If they drag the terminal window's resize handle to make it larger, the system instantly detects the pixel change, recalculates the column and row geometries, and transmits a resize signal to the remote server, ensuring that the command-line interface perfectly reflows its text without breaking the layout.

Furthermore, the Admin Mode addresses the agonizing pain of migrating massive amounts of data between server directories. Traditionally, moving gigabytes of data requires complex command-line syntax. In Admin Mode, the user simply clicks the "Sync Directories" button. A clean dialog asks them for the Source and Destination, offering checkboxes for advanced features like "Mirror" (to delete missing files) or "Dry Run." Upon execution, the system launches an OS-level Rsync process in the background. The user gets a beautiful, real-time progress bar tracking the exact byte-for-byte transfer status across the server's internal network, bringing the tactile, visual comfort of a modern operating system to the most complex tasks of Linux server administration.

18. The Help and Ticketing System

18.1 Technical Description

The Help and Ticketing System, initialized via the ui.modules.help_ui.php module and supported by the actionHandleTicket and actionGetHelpData backend controllers, serves as an integrated user-support architecture. It bifurcates into two heavily interconnected subsystems: a dynamic, JSON-driven documentation rendering engine, and a persistent, stateful support ticketing matrix that interfaces directly with the application's semantic versioning and release management logic.

Dynamic Documentation Engine The documentation subsystem eschews static HTML payloads in favor of a dynamic, client-side rendering pipeline utilizing structured JSON datasets. When the user requests the help interface, the frontend dispatches a get_help_data Request to the backend, transmitting the currently active localization code (e.g., en, de). The backend fetches the corresponding JSON file from the /help/ directory and streams the serialized object to the client.

Upon reception, the frontend executes a Role-Based Access Control (RBAC) filtering algorithm before rendering. It iterates through the root nodes of the JSON object, evaluating a designated role attribute against the active user's permissions. If a help block is designated as write (e.g., instructions on uploading files or creating directories) and the current user's session role is read-only, that specific block is purged from memory. This guarantees that users are only presented with documentation relevant to their cryptographic clearance and capability matrix.

To transform the JSON arrays into a rich Graphical User Interface (GUI), the module employs a custom pseudo-markup parser (renderBlocks). This parser translates strict JSON node types (h1, h2, p, tip, warn, ul, grid, table, treemap) into their respective DOM equivalents. Furthermore, it parses inline regex-based shortcodes (e.g., [desktop], [key:Ctrl], *bold*) to inject localized SVG icons and standardized CSS classes uniformly across the documentation.

To facilitate rapid information retrieval, the engine implements a deep-traversal client-side search algorithm. When a user types into the search field, the engine does not merely match against top-level article titles or keyword arrays. Instead, it recursively traverses the nested DOM mapping within the JSON structure, evaluating the search string against individual paragraph

texts, table cells, and grid card descriptions (matchContent). This ensures immediate, highly accurate filtering of the documentation tree without generating auxiliary backend requests.

Ticketing and Release Management Architecture The secondary component of this module is the Support Ticketing System, which operates as an asynchronous Create, Read, Update, Delete (CRUD) interface backed by a flat-file JSON database (tickets.json).

When a standard user opens the ticketing interface, the backend queries the database and filters the response array to ensure the user only receives ticket objects matching their explicit username identifier, ensuring privacy. However, if the user possesses the full administrator role, the backend delivers the entire database payload.

The frontend employs a mathematical sorting algorithm (sortTickets()) to organize the rendered ticket cards. It assigns static integer weights to semantic status strings: Open (3), In Progress (2), Later (1), and Closed (0). The primary sort evaluates this weight. If the weights are identical, the algorithm evaluates an integer-based priority parameter. If priorities are identical, it falls back to sorting by the Epoch creation timestamp (timestamp) in descending order.

For administrative users, the DOM injects a specialized control panel onto every ticket card. This allows the administrator to manipulate the ticket's state, alter its priority, and append operational comments. Crucially, the system acts as a release-management bridge via the ceCopyToChangelog function. When an administrator resolves a ticket, they can instruct the system to promote the fix to the application's changelog. The backend ticket-changelog controller opens the physical versioninfo.txt file on the server. Depending on the administrator's selection, it executes a Semantic Versioning (SemVer) incrementation:

- **Patch:** Increments the third integer (x.y.Z+1).
- **Minor:** Increments the second integer and resets the patch (x.Y+1.0).
- **Major:** Increments the primary integer and resets minor/patch (X+1.0.0). The backend then prepends the appropriate semantic prefix based on the ticket type (e.g., mapping Bug to Fix:, Feature to Feature:) and writes the sanitized ticket title directly into the versioninfo.txt file,

automatically closing the ticket and permanently recording the application update.

To ensure long-term system performance and prevent the JSON database from bloating infinitely, the backend ticket saver implements an automated garbage collection routine. It calculates an Epoch cutoff threshold (current time minus 90 days). Any ticket carrying a Closed status with a creation timestamp older than this threshold is silently purged from the array during the write operation.

18.2 The Workflow

The operational sequence for documentation retrieval and ticket lifecycle management is executed as follows:

- **Help Initialization and Documentation Retrieval:**
  - The user invokes the Help modal via the top ribbon toolbar.
  - The myCloudOpenHelp() function builds the modal framework and evaluates the user's localized language preference.
  - The frontend executes fetchHelpData(), transmitting the language code to the backend.
  - The backend reads the appropriate JSON file and returns it.
  - The frontend filters the JSON array against the user's RBAC matrix.
  - The remaining objects are parsed by renderList(), which builds the left-hand navigation tree and auto-renders the first visible topic into the main reading pane via renderBlocks().
- **Deep Content Searching:**
  - The user inputs text into the search field.
  - The oninput event listener triggers an immediate, debounced re-evaluation of the JSON array in RAM.
  - The algorithm compares the query against title attributes, explicit keyword arrays, and recursively against nested text nodes (paragraphs, table cells).
  - The left-hand navigation tree is instantaneously rebuilt to display only nodes containing matching data.

- **Ticket Creation (Standard User):**
    - The user switches to the Support tab and toggles the "+ Create New Ticket" form.
    - The user selects a semantic category (Bug, Feature, Improvement, Security, Task), provides a title, and describes the issue.
    - The ceSubmitTicket() function serializes the input and POSTs it to the ticket-create backend endpoint.
    - The backend generates a unique tkt_ hexadecimal ID, appends the user's session identifier and current Epoch timestamp, and saves the object to tickets.json.
    - The frontend triggers ceLoadTickets() to asynchronously refresh the view and display the newly created card.
- **Ticket Management and Release Promotion (Administrator):**
    - An administrator opens the Support tab. The backend delivers the complete tickets.json payload, and the frontend renders all cards with injected administrative controls.
    - The administrator alters a ticket's status from Open to In Progress and appends a comment. This fires ceUpdateTicket(), silently updating the database.
    - Once the issue is resolved in the codebase, the administrator selects a Semantic Versioning increment (e.g., minor) from the dropdown and clicks "To Changelog."
    - The backend parses the current version from versioninfo.txt, executes the mathematical increment, concatenates the ticket type prefix and title, prepends it to the changelog, marks the ticket as Closed, and saves both files.

18.3 Usability

The Help and Ticketing System is designed to provide immediate, context-aware assistance without forcing the user to abandon their current workflow or navigate to an external support portal. When a user clicks the Help icon, a large, clean modal overlay appears. The left side features a navigation list of topics, while the right side displays beautifully formatted articles. These articles are not merely walls of text; they utilize native iconography, distinct

warning boxes, interactive data tables, and dynamic grid layouts to explain complex cloud concepts intuitively. Because the system intelligently filters content, a user who only has permission to view files will never be confused by documentation explaining how to upload, rename, or delete data—the manual literally adapts itself to their specific capabilities.

If a user cannot find what they are looking for by browsing the navigation tree, they can utilize the highly responsive search bar. As they type a specific error message or a concept like "password," the navigation list shrinks in real-time, instantly isolating only the exact articles that mention the queried phrase, whether that phrase is in the title or buried deep within a paragraph in the third section of an article.

Should the documentation prove insufficient, or if the user encounters a bug, they can click the "Support" button within the same window. Here, they are presented with a clean, card-based interface detailing any requests they have previously submitted. Creating a new request is as simple as clicking a button, choosing whether it is a bug or a feature request, and typing a short message. They can track the status of their request natively; they will see when an administrator changes the status to "In Progress" and can read direct feedback left by the administrative team.

For the system administrators, this module functions as an embedded project management suite. They do not need to rely on external software like Jira or GitHub issues to manage user feedback. They receive all tickets directly within the cloud interface, sorted automatically so that high-priority, open bugs sit at the very top of their list. The most powerful aspect of this system is its integration with the application's actual release cycle. When an administrator fixes a reported bug, they do not have to manually update the application's version number or type out a changelog document. They simply click "To Changelog" directly on the ticket card. The system automatically calculates the next version number, translates the ticket into a professional release note, updates the application's welcome screen, and closes the user's ticket simultaneously, creating a perfectly streamlined loop between user feedback and software deployment.

19. The Administrative UI and Configuration Management

19.1 Technical Description

The Administrative User Interface (UI), governed by the Native Admin & Config Management Module, is an isolated, highly privileged Sub-Single Page Application (SPA) embedded within the broader cloud platform. It facilitates the direct manipulation of user databases ($user_db) and core system configurations (config.php). Because this module writes executable PHP code and alters access control matrices, it operates under a strict zero-trust model utilizing lexical parsing, atomic write operations, and aggressive client-side state monitoring.

Security Perimeters and Access Control The module is completely inaccessible to standard users. The frontend JavaScript payload and HTML structures are only injected into the Document Object Model (DOM) if the backend evaluates getUserRole($_SESSION['username']) === 'admin'. Furthermore, the backend AJAX handler (CloudAdmin_handle_ajax) immediately intercepts incoming POST requests, halting execution if the session lacks the administrator role or if the strictly validated Cross-Site Request Forgery (CSRF) token (hash_equals($_SESSION['myCloud_csrf_token'], $_POST['csrf_token'])) is absent or mismatched.

Lexical Configuration Parsing and State Machine Manipulation Traditional PHP administrative panels often manage configuration files by overwriting them entirely using var_export(), which destroys inline comments, formatting, and dynamic constants. To preserve the structural integrity of config.php, this module implements a custom Abstract Syntax Tree (AST) manipulation engine utilizing PHP's native token_get_all() function.

The ca_extract_var and ca_reconstruct_var functions operate as deterministic state machines (FIND_VAR, FIND_EQUALS, SKIP_UNTIL_SEMI). When the backend needs to read or write a specific configuration key (e.g., cloud_public_quotas), the state machine iterates through the tokenized PHP code. It locates the T_VARIABLE matching the key, safely steps over T_WHITESPACE and T_COMMENT tokens, and extracts or replaces the assigned value precisely up to the terminating semicolon. This sophisticated approach allows the configuration file to maintain dynamic PHP expressions (such as __DIR__ . '/data') rather than forcing all variables into static string literals.

Atomic File Operations and Syntax Verification To prevent the application from encountering fatal parse errors due to corrupted configuration writes, the module implements a robust atomic write sequence within the CloudAdmin_atomic_write_vars function.

1. The modified configuration string is written to a temporary file (tempnam()).
2. The backend initiates a strict syntax validation check against this temporary file, utilizing either the built-in php_check_syntax() function or a CLI sub-process executing php -l.
3. If a syntax error is detected, the temporary file is immediately unlinked, the operation aborts, and a "Syntax Error" status is returned to the client.
4. If validation passes, the system acquires an exclusive, blocking filesystem lock (flock($fp, LOCK_EX)) on the live config.php file, truncates its length to zero (ftruncate), rewrites the validated content, flushes the I/O buffer, and releases the lock.
5. Finally, opcache_invalidate() is triggered to ensure the PHP Zend Engine immediately compiles the new configuration, preventing stale bytecode from being served to active user sessions.

Frontend State Management and Dirty-Tracking The frontend operates on a complex memory state object (ca_State), tracking users, available roles, and the configuration schema. To protect administrators from accidental data loss, the module implements aggressive "dirty state" tracking (ca_Is_Dirty, ca_Is_Dirty_Cfg). Every <input>, <select>, and <textarea> within the administrative DOM is bound to oninput and onchange event listeners.

When a value is modified, the JavaScript engine compares the current DOM value against a cached data-orig attribute. If a deviation is detected, the specific key is pushed into a Set object (ca_Dirty_Cfg or ca_Dirty_Users), and CSS classes (ca-dirty-border) are dynamically injected to highlight the unsaved field. A beforeunload event listener is bound to the window object, hooking into the browser's native navigation controls to physically prevent the user from closing the tab, refreshing the page, or navigating backward if the dirty Set size is greater than zero.

Dynamic UI Generation and Drag-and-Drop Mechanics The "Cloud Configuration" tab dynamically generates its layout based on a strict $schema array defined in the backend. This schema categorizes variables, defines their expected datatypes (booleans, scalars, arrays), and dictates the rendering of specific UI components (e.g., toggle switches for booleans, wide text inputs for long strings).

Within the "User Management" tab, administrators assign physical directory paths to user accounts. This interface utilizes the HTML5 Drag-and-Drop API to allow administrators to reorder cloud mounts visually. The ca_get_drag_after_element function executes high-frequency geometric calculations during the dragover event, computing the bounding rectangles (getBoundingClientRect()) of all sibling rows to determine the precise Y-axis insertion point based on the user's cursor coordinates, ensuring a fluid, native-feeling sorting mechanism. Furthermore, any paths assigned to users are strictly validated against PHP's open_basedir restrictions on the backend to prevent administrators from accidentally or maliciously mounting unauthorized system directories (e.g., /etc or /var/log).

19.2 The Workflow

The operational flow of the Administrative UI manages data ingestion, user manipulation, and system configuration through the following distinct phases:

- **Phase 1: Initialization and Data Ingestion**
    - The administrator switches to the Admin tab. The ca_init() JavaScript function fires, exposing the loading spinners.
    - An AJAX POST request is dispatched with the load action.
    - The backend securely reads the user database and parses config.php via lexical tokenization, referencing the $schema to categorize the raw PHP variables.
    - The backend returns a serialized JSON payload containing the current user objects, available RBAC roles, and the structured configuration array.
    - The frontend receives the payload, populates the ca_State object, and invokes DOM rendering functions to build the sidebar, user cards, and configuration grids.

- **Phase 2: User Account Manipulation**
    - The administrator selects a user from the sidebar list. The ca_load_user() function extracts the user object from ca_State and populates the form fields.
    - If the administrator modifies the password, the ca_validate_password() routine enforces length and complexity algorithms locally. Concurrently, the ca_is_pwned() function securely queries the HaveIBeenPwned API to block the assignment of compromised credentials.
    - The administrator adds a new Cloud Mount, providing a directory path and permission level.
    - The ca_mark_dirty() observer flags the user as unsaved, triggering the appearance of the floating "Save User" button.
- **Phase 3: Global Configuration Editing**
    - The administrator switches to the "Cloud Configuration" tab. A secondary warning intercepts the switch if user data remains unsaved.
    - The configuration UI renders, utilizing the ca_filter_config() function to allow instantaneous, client-side regex-based searching across variable names, descriptions, and current values.
    - The administrator alters a setting (e.g., adjusting the session timeout integer or toggling a WAF boolean). The ca_mark_dirty_cfg() function highlights the altered fields and tracks the modified keys.
- **Phase 4: Validation, Commit, and Atomic Write**
    - The administrator clicks the "Save" action.
    - The frontend serializes the modified DOM states back into a JSON payload and dispatches it via a fetch POST request.
    - **For Users:** The backend validates any new cloud paths against open_basedir, updates the user array, hashes any new passwords via Argon2id, and executes an atomic lock-and-write on the user database.

- **For Configurations:** The backend iterates through the payload, sanitizing strings and identifying valid PHP expressions. It executes the AST token replacement on the configuration file content in RAM, performs a php -l syntax check on a temporary file, and permanently writes the data via an exclusive lock, followed by an OPcache invalidation.
- The backend responds with an OK status. The frontend clears all dirty tracking Sets, removes visual warning borders, and silently re-initializes the UI to reflect the newly committed server state.

19.3 Usability

From the perspective of a system administrator, the Administrative UI functions as a highly responsive, stress-free control center that eliminates the traditional risks associated with managing PHP applications. Historically, altering core web application parameters required accessing the server via SSH or FTP, downloading a config.php file, carefully editing code strings in a text editor, and hoping a misplaced semicolon or deleted quotation mark didn't bring the entire platform offline with a fatal parse error. This module entirely abstracts that danger.

When an administrator opens the Cloud Configuration tab, they are presented with a clean, heavily documented graphical interface. Complex arrays and boolean flags are reduced to intuitive toggle switches and categorized text fields. If an administrator needs to locate a specific security parameter among dozens of settings, they simply begin typing in the search bar at the top of the screen. The interface reacts instantly, collapsing irrelevant categories and highlighting matching variables, descriptions, or values in real-time, functioning much like the settings search in a modern operating system.

The application aggressively protects the administrator from making operational errors. If an administrator alters a user's permissions, changes a setting, and then accidentally attempts to close the browser tab or hit the "Back" button, the interface physically arrests the navigation. A stark warning dialog appears, notifying them that they have unsaved changes and forcing them to confirm the abandonment of their work. Similarly, if they click away to a different tab within the application, a custom modal interrupts them, clearly listing exactly which variables they forgot to save. As they edit fields, the specific inputs develop a subtle, colored border, acting as a visual

breadcrumb trail of exactly what has been altered during the current session.

Managing user access is equally tactile and robust. To assign a new storage directory to a user, the administrator clicks "Add Mount," types the absolute path of the directory, and selects the permission level from a dropdown menu. If they type a path that violates the server's strict security boundaries, the application immediately refuses the save operation and provides a clear error message, preventing potential security breaches. Organizing these mounts is handled entirely via mouse or touch input; the administrator simply grabs a row by its handle icon and drags it up or down to set the priority order. Furthermore, when resetting a user's password, the administrator benefits from the same enterprise-grade security checks enforced on the frontend; the system will actively refuse to save the profile if the administrator attempts to assign a weak or publicly breached password to the account, ensuring that the platform's security posture remains uncompromised from the top down.

20. Cronjob Housekeeping

20.1 Technical Description

The Housekeeping module, encapsulated within cronjobs.php, operates as a detached, high-performance background maintenance engine designed to be executed via server-side CRON schedules or manual CLI (Command Line Interface) invocation. It strictly enforces a php_sapi_name() === 'cli' check, terminating immediately if accessed via standard HTTP requests, thereby neutralizing remote execution vulnerabilities.

The primary directive of the myCloudHousekeeper class is to perform asynchronous data management tasks that are computationally prohibitive for synchronous web requests. These tasks encompass bulk thumbnail generation, aggressive cache pruning, deep search indexing via Recoll, and systemic data retention enforcement (Recycle Bin and temporary file purging).

Concurrency and Process Management To handle datasets exceeding hundreds of thousands of files efficiently, the module implements true process-level parallelism utilizing the PCNTL (Process Control) extension. Upon initialization, the engine enumerates the server's logical CPU cores via /proc/cpuinfo or the nproc binary. It establishes the $maxWorkers limit to N-1 (leaving one core free for system stability).

As the primary thread traverses the filesystem, it pushes thumbnail generation tasks into a $batchJobs array. Once the array reaches the $batchSize limit (default 50), the engine invokes pcntl_fork(). The parent process records the child's Process ID (PID) in a tracking array and resumes scanning. The child process detaches, consumes the batch array, performs the heavy image manipulation, and then commits suicide via a SIGKILL signal (posix_kill(getmypid(), SIGKILL)). This aggressive termination strategy prevents PHP shutdown hooks or garbage collection routines from corrupting shared memory or causing double-free segfaults in the Imagick extension.

To safeguard system performance during these massive I/O operations, the primary script explicitly demotes itself to the lowest possible CPU priority utilizing @pcntl_setpriority(19, getmypid(), 0).

Locking and Stalled Process Detection Because CRON jobs may overlap if a scan takes longer than the scheduled interval, the module implements a highly robust mutual-exclusion lock (acquireLock()). It attempts to acquire a

non-blocking LOCK_EX | LOCK_NB on a system temporary file (My-Cloud_Housekeeper.lock). If successful, it writes its PID to the file.

If the lock fails, it signifies another instance is running. The engine then engages a stalled-process detection algorithm. It calculates the delta between the current Epoch time and the lock file's modification time (mtime). The active instance constantly updates this mtime via the updateHeartbeat() method. If the delta exceeds the $stalledThreshold (10 minutes), the engine assumes the previous instance crashed or hung. It reads the PID from the lock file, verifies its existence via posix_kill($otherPid, 0), and unconditionally executes a SIGKILL to reap the zombie process before seizing the lock and initiating a new run.

Hybrid RAM/Disk Orphan Detection One of the most complex tasks is purging obsolete thumbnails from the cache directories ($iconCachePath, $previewCachePath) after a user deletes or moves the original files. Iterating through hundreds of thousands of cache files and performing a file_exists() disk I/O check against the source directory for *every single file* would cripple server performance.

To optimize this, the engine employs a Hybrid RAM/Disk strategy. During the Phase 1 source scan, it builds an immense $validSourcePaths associative array (hash map) in RAM, utilizing the relative file paths as keys. During the Phase 2 cache scan, it strips the _thumb.jpg suffix from the cache file and performs an O(1) memory lookup (isset($this->validSourcePaths[$originalSafePath])). If the key is missing, the cache file is orphaned and instantly unlinked.

If the server possesses limited RAM and the scan hits a safety threshold (800MB), the engine dynamically dumps the array to prevent an Out-Of-Memory (OOM) fatal error, setting $memoryLimitReached = true. The Phase 2 scanner then gracefully degrades to the slower, disk-based file_exists() check, ensuring completion regardless of hardware constraints.

Media Processing Pipeline (Imagick vs. GD) The image processing engine (generateImage()) prioritizes the Imagick extension for superior color handling and performance, falling back to GD if absent.

- **Orientation and Color Space:** The engine utilizes $im->autoOrient() to bake EXIF rotation data into the pixel matrix natively. It actively checks the ICC profile; if it detects a CMYK colorspace (common in print media), it mathematically forces a conversion to

Imagick::COLORSPACE_SRGB to prevent severe color washing and neon distortion when rendered in web browsers.

- **Resizing:** The engine utilizes $im->thumbnailImage() with the $bestfit flag set to true, calculating the geometric ratio dynamically to ensure the image scales down without distorting or padding the aspect ratio.
- **Atomic Writes:** Image artifacts are written to a .tmp file. Once the write completes successfully, the engine executes a rename() to swap the temporary file to the final destination. This guarantees that if the server crashes mid-write, a corrupted, half-rendered JPEG is not cached.

Video Frame Extraction For video files (mp4, webm, mov, mkv, avi), the engine acts as an FFmpeg bridge. It executes a CLI command (ffmpeg -y -ss 00:00:01 -i [source] -vframes 1 ...) to seek rapidly to the 1-second mark and extract a single frame to the temporary directory. This extracted frame is then piped into the standard image processing pipeline as a JPEG, allowing seamless video thumbnail generation without specialized PHP libraries.

Search Indexing (Recoll) The module automates the maintenance of the deep-content search engine. It iterates through the permitted cloud roots, dynamically generating a recoll.conf file within a hidden .recoll directory. This configuration dictates the top-level directory and explicitly maps un-indexable binary extensions (.exe, .dll, .mp4) to the noContentSuffixes variable to prevent the indexer from wasting CPU cycles attempting to extract text from media files. Finally, the engine spawns a shell process (ionice -c 3 nice -n 19 recollindex...) utilizing extreme I/O and CPU demotion flags to compile the Xapian database silently in the background.

Database Pruning and Temp Cleanup The engine actively manages the lifecycle of the flat-file JSON databases (shares.json, tickets.json). It acquires an exclusive lock, reads the arrays, and filters out any objects where the expire timestamp has passed the current Epoch. If the array size shrinks, it truncates and rewrites the file, keeping the database lean. Concurrently, it scans the system's temporary directory (sys_get_temp_dir()), matching filenames against a strict regex pattern (/^(ce_dl_|cloudex_prog_|myCloud_dl_...)/). Any temporary files or abandoned OCR extraction directories older than 24 hours are unlinked.

20.2 The Workflow

The execution of the cronjobs.php script follows a rigid, highly optimized linear path:

1. **Initialization:**
    - Script execution begins via CLI invocation.
    - myCloudHousekeeper class instantiates, registering asynchronous signal handlers (SIGINT, SIGTERM) to allow graceful interruption.
    - CPU cores are counted, and the PCNTL worker limit is established.
2. **Lock Acquisition:**
    - The acquireLock() method fires.
    - If a lock exists, the engine evaluates the heartbeat. If stalled, it murders the zombie process and seizes control.
    - Execution priority is lowered to nice 19.
3. **Database and System Cleanup (Pre-Scan Phase):**
    - If the $doRecycler flag is active, the engine traverses all .recycle_bin directories across all roots. It evaluates the .meta files against the configured $retentionDays limit and recursively obliterates expired datasets.
    - Temporary system files generated by FFmpeg, Ghostscript, or orphaned download tokens are purged.
    - The shares.json and tickets.json databases are pruned of expired records.
4. **Search Index Compilation:**
    - If the $doSearchIndex flag is active, the engine updates recoll.conf files.
    - Background recollindex processes are spawned with ionice constraints.
5. **Source Traversal (Phase 1):**

- If the $doCache flag is active, the processSourceDirectory method initiates a RecursiveIteratorIterator.
- For every valid file, the script evaluates the modification time (mtime) against existing cache files.
- If generation is required, the job is pushed to the $batchJobs array.
- The relative path is stored in the $validSourcePaths RAM map.

6. **Parallel Processing:**
   - When the batch limit is hit, pcntl_fork() spawns a child process.
   - The child process iterates through the batch, executing generateImage() for icons and previews.
   - The parent thread monitors worker capacity via checkWorkers() and continues scanning.
7. **Cache Cleanup (Phase 2):**
   - After the source scan finishes, the parent thread waits for all children to die (waitForWorkers()).
   - The script traverses the $iconCachePath and $previewCachePath.
   - Each cache file's relative path is checked against the $validSourcePaths RAM map. If missing, the orphan is unlinked.
8. **Termination:**
   - The execution timer is calculated and logged.
   - The MyCloud_Housekeeper.lock file is released and deleted, allowing the next scheduled CRON job to execute.

20.3 Usability

While the Housekeeping module possesses no graphical user interface, its silent operation is absolutely critical to the platform's usability, speed, and storage efficiency. To a server administrator, this script represents a "set and forget" maintenance layer. By simply adding a single line to the server's crontab (e.g., 0 * * * * php /path/to/cloud/cronjobs.php), the entire cloud infrastructure becomes self-healing and self-optimizing.

From the end-user's perspective, the effects of this module are profound, primarily manifesting as extreme interface speed. When a user uploads 500 high-resolution photographs, they do not have to wait for the web server to painfully generate 500 thumbnails before they can see their files. The upload finishes instantly, and the Housekeeping script, running quietly in the background on spare CPU cycles, churns through the new files, generating the optimized thumbnails and preparing them for the next time the user opens the Gallery View.

Furthermore, the module guarantees that the cloud does not slowly choke on its own digital exhaust. If a user deletes a folder, they don't need to manually hunt down and delete the cached thumbnails associated with those images; the Housekeeping script detects the missing original files and instantly wipes the orphaned cache data, reclaiming precious server hard drive space. It performs the same efficiency on the Recycle Bin. Users can delete files knowing they have a safety net, but the server administrator knows that the hard drive won't fill up forever; the moment a file sits in the bin past the 7-day retention limit, the Housekeeping script vaporizes it.

Even the deep search functionality relies entirely on this module. As users add, edit, and move documents throughout the day, the Housekeeping script quietly recompiles the Xapian database indexes in the background. Because it uses specialized commands to limit its own CPU and hard drive access, it never slows down the active web server. The users simply experience the magic of typing a keyword into the search bar and instantly finding a document based on a paragraph buried on page 42, entirely unaware of the massive, parallel-processing engine working tirelessly behind the scenes to make it happen.

21. Core File Operations and Transfer Logic

21.1 Technical Description

The Core File Operations module dictates the mechanisms by which data moves between the client's local machine and the server's storage arrays, as well as how data is manipulated internally within the cloud filesystem. This module is primarily orchestrated by the assets.js.core_engine.php JavaScript file on the frontend and the corresponding actionUpload, actionDelete, actionCopyMove, and download routing functions within controller.server.php.

The upload architecture abandons the modern fetch API in favor of the legacy XMLHttpRequest (XHR) object. This deliberate engineering choice is due to XHR's native support for the upload.onprogress event listener, which is mathematically necessary to provide users with highly accurate, real-time byte-transfer percentage calculations. When a user initiates an upload, the system generates a FormData payload. To prevent arbitrary file overwrite vulnerabilities, the frontend executes a pre-flight collision check (myCloudState.allItems). If a filename collision is detected, execution halts, and a resolution prompt is triggered. If the target directory is an End-to-End Encrypted (E2EE) vault, the file payload is intercepted before the XHR transmission. The JavaScript WebCrypto API ingests the raw File object, encrypts the buffer using the vault's Data Encryption Key (DEK), and replaces the plaintext File object in the FormData payload with the resulting ciphertext Blob and obfuscated filename.

For batch file manipulations (copying, moving, deleting), the frontend utilizes the myCloudBatchProcess() orchestrator. Rather than dispatching fifty simultaneous asynchronous requests—which could trigger the backend's rate-limiting Web Application Firewall (WAF) or exhaust PHP-FPM worker pools—this orchestrator enforces strict sequential execution. It wraps the asynchronous fetch calls in a Promise chain, ensuring the next file operation only begins after the previous one successfully returns an HTTP 200 OK status.

A unique architectural challenge arises when a user attempts to move a file *between* two differently encrypted vaults, or between an encrypted vault and a plaintext directory. The server cannot perform this move natively because it lacks the encryption keys. The system solves this via the myCloudE2ETransfer orchestrator. This routine detects a "Cryptographic

Boundary Crossing." It downloads the source ciphertext to the client's RAM, decrypts it using the source vault's DEK, re-encrypts the buffer using the destination vault's DEK (or leaves it as plaintext), uploads the new payload to the destination, and finally commands the server to execute a permanent deletion of the source file. This complex procedure maintains absolute zero-knowledge security on the server while preserving data mobility.

Batch downloading utilizes advanced modern web capabilities. If the user selects multiple files and requests a download, the system attempts to invoke the native File System Access API (window.showDirectoryPicker()). This prompts the user for a local destination folder once, and then the JavaScript engine streams the decrypted Blobs directly into the local filesystem sequentially. If the browser lacks support for this modern API, the system falls back to generating ephemeral Blob URLs (URL.createObjectURL()) and programmatically clicking hidden <a> anchor tags to force the browser's legacy download manager to intercept the files.

21.2 The Workflow

The complete lifecycle of a file transfer or manipulation operation follows this strict execution sequence:

- **Ingestion and Collision Evaluation (Uploads):**
    - The user drags an array of files into the browser viewport, triggering the drop event listener.
    - The myCloudScanItems() function utilizes the webkitGetAsEntry API to recursively unpack local directory structures into a flat array of file objects.
    - For each file, the system checks the active directory's memory state to detect naming collisions.
    - If a collision occurs, a modal interrupts the loop, forcing the user to select overwrite, keep_both, or skip. If keep_both is selected, an incrementing integer is appended to the filename.
- **Cryptographic Interception:**
    - The system evaluates the target directory against the myCloudState.encryptedDirs registry.
    - If encrypted, the plaintext File object is converted to an ArrayBuffer.

- The WebCrypto API encrypts the buffer, generating a ciphertext Blob and an .enc appended filename.

- **Transmission and Progress Tracking:**
  - An XMLHttpRequest is instantiated. The myCloud_token (CSRF), active directory path, and file Blob are appended to a FormData object.
  - The upload.onprogress listener fires continuously, updating the DOM progress bar with the formula (e.loaded / e.total) * 100.
  - The backend actionUpload receives the payload, passes the filename through the sanitizeAndValidateName WAF jail, and executes move_uploaded_file().
- **Cross-Boundary E2E Transfer (Move/Copy):**
  - The user commands a file move. The frontend evaluates get-CryptoRoot(source) against getCryptoRoot(destination).
  - If the roots differ, myCloudE2ETransfer initializes.
  - The client fetches a short-lived download token for the source file and downloads the ciphertext Blob into RAM.
  - The client decrypts the Blob.
  - The client encrypts the Blob using the destination's crypto-graphic parameters.
  - The client POSTs the new ciphertext to the destination.
  - The client POSTs a delete command targeting the original source file.
- **DOM Reconciliation:**
  - Upon receiving the OK status from the backend, the frontend tears down the progress UI.
  - The myCloudSurgicalRemove() function locates the specific DOM nodes of the moved/deleted files and destroys them in-stantly to provide immediate visual feedback.

- A silent, background myCloudFetchDirectory() request updates the memory state to ensure total synchronization with the server's physical disk.

21.3 Usability

The file transfer logic is engineered to make complex, highly secure cloud operations feel indistinguishable from managing files on a local hard drive. When a user wants to back up a project, they simply highlight a group of folders on their computer desktop and drag them directly into the browser window. The interface immediately reacts, highlighting the drop zone in blue. Upon releasing the mouse, a sleek progress indicator appears at the bottom of the screen, providing granular, real-time percentage updates so the user knows exactly how long the transfer will take.

Crucially, the system protects the user from catastrophic data loss without being overly intrusive. If the user accidentally drags a file into a folder where a file with the identical name already exists, the application immediately halts that specific file's transfer. A clean, localized dialog box appears displaying the conflicting filename, offering the choice to overwrite the old file, skip it, or keep both. If the user chooses to keep both, the system intelligently renames the incoming file (e.g., appending "(1)") and seamlessly continues the batch upload.

The most impressive usability achievement of this module is its handling of End-to-End Encryption. To the user, dragging a file into a "Secure Vault" looks and feels exactly the same as dragging it into a standard folder. The complex mathematical operations—converting the file to binary, generating initialization vectors, and encrypting the payload—happen so efficiently in the computer's memory that the upload process appears to proceed normally. Even the incredibly complex act of moving a file between two differently encrypted vaults is reduced to a simple drag-and-drop action. The user sees a slightly different progress bar noting an "E2E Transfer," while the browser does the heavy lifting of downloading, decrypting, re-encrypting, and uploading the file in the background, entirely masking the immense technical complexity required to maintain their absolute privacy.

22. User Identity and Access Management

22.1 Technical Description

The User Identity and Access Management (IAM) module forms the core authorization perimeter of the application. It governs how identities are provisioned, authenticated, and subsequently restricted within the application's physical and graphical boundaries. The module relies on a hybrid flat-file database ($user_db) managed by the main_login.php controller and enforced globally across both the frontend UI and the backend PHP execution threads.

Identity Provisioning and the Flat-File Database User accounts and their associated parameters are not stored in a SQL database. They are serialized into raw PHP arrays ($users and $user_details) within a strictly protected file that resides outside the web server's public document root. The $users array maps the plaintext username string directly to an Argon2id password hash. The $user_details array contains a corresponding object for that username, defining their global role (admin, cloud, or user), their email address for 2FA routing, a boolean flag enabling or disabling WebDAV protocol access, and a cloud array.

The cloud array is the foundation of the platform's multi-tenant capabilities. It defines discrete "Mount Points" for the user. Each mount point object contains a physical server path (e.g., /mnt/storage/marketing_team), a default interface string (e.g., gallery or default), and a strict permission string (full, modify, edit-only, read-only, or admin_mode). This architecture allows a single identity to access multiple, completely segregated storage volumes with different clearance levels on each volume.

The Role-Based Access Control (RBAC) Matrix Authorization enforcement is centralized around the $MYCLOUD_RIGHTS_MATRIX. This matrix explicitly maps permission strings to an array of blocked operational commands. By default, the system operates on a denylist principle for high-level roles and an allowlist principle for restrictive roles. For example, a user assigned the read-only role on a specific cloud mount will have their matrix profile set to ['blocked' => '*'], meaning the backend will mathematically reject any POST request aiming to execute file manipulations (uploads, deletes, renames). A user with the modify role will have a matrix that blocks destructive structural commands (like altering the cloud configuration or executing remote SSH commands) but permits standard filesystem I/O.

Frontend UI Reflection (myCloudActionAllowed) The security posture of the backend is intrinsically linked to the frontend rendering engine. It is considered poor UX to present a user with an "Upload" button, only to throw a "Permission Denied" error when they click it. To solve this, the JavaScript payload initializes the myCloudActionAllowed(action, currentRight) function.

During the DOM construction phase (e.g., generating the top toolbar or building a right-click context menu), every single UI component is evaluated against this function. The function interrogates the MYCLOUD_RIGHTS_MATRIX loaded into the browser's memory. If the matrix dictates that the current user's role on the active cloud tab blocks the delete action, the rendering engine entirely skips the creation of the "Delete" button. The HTML node simply is never injected into the Document Object Model, completely removing the vector for unauthorized interaction.

22.2 The Workflow

The identity management and authorization pipeline operates continuously across the administrative and user lifecycles:

- **Identity Provisioning (Administrator Action):**
    - An administrator accesses the Native Admin UI and clicks "+ New User".
    - The administrator defines the username, inputs a password, assigns a global role, and maps one or more physical directory paths (Cloud Mounts) to the account, selecting a specific permission tier for each.
    - The payload is POSTed to the save_user endpoint.
    - The backend password_hash() function converts the plaintext password to an Argon2id hash.
    - The backend validates that all assigned directory paths comply with the server's open_basedir restrictions.
    - The CloudAdmin_atomic_write_vars function executes a locking AST token replacement, permanently writing the new identity into the $user_db PHP arrays.
- **Authentication and Role Extraction:**

- The user authenticates via main_login.php.
- The backend queries the $user_details array, extracting the user's defined Cloud Mounts and global role into the active session variables.
- The user is redirected to the main application interface.

- **Contextual Capability Rendering (Frontend):**
  - The UI engine assesses the active cloud tab (e.g., the user switched from their "Private" cloud to the "Corporate" cloud).
  - The engine extracts the specific permission tier assigned to that tab (e.g., downgrading from full to read-only).
  - myCloudUpdateToolbarState() executes, hiding all editing, uploading, and deletion buttons from the interface.
- **Backend Gatekeeping (Execution):**
  - A malicious user attempts to bypass the frontend UI by manually dispatching a myCloud_action=delete POST request via the browser console.
  - The controller.server.php router intercepts the request.
  - It looks up the active cloud key in the user's session configuration, extracts the read-only role, and evaluates it against the $MYCLOUD_RIGHTS_MATRIX.
  - The backend detects the restriction, immediately aborts execution, and returns {"status":"ERR", "msg":"Permission denied"}.

22.3 Usability

The User Identity and Access Management system is designed to provide unparalleled flexibility for administrators while ensuring a perfectly tailored, confusion-free experience for the end-user.

From an administrative perspective, managing a team is remarkably straightforward. Through the visual administrative dashboard, an IT manager can create a new employee account in seconds. They can grant this employee full read and write access to their personal "Home" folder, but simultaneously mount the "Company Policies" folder to their account with strict "Read-Only" permissions. The administrator does not need to configure

complex operating system-level file permissions or manage confusing active directory groups; the entire access control structure is managed via simple, intuitive dropdown menus in the web interface.

From the end-user's viewpoint, the system's security mechanisms are completely invisible. The user simply logs in and sees their assigned folders as tabs at the top of the screen. When they click on their personal "Home" folder, the application blooms with capability: upload buttons appear, the drag-and-drop zone activates, and right-clicking files offers a plethora of editing and manipulation tools.

However, when that same user clicks over to the "Company Policies" tab, the application gracefully adapts. The upload buttons vanish. The "New Folder" and "Delete" icons disappear from the toolbar. The interface becomes a pristine, streamlined viewing gallery. The user is never frustrated by clicking a button only to be slapped with a harsh "Access Denied" error message; the application simply removes the tools they are not allowed to use, fostering a safe, predictable environment where users cannot inadvertently damage corporate data or violate security policies.

23. System Logging, Telemetry, and Diagnostics

23.1 Technical Description

The System Logging, Telemetry, and Diagnostics module is an essential administrative framework that provides deep visibility into the application's operational state, security perimeter, and user activity. It operates asynchronously to prevent I/O bottlenecks and segregates data into discrete flat-file ledgers.

Backend Auditing and Log Segregation The backend logging infrastructure is divided into two primary streams to separate security/authentication events from standard filesystem operations:

- **The Authentication and Security Ledger ($log_file):** This stream captures perimeter events. It is invoked during login attempts, WAF (Web Application Firewall) interceptions, and cryptographic state changes. The WriteLogLine() function formats these entries, appending severity tags (e.g., success, warning, error, info). For instance, if the WAF blocks a malicious payload, it records the exact ruleset triggered (WAF 1 block - Score: X). It also logs k-Anonymity breach detections (HIBP API), CSRF token mismatches, and Auto-Login (App Mode/PWA) authentications.
- **The Cloud Operational Ledger ($cloud_logfile):** This stream acts as a granular audit trail for data manipulation. Within controller.server.php, the log() method records every discrete filesystem action (UPLOAD, DELETE, MOVE, CRYPTO_INIT, RESTORE). The log entry is structured as a tab-separated string containing the Epoch timestamp, the active username, the active cloud key, the action performed, the source path, the destination path, and the outcome (OK or ERR). This format ensures the log can be effortlessly parsed by external SIEM (Security Information and Event Management) tools like Splunk or ELK stacks.

Frontend Telemetry and Hidden Diagnostic Capabilities Beyond standard error catching in the browser console, the frontend implements a unique, hidden diagnostic and telemetry routine (_sys_diag_init()). Obscured behind a double-click event listener attached to the main application SVG logo

within the Cloud Switcher (myCloudCloudSwitcher), this routine injects a high-priority, full-screen canvas overlay.

While visually disguised as a simple block-falling game (Tetris clone), it serves as a robust, client-side hardware and event-loop diagnostic tool. It forces the browser to instantiate a CanvasRenderingContext2D, establish an aggressive setInterval timing loop (400ms), and bind overriding keydown event listeners to test input latency, rendering pipeline health, and Javascript execution thread stability under simulated load.

23.2 The Workflow

The lifecycle of an audited event follows a structured pipeline:

- **Event Triggering:**
  - A user or system process initiates an action (e.g., a failed login, an SFTP directory sync, or a file deletion).
  - The corresponding controller method intercepts the action outcome.
- **Context Assembly:**
  - The logging function (WriteLogLine or $this->log) gathers environmental context: the current Unix timestamp (date('Y-m-d H:i:s')), the remote IP address, the authenticated username, and the target file paths.
- **I/O Execution:**
  - The system attempts to open the target log file.
  - An exclusive lock (LOCK_EX) is requested to ensure atomic writes in a highly concurrent, multi-user environment.
  - The formatted string is appended to the file (FILE_APPEND).
  - The lock is released, and the primary PHP execution thread resumes.
- **Client-Side Diagnostic Trigger (Optional):**
  - The user rapidly double-clicks the application logo.
  - The DOM immediately injects a fixed z-index: 2147483647 overlay.

- The diagnostic script tests canvas rendering, garbage collection (clearInterval), and multi-key input handling, terminating upon pressing the 'Escape' key.

23.3 Usability

For system administrators and security officers, the logging module is the definitive source of truth. Because the application strictly segregates authentication logs from file-operation logs, administrators do not have to sift through thousands of routine "file downloaded" entries to investigate a potential brute-force attack on a user account. If an employee claims they accidentally deleted a critical, encrypted folder, the administrator can parse the $cloud_logfile, identify the exact timestamp of the RECYCLE or DELETE event, and trace the file's original path to facilitate a swift recovery.

The hidden diagnostic tool provides a unique, frictionless method for troubleshooting client-side browser issues. If a user reports that the application interface feels "sluggish" or that keyboard shortcuts are not responding, the support technician can instruct the user to double-click the logo. The resulting diagnostic overlay immediately confirms whether the browser's hardware-accelerated canvas is functioning and whether the keyboard event listeners are being successfully intercepted by the DOM, effectively isolating network latency issues from local browser rendering bottlenecks.

24. Security Architecture and Web Application Firewall

24.1 Technical Description

The Security Architecture module is a comprehensive, defense-in-depth framework integrated natively into the application's routing logic. It encompasses a localized Web Application Firewall (WAF), strict Content Security Policies (CSP), and advanced cryptographic state validation to protect the application from both automated botnets and targeted exploitation.

The Localized Web Application Firewall (WAF) Before the main_login.php script processes any authentication payload, it instantiates the ClientSecurity class. This WAF operates entirely in-memory and evaluates the incoming request against a configurable array of threat vectors:

- **Flood Control & Rate Limiting:** Measures the velocity of requests from specific IP subnets to prevent Layer 7 Denial of Service (DoS) attacks.
- **HTTP & Referrer Checks:** Validates the structural integrity of the HTTP headers, ensuring requests originate from permitted domains and rejecting malformed or spoofed User-Agents.
- **Geo-IP & ASN Checking:** Utilizes MaxMind databases to cross-reference the origin IP against geographical and Autonomous System Number (ASN) blacklists, instantly dropping connections from known Tor exit nodes, anonymous proxies, or restricted nation-states.
- **Keyword & Payload Inspection:** Scans POST and GET bodies for SQL injection patterns, XSS vectors, and path traversal strings (../), neutralizing them before they reach the core business logic. If the WAF calculates a threat score exceeding the permitted threshold, it aborts the PHP thread and utilizes JavaScript window.location.replace to trap the attacker in a continuous sinkhole redirect loop.

Content Security Policy (CSP) and Header Hardening The application strictly defines its execution boundaries via HTTP response headers. The CSP (default-src 'self'; script-src 'self' https: 'unsafe-inline' 'unsafe-eval';) guarantees that the browser will only execute scripts originating from the application's own domain or explicitly permitted external CDNs (like OnlyOffice). It absolutely prohibits the execution of unauthorized inline scripts injected via XSS, and blocks the embedding of the application within malicious external <iframe> tags via frame-ancestors 'self'.

Cross-Site Request Forgery (CSRF) Mitigation and Auto-Recovery Every state-mutating action within the application—from logging in, to deleting a file, to opening an encrypted vault—requires a 64-byte hexadecimal CSRF token ($_SESSION['myCloud_csrf_token']). The frontend injects this token into every fetch and FormData payload. The backend evaluates it using hash_equals() to prevent timing attacks.

Because the application relies heavily on Single Page Application (SPA) mechanics, a user might leave a tab open for days. When their session token expires, their next action would normally fail catastrophically. To solve this, the frontend overwrites the native window.fetch API. If a request fails due to a CSRF_FAILED error, the JavaScript engine intercepts the failure, silently pauses the user's action, dispatches a request to the refresh_csrf endpoint, retrieves a freshly minted token, injects it into the original payload, and re-executes the user's action seamlessly.

24.2 The Workflow

- **Request Interception:**
    - An HTTP request hits the web server.
    - The main.php bootloader applies CSP, X-Content-Type-Options: nosniff, and Permissions-Policy headers.
- **WAF Evaluation:**
    - The ClientSecurity object compiles the client's IP, headers, and payload into an evaluation matrix.
    - Rulesets (GeoIP, Keyword, ASN) process the matrix.
    - If flagged, the status returns BLOCK, generating a secure log entry and redirecting the attacker.
- **Token and Signature Validation:**
    - For authenticated actions, the inbound myCloud_token is compared against the session.
    - For App Mode (PWA) logins, the $expected_app_mode HMAC signature is recalculated using the client's IP, User-Agent, and token selector. If it matches the inbound signature, the login form is bypassed.
- **Execution or Auto-Recovery:**

- If validation passes, the controller executes the file operation.
- If CSRF validation fails during an AJAX request, the backend returns a specific CSRF_FAILED JSON code. The frontend's originalFetch wrapper halts, negotiates a new token, and retries the exact operation without requiring user intervention.

24.3 Usability

The application's security architecture is designed to be a silent, impenetrable fortress that never interferes with the legitimate user's workflow. To the end-user, the WAF and CSP headers are entirely invisible. They simply experience a fast, responsive application.

However, the usability benefits of this underlying architecture are profound. In traditional web applications, if a user leaves a tab open overnight and attempts to upload a file the next morning, the application will crash, discard their uploaded file, and throw them back to the login screen with an "Expired Session" error. Thanks to the automated CSRF recovery engine, MyDocpile handles this scenario elegantly. The user clicks "Upload," the system detects the expired token, silently negotiates a new cryptographic handshake in the background within milliseconds, and successfully uploads the file as if nothing was wrong.

For system administrators, the WAF provides immense peace of mind. Without requiring complex firewall configurations at the server's OS level, the administrator knows that malicious bots scanning for SQL vulnerabilities or attackers attempting brute-force logins from foreign proxies are being mathematically discarded before they even touch the authentication database.

Appendix A: Emergency Decryption of E2E Encrypted Files

A.1 Technical Description

The Emergency Vault Decryptor (emergency_decrypt.php) is a standalone, Command Line Interface (CLI) script engineered for disaster recovery scenarios. It provides a highly optimized, server-side mechanism to recursively decrypt a user's End-to-End Encrypted (E2EE) vault without relying on the web-based frontend, the JavaScript WebCrypto engine, or any active session state.

The Absolute Password Constraint and Zero-Knowledge Guarantee The most critical architectural concept underpinning this module—and the entire E2EE ecosystem of the application—is that **the vault password is an absolute, mathematically irreversible prerequisite for decryption.** The system operates on a strict zero-knowledge paradigm. There are no master keys, no administrative backdoors, and no cryptographic escrow systems.

If a user loses their vault password, the data within that vault is permanently inaccessible. The server holds only the heavily iterated Key Encryption Key (KEK) derivation parameters and the wrapped Data Encryption Key (DEK). Because the DEK is encrypted using AES-256-GCM, any attempt to unwrap it without the exact KEK (derived exclusively from the user's memorized password) will cause the GCM authentication tag verification to fail, resulting in a mathematical dead end. The Emergency Decryptor script cannot bypass this; it merely relocates the cryptographic calculation from the client's browser to the server's CPU.

Cryptographic Envelope Parsing and Key Unwrapping The script exclusively targets V2 AES-GCM vaults. Execution begins by locating and parsing the .mycloud_crypto_salt file residing at the root of the specified vault directory. This file contains a Base64-encoded JSON payload housing four critical components: the vault version, the cryptographic salt, the Initialization Vector (IV), and the wrapped DEK.

To unwrap the DEK, the script replicates the browser's WebCrypto PBKDF2 derivation locally utilizing PHP's hash_pbkdf2() function. It combines the user-provided password and the decoded 32-byte salt, processing them through 600,000 iterations of the SHA-512 hashing algorithm to derive the 32-byte (256-bit) KEK. This massive iteration count aggressively penalizes brute-force attacks.

Once the KEK is derived in RAM, the script splits the wrapped DEK payload into its ciphertext and its trailing 16-byte GCM authentication tag. It utilizes openssl_decrypt() in aes-256-gcm mode. If the password provided by the user is even slightly incorrect, the derived KEK will be invalid, the authentication tag will not match the ciphertext, and OpenSSL will immediately return false, halting the script entirely.

Ciphertext Translation and Extraction If the DEK is successfully unwrapped, the script initiates a deep recursive traversal of the vault directory utilizing PHP's RecursiveIteratorIterator. The decryption process is bipartite, handling filenames and file contents independently:

1. **Filename Decryption:** Filenames in the vault are Base64Url encoded to safely traverse standard filesystem limitations. The script isolates each directory segment, identifies files bearing the .enc suffix, and reverses the URL-safe encoding (translating - and _ back to + and /, and mathematically restoring any stripped = padding). Once decoded into a raw byte array, the script extracts the prepended 12-byte IV, the ciphertext, and the appended 16-byte authentication tag. The DEK is then used to decrypt the string back into its original, plaintext filename.
2. **Content Decryption:** The script reads the physical file into a memory buffer. It performs a rigid structural validation, verifying the file is at least 44 bytes long (the absolute minimum size comprising a 16-byte legacy padding block, a 12-byte IV, and a 16-byte GCM tag). The script extracts the IV (substr($buffer, 16, 12)), the core ciphertext, and the trailing tag. It streams these components through openssl_decrypt. Upon successful tag verification and decryption, the resulting plaintext binary is written to the newly constructed, decrypted directory tree.

A.2 The Workflow

The operational execution of the Emergency Decryptor is linear and operates entirely within a terminal environment:

- **Step 1: Invocation and Parameter Binding**
    - The system administrator invokes the script via the PHP CLI, passing the absolute path to the encrypted vault, and optionally,

a target output directory: php emergency_decrypt.php /var/www/cloud/data/vault /tmp/recovered_files.

- The script validates its execution environment, terminating if invoked via a web server API (CGI/FPM).

- **Step 2: Credential Acquisition and Masking**
  - The script verifies the existence of the vault and the .mycloud_crypto_salt file.
  - It prompts the administrator for the vault password.
  - To maintain operational security and prevent the password from being logged in bash history or shoulder-surfed, the script executes a system call (stty -echo) to suppress terminal echo while the password is typed, restoring echo immediately afterward.
- **Step 3: Cryptographic Handshake**
  - The JSON configuration is decoded.
  - The CPU executes the 600,000 PBKDF2 iterations to derive the KEK.
  - The script attempts to unwrap the DEK. If the password is wrong, the script dies with a fatal error: ❌ Error: Incorrect password or corrupted vault salt.
- **Step 4: Recursive Recovery**
  - Upon successful DEK extraction, the script creates the target output directory tree.
  - It enters a foreach loop, iterating over every file and sub-directory within the vault.
  - For each file, the path is exploded, and every segment is individually decrypted to reconstruct the original plaintext directory hierarchy.
  - The physical directories are generated via mkdir().
  - The file payload is read, parsed, decrypted, and dumped into the corresponding location in the output directory.
- **Step 5: Status Reporting**

- As each file is successfully decrypted, its reconstructed path is echoed to standard output.
- Corrupted files (e.g., those truncated by an interrupted upload resulting in a size under 44 bytes) or files failing GCM authentication are skipped, flagged with a warning, and added to the failure counter.
- Upon completion, a final summary tallying successful and failed recoveries is printed to the terminal.

A.3 Usability

From the perspective of a system administrator, the Emergency Decryptor is an invaluable "break-glass" utility designed for catastrophic failure scenarios. While the primary application relies on the user's browser to handle all encryption and decryption to guarantee privacy, situations arise where the web interface is no longer viable. If the server experiences a catastrophic PHP-FPM failure, if the frontend JavaScript application becomes corrupted during an update, or if the administrator needs to migrate raw user data from a decommissioned server to a completely different storage array, relying on the web UI is impossible.

This CLI tool detaches the data recovery process entirely from the web stack. It requires no active database connection, no running web server, and no browser interactions. The administrator only needs the raw filesystem data and the PHP CLI binary.

However, the usability of this tool is strictly bound by the application's core security philosophy. When an administrator utilizes this script, they are forced to confront the reality of the zero-knowledge architecture. The administrator cannot simply point the script at a folder and extract the data; they *must* have the user present, or the user must provide them with the vault password through a secure out-of-band channel.

When the script prompts Enter Vault Password:, it is not a mere formality. The terminal actively hides the keystrokes to protect the user's privacy even from the administrator running the command. If the user has forgotten their password, the administrator is completely powerless to help them. The script will reject the attempt, and the data will remain as mathematically secure, and utterly inaccessible, ciphertext. This strict limitation reinforces trust: users can be absolutely certain that even a rogue server administrator

with direct, physical access to the server's hard drives cannot read their confidential documents without their explicit cooperation and the provision of their secret password.

Appendix B: The Heartbeat Mechanism

B.1 Technical Description

The Heartbeat module (core.heartbeat.php) is a critical, asynchronous client-side daemon designed to synchronize the user's physical presence with the backend's cryptographic session state. It serves a dual mandate: preventing premature session termination during active but non-navigational tasks, and rigorously enforcing idle timeouts to mitigate physical session hijacking on unattended workstations.

The architecture is implemented as an Immediately Invoked Function Expression (IIFE) to prevent global namespace pollution, establishing a singleton instance via the window.__heartbeatInitialized flag. The module bridges server-side configuration with client-side execution by dynamically injecting the PHP $timeout_in_minutes parameter directly into the JavaScript engine, converting it into a hard millisecond threshold (inactivityLimit).

To measure genuine user presence, the script binds passive event listeners to the document object, capturing high-frequency human interaction vectors including click, mousemove, keydown, scroll, touchstart, and touchmove. These events continually update a localized lastActivity Epoch timestamp and toggle a userInteracted boolean flag.

The communication pipeline relies on a non-blocking fetch request utilizing credentials: 'include' and cache: 'no-store' to ensure intermediate proxy servers do not cache the payload. To optimize network overhead, the heartbeat does not fire on every mouse movement. Instead, it utilizes a dual-polling strategy:

- **Scheduled Synchronization:** A hardcoded interval dispatches a pulse to the server every 180 seconds.
- **Dynamic Synchronization:** If the user interacts with the Document Object Model (DOM) after a 120-second lull, a pulse is dispatched immediately, carrying an &update=1 parameter to notify the backend session handler to slide the cookie expiration window forward.

A paramount security consideration of this module is its localized enforcement mechanism. Relying solely on the server to terminate an idle session is insufficient, as the client's browser would continue to display sensitive, cached DOM elements until the user attempted a new network request. To counter this, the Heartbeat module runs an independent

checkClientInactivity loop every 40 seconds in the browser. If this local loop detects that the lastActivity timestamp has exceeded the inactivityLimit, it executes a fatal teardown of the UI, even if the network is disconnected and the server cannot be reached.

B.2 The Workflow

The operational lifecycle of the Heartbeat system operates in a continuous, background monitoring loop:

- **Initialization and Parameter Binding:**
  - The PHP pre-processor calculates the session timeout boundary and injects the integer into the JavaScript payload.
  - The script verifies it is the sole active instance.
  - DOM event listeners are attached to the root document level to capture all bubbling user interaction events.
- **Activity Tracking:**
  - The user moves the mouse or scrolls the page.
  - The recordInteraction() function executes, updating the lastActivity memory variable to Date.now().
  - If the time elapsed since the lastHeartbeatTime exceeds 120,000 milliseconds, the script jumps immediately to the transmission phase.
- **Transmission and Server Reconciliation:**
  - The sendHeartbeat() function fires either via the 180-second setInterval or the dynamic interaction trigger.
  - An asynchronous GET request is dispatched to the backend router (?heartbeat=1).
  - The server evaluates the session token. If valid, the session garbage collector resets its timer. If the session has already been destroyed (e.g., via concurrent login termination or server-side expiration), the server responds with a JSON payload of status === 'expired'.
- **Teardown and Expulsion:**

- If the server response indicates expiration, or if the localized 40-second monitoring loop (checkClientInactivity()) calculates that the lastActivity delta exceeds the absolute timeout threshold, the system invokes handleExpiredSession().
- This termination function evaluates the current window context. If the application is running inside a nested iframe (such as the Office View or PDF previewer), it actively breaks out of the encapsulation by targeting window.parent.
- The script forces a hard document reload (location.reload()), flushing the active DOM from RAM and dumping the user back to the cryptographic perimeter of the login screen.

B.3 Usability and Security Considerations

From a usability perspective, the heartbeat mechanism operates entirely invisibly but is essential for a fluid, frustration-free workflow. When a user is reading a highly technical, fifty-page PDF document, or carefully analyzing a massive spreadsheet within the Office View, they may not click a hyperlink or trigger a full page reload for twenty or thirty minutes. In a standard web application, the server's session manager would classify this lack of HTTP requests as abandonment and silently terminate the session. When the user finally attempted to save their work or close the document, they would be greeted by a catastrophic error or thrown out of the system, resulting in lost data. The heartbeat intelligently captures micro-interactions—a simple scroll of the mouse wheel or a tap on a touch screen—and silently whispers to the server to keep the session alive, accommodating deep-focus work environments natively.

Conversely, this module acts as a ruthless and uncompromising security enforcer. The primary threat vector in enterprise cloud environments is opportunistic physical access—a user walking away from their workstation in a public office or shared environment while remaining logged in. Because the Heartbeat module tracks exact interaction times, the moment the user steps away, the countdown begins. The security architecture does not rely on the server to push a termination signal, which could be thwarted if the computer simply lost its Wi-Fi connection. Instead, the browser's own internal clock acts as a dead man's switch. The instant the local idle limit is breached, the client aggressively purges the interface. It automatically collapses any open modals, breaks out of any document preview iframes, and forcefully reloads

the page. This guarantees that any sensitive corporate data, financial records, or active code editors are instantly wiped from the physical monitor, ensuring the privacy boundary remains absolute regardless of network status or user negligence.

Appendix C: Multilanguage Files

C.1 Technical Description

The application relies on a static, file-based dictionary system to manage its internationalization (i18n) capabilities. Each supported language is defined by a dedicated PHP file (e.g., en.php, de.php) residing within the /lang/ directory. These files operate strictly as data providers, returning a single, comprehensive associative array of key-value pairs when included by the backend processor.

The structure of these dictionary arrays is designed for rapid memory loading and immediate O(1) key lookups. The keys act as immutable internal identifiers referenced throughout the backend PHP logic and the frontend JavaScript engine (e.g., view_office, pdf_unstack, confirm_move_title). The corresponding values are the localized text strings.

A critical design feature of these translation files is their support for dynamic data interpolation and rich text formatting. Rather than breaking sentences into fragments to inject variables programmatically, the translation strings utilize standard sprintf format specifiers. For example, the key pdf_merge_where maps to the string "Where should %s be placed relative to %s?". This allows the routing engines to pass dynamic variables—such as newly calculated filenames or numeric counters—directly into the localized string without disrupting the grammatical structure of the target language.

Furthermore, the translation strings natively support embedded HTML and CSS variables. As seen in complex keys like drag_move_fmt, the value can dictate its own typographical hierarchy: "Are you sure you want to move %c item(s) to:
<b style=\"color:var(--accent-color)\">%t?". By embedding generic CSS variables (var(--accent-color)) within the translation string itself, the dictionary ensures that dynamically generated UI modals respect the user's active theme (e.g., Dark Mode or Custom Accents) while maintaining the localized phrasing.

Finally, each dictionary file reserves the __lang_label meta-key. This key does not translate a UI component; instead, it defines the human-readable name of the language file itself (e.g., 'English' or 'Deutsch'). The frontend Settings module parses this specific key across all available files in the directory to dynamically populate the language selection dropdown, ensuring that users can always identify their preferred language in its native script.

Appendix D: Help File Layout

D.1 Technical Description

The application’s documentation engine is decoupled from hardcoded HTML, relying instead on a strict, localized JSON schema. These configuration files reside within the /help/ directory and are named according to their corresponding ISO 639-1 language code (e.g., en.json, de.json). The architecture is designed for rapid client-side parsing, dynamic Role-Based Access Control (RBAC) filtering, and recursive text searching.

D.1.1 Root Schema and Topic Metadata The root of the JSON file is a standard array consisting of "Topic" objects. Each object represents a discrete section in the left-hand navigation tree of the Help Modal. A Topic object requires the following primary metadata:

- id: A unique string identifier for the topic node.
- icon: A string key (e.g., "start", "settings") that maps to a predefined dictionary of SVG assets in the frontend JavaScript engine.
- title: The human-readable string rendered in the navigation menu and search results.
- keywords: A hidden string of space-separated terms. This acts as a localized index to dramatically improve the fuzzy-matching accuracy of the client-side search algorithm without cluttering the visible text.
- role *(Optional)*: If defined (e.g., "role": "write"), the frontend pre-processor evaluates this against the user's active permissions. If the user lacks the required clearance, the entire Topic object is purged from memory before rendering.

D.1.2 Content Block Architecture The body of a Help article is defined by the content array within the Topic object. This array contains sequential blocks, dictating the exact top-to-bottom rendering order of the DOM elements. The renderBlocks() JavaScript parser evaluates the type parameter of each block to construct the appropriate HTML structural nodes:

- **Typography:** h1, h2, h3, and p map directly to their respective HTML equivalents, injected with standardized CSS classes (.ce-help-h1, .ce-help-p).

- **Callouts:** warn and tip generate specialized flexbox containers featuring distinct background colors and pre-pended SVG icons (a warning triangle or a lightbulb) to draw user attention to critical operational details.
- **Structured Data:** The ul type accepts an items array to generate unordered lists. The table type accepts a headers array and a rows array (containing c1 and c2 keys) to build structured data grids.
- **Complex Layouts:** The grid type accepts a cards array, constructing a responsive CSS flex-grid of cards containing title and text nodes, utilized for comparing features or outlining step-by-step logic. The treemap type is a highly specialized block that generates a visual, proportional CSS grid demonstrating storage usage hierarchies.

Note on Granular RBAC: The role attribute can be applied not just at the root Topic level, but directly to individual items within a content array or a ul list. This allows a single article to serve both standard users and administrators, dynamically hiding administrative bullet points from unprivileged users.

D.1.3 Lexical Shortcodes and Interpolation To ensure the JSON file remains strictly formatted and easily maintainable by translators without requiring them to write raw HTML, the engine implements a custom pseudo-markup parser. Before any text node is injected into the DOM, it passes through a Regular Expression (RegEx) filter that replaces proprietary shortcodes with standardized HTML components:

- **Typography:** Text wrapped in asterisks (*text*) is converted to . Text wrapped in [i]text[/i] is converted to .
- **UI Components:** The [badge:Text] shortcode generates a colored, rounded CSS pill, used to reference UI buttons. The [key:Enter] shortcode generates a stylized <kbd> element to denote physical keyboard strokes.
- **Dynamic Assets:** Tokens such as [desktop], [tablet], and [phone] are intercepted and replaced with their corresponding inline SVG path strings, ensuring device iconography scales perfectly with the user's selected font size.

Appendix E: Server Data File Structure

E.1 Technical Overview of the Flat-File Architecture

MyDocpile deliberately eschews traditional relational database management systems (RDBMS) such as MySQL or PostgreSQL. Instead, the application's entire state, configuration, user profiling, and cryptographic metadata are managed through a highly optimized, flat-file JSON (JavaScript Object Notation) architecture, primarily localized within the designated data/, configuration/, and user_profiles/ directories.

This architectural decision is rooted in the principles of maximum portability, zero-dependency deployment, and instantaneous filesystem-level synchronization. By serializing all state data into structured text files, the entire cloud platform—including active sessions, shared links, and user preferences—can be backed up, cloned, or migrated to a new physical server utilizing a single rsync command.

To mitigate the inherent risks of data corruption associated with concurrent flat-file I/O in a multi-threaded web server environment, the platform implements rigorous atomic write operations. Data is never directly overwritten in place. Instead, the PHP backend writes the updated JSON payload to a temporary file (tempnam()). It then acquires an exclusive, blocking filesystem lock (flock($fp, LOCK_EX)) on the target file, truncates it, streams the temporary data, flushes the I/O buffers, and releases the lock. For highly sensitive files, the system utilizes the OS-level atomic rename() function to swap the temporary file over the active file in a single CPU cycle, guaranteeing that a sudden power loss or PHP-FPM crash will never leave a JSON database in a truncated or unparseable state.

The following sections detail the specific structures, lifecycles, and schemas of the critical data files managed by the server.

E.2 User Profiles and Interface State (user_profiles/)

The user_profiles directory serves as the persistence layer for the frontend Single Page Application (SPA). Because the frontend operates dynamically, it continuously serializes its memory state and transmits it to the backend for storage. This directory contains several files per user, keyed by their username.

1. Main Settings Database ([username].json)

- **Description:** This file contains the master configuration object for the user's interface. It is sub-divided into device-specific nodes (desktop, tablet, phone) to allow the UI to adapt based on the accessing hardware.
- **Contents:** Stores boolean toggles (e.g., darkMode, treeOpen, singleClick), integers (fontSize, sidebarSize, appLaunchCount), and UI tracking arrays (renameHistory, visibleTags, tagOrder). It also holds the tagNames dictionary, mapping hex color codes to user-defined strings (e.g., "#e81123": "Urgent").
- **Lifecycle:** Created automatically during the First Run Assistant (FRA). It is updated continuously via a debounced AJAX POST request (save_settings) whenever the user interacts with a toggle or slider in the Settings modal.

2. View Mode Matrix ([username]_views.json)

- **Description:** A key-value dictionary that maps absolute directory paths to specific layout modes.
- **Contents:** The keys are directory paths (e.g., /photos/vacation), and the values are string identifiers representing the render engine state ("list", "symbol", "gallery").
- **Lifecycle:** Updated whenever a user clicks the "Toggle View" button in the toolbar. The frontend parsing engine utilizes this file to calculate view inheritance; if a user navigates to /photos/vacation/2023, the system walks up the JSON keys to inherit the "gallery" view state of the parent folder.

3. State Persistence Ledgers (_favs.json, _paths.json, _tags.json)

- **Favorites (_favs.json):** An array of objects containing path, label, and isDir keys. This populates the quick-access Star menu.
- **Paths (_paths.json):** Tracks the user's exact navigational location. Contains an object mapped to the active cloud key (e.g., {"work_cloud": {"std": "/docs", "cmdLeft": "/docs/invoices", "cmdRight": "/archive"}}). This guarantees that if a user closes the browser, their next session resumes in the exact same directories, preserving both standard and Commander Mode states.

- **Tags (_tags.json):** A complex multidimensional dictionary mapping cloud keys to file paths, which in turn map to arrays of hexadecimal color codes. This powers the visual tagging and sparse-tree search filtering.

E.3 Security, Authorization, and Locking Ledgers

These files govern the perimeter defense of the application, tracking authentication states and penalizing malicious actors.

1. Stateful Authentication Tokens (stateful_tokens.json)

- **Description:** Replaces traditional database tables for persistent "Remember Me" functionality.
- **Schema:** The root keys are 16-byte hex strings acting as public selectors. The values are objects containing the username, expires (Epoch timestamp), hash (an Argon2id or Bcrypt hash of the 32-byte secret validator stored on the client's cookie), and an hmac (a mathematically signed signature of the payload).
- **Lifecycle:** Written upon successful login if the user requested to be remembered. The cronjobs.php housekeeper script or the login router itself actively parses this file, automatically unset()ting and unlinking any objects where the expires integer is less than the current server time, preventing the file from growing indefinitely.

2. Brute-Force and Rate Limiting Databases (bruteforce.json, admin_lockout.json)

- **Description:** Ledgers designed to track and penalize failed authentication attempts.
- **Schema:** The keys are network identifiers (either an IPv4/IPv6 subnet mask for standard logins, or a concatenated Username_IP string for Admin Mode SFTP logins). The values track the count of consecutive failures, the last_attempt timestamp, and the escalation level.
- **Lifecycle:** When a login fails, the counter increments. If the count breaches the allowed threshold, the user is locked out. The lockout duration is calculated dynamically based on the escalation level. Upon a successful login, the specific subnet/user key is entirely deleted from the file, resetting their reputation score.

E.4 Application State and Sharing Databases (data/)

These files manage long-term system configurations and external interactions that bridge multiple users or external environments.

1. The Public Sharing Matrix (shares.json)

- **Description:** The authoritative ledger for all publicly exposed files and directories.
- **Schema:** The root keys are 50-character, cryptographically secure GUIDs (Globally Unique Identifiers). Each GUID maps to an object containing:
    - path: The absolute server path to the exposed directory/file.
    - permission: A string (read, modify, upload) dictating the external UI capabilities.
    - password: A Bcrypt hash of the access password (if configured).
    - expires: An Epoch timestamp dictating the exact second the link self-destructs.
    - downloads & max_downloads: Integers tracking consumption limits.
    - attempts & locked_until: Integrated brute-force protection to prevent password guessing on specific public links.
- **Lifecycle:** Modified by internal users via the Share UI. Monitored continuously by cronjobs.php, which actively deletes objects whose expires value is in the past. If a share reaches its max_downloads threshold during a public file request, the public router itself instantly deletes the JSON node.

2. The Support Ticketing System (tickets.json)

- **Description:** A self-contained Helpdesk and Release Management database.
- **Schema:** An array of objects. Each object contains an id (e.g., tkt_64a9b...), the originating user, the semantic type (Bug, Feature, Security), a title, desc (description), status (Open, In Progress, Closed), and an admin_comment.

- **Lifecycle:** Standard users push new objects to the array. Administrators modify the status and admin_comment strings. To prevent infinite file growth, the backend enforces a 90-day retention policy; any ticket marked Closed with a timestamp older than 90 days is silently dropped during the next save operation.

E.5 The Inter-Process Communication (IPC) Matrix (data/ssh_ipc/)

This directory is uniquely volatile. It does not store long-term data; rather, it acts as a physical nervous system facilitating communication between the primary PHP-FPM web threads and the detached SFTPDaemonProxy background processes running the Admin Mode. Because standard PHP cannot share RAM across processes easily, the filesystem is used as a high-speed message bus.

1. Daemon Control Files (daemon.pid, config.json)

- When an administrator requests a heavy SFTP operation, a unique sub-directory is created (hashed from their session and username).
- A detached CLI process is spawned. It writes its Process ID to daemon.pid. The web thread monitors this file to ensure the daemon is alive.
- config.json temporarily holds the decrypted SSH password and host credentials. The daemon reads this file to connect to the remote server and then *immediately* deletes it to minimize the exposure window of plaintext credentials on the disk.

2. Task Queues and Responses (req_*.json, req_*.res)

- The web thread serializes the administrator's command (e.g., action: delete, src: /var/www/old_site) into a req_[uniqid].tmp file, then atomically renames it to req_[uniqid].json.
- The daemon, polling the directory in a tight loop, detects the .json file, parses it, executes the SFTP operation, and writes the output array to req_[uniqid].res.tmp, renaming it to .res.
- The web thread detects the .res file, transmits the success/failure code to the browser, and unlinks both the .json and .res files.

3. Terminal Streams (*.buf, *.resize)

- To support the live SSH terminal, the browser POSTs keystrokes to the web server, which appends the Base64-decoded characters to a .buf file using a strict LOCK_EX lock.
- The streaming Server-Sent Events (SSE) loop continuously reads this .buf file, pipes the characters into the active SSH socket, and truncates the file to 0 bytes.
- If the user resizes their browser window, the new dimensions are written to a .resize file. The SSE loop detects this, issues a setWindowSize command to the SSH socket, and unlinks the file.

E.6 Cryptographic and Structural In-Tree Data

Unlike the global configuration files, these specific data artifacts are embedded directly within the user's actual file directories, traveling with the files if the directories are moved or synchronized via standard OS-level tools.

1. End-to-End Encryption Roots (.mycloud_crypto_salt)

- **Description:** The definitive marker of an E2EE vault boundary.
- **Contents:** A Base64-encoded JSON payload containing the vault version (e.g., 2), the 32-byte cryptographic salt, the 12-byte iv (Initialization Vector), and the wrappedDEK (Data Encryption Key).
- **Lifecycle:** Created when a user encrypts a folder. It is read every time a user unlocks the vault. If this file is deleted, the entire directory is permanently crypto-shredded and rendered completely unrecoverable, as the DEK is irretrievably lost.

2. Recycle Bin Metadata (.recycle_bin/*.meta)

- **Description:** Operational metadata mapping deleted files to their original locations.
- **Contents:** A tiny JSON object containing the origin (the absolute path where the file resided before deletion) and time (the Epoch timestamp of the deletion event).
- **Lifecycle:** Generated when a user deletes a file with the Recycle Bin enabled. It is named identically to the recycled file (e.g., 16790432_file.pdf.meta). The cronjobs.php housekeeper parses this file. If the time value exceeds the system's retention policy (default 7

days), both the .meta file and the corresponding deleted payload are permanently purged from the disk.

Appendix F: Relevant config.php Variables

F.1. Technical Description

The application's behavior, security perimeters, and physical hardware integrations are entirely governed by a centralized configuration matrix residing in config.php. Unlike relational database-driven settings, this flat-file approach ensures that fundamental system parameters are available to the PHP pre-processor in the first millisecond of execution, before any secondary file I/O or session state is established. The variables defined within this file are strictly categorized into Paths, Files, Security, Session/Network, and Cloud operations.

Core Paths and Data Infrastructure The spatial architecture of the application relies on absolute path declarations to maintain strict cryptographic jails and prevent Directory Traversal attacks.

- $work_dir: The absolute root of the application's backend architecture. The application enforces a critical security paradigm: this directory must reside outside the web-server's public www-root (e.g., /var/www/mycloud_core rather than /var/www/html).
- $action_dir and $list_dir: Sub-directories within the working directory utilized for specific backend routing logic and payload processing.
- $cloud_dir: The physical directory housing the frontend cloud PHP files (the public-facing endpoints).
- $temp_dir: The designated scratchpad for the application. All temporary files (e.g., in-transit zip archives, extracted ONLYOFFICE plaintext bridges, and IPC daemon communication files) are written here.
- $cloud_user_profiles: The absolute path where the application stores the granular, JSON-serialized configuration states, custom color tags, and layout preferences for each individual user account.
- $cloud_icon_cache and $cloud_preview_cache: The isolated storage volumes where the background cronjobs.php housekeeper deposits the flattened, EXIF-stripped JPEG thumbnails and downscaled image previews.

Database and Logging Endpoints The application utilizes independent flat-file ledgers to prevent file-locking bottlenecks during highly concurrent operations.

- $user_db: The core authentication database mapping usernames to their Argon2id or BCrypt hashes, alongside their Role-Based Access Control (RBAC) classifications.
- $cloud_share_db: The routing table for the Public Share module, linking 50-character hexadecimal GUIDs to physical internal paths and expiration timestamps.
- $login_stateful_tokens: The JSON ledger managing persistent "Remember Me" authentication tokens, tracking the cryptographic selectors, validators, and expiration epochs.
- $login_bruteforce_file and $global_login_rate_file: The security ledgers that track failed authentication attempts per subnet mask and global authentication throughput, respectively.
- $security_max_requests_file: The ledger utilized by the API rate-limiting WAF to mitigate Layer 7 Denial of Service (DoS) attacks.
- $log_file and $cloud_logfile: The destinations for standard authentication logging and extensive operational logging (tracking every discrete copy, move, delete, or upload event).
- $cloud_admin_mode_cloudlist: An optional, highly restricted file mapping internal user hashes to remote SFTP credentials, enabling the Auto-Login bypass for the Admin Mode module.

Cryptographic and Perimeter Security These variables establish the strict mathematical thresholds for the application's defensive mechanisms.

- $force_argon2_only: A boolean flag that, when true, forces the main_login.php controller to reject any legacy authentication hashes (e.g., SHA-256) and mandates immediate migration to the memory-hard Argon2id standard.
- $login_failures, $login_block_seconds, $brute_force_window, $brute_force_factor: The variables dictating the exponential backoff algorithm. If an IPv4/IPv6 subnet breaches the $login_failures count, the system calculates a lockout duration of $login_block_seconds * pow($brute_force_factor, $level). The $brute_force_window dictates how long the server remembers a previous lockout before resetting the offender's escalation level.

- $global_login_max_hits and $global_login_window: The absolute ceiling for application-wide authentication attempts. If the total hits exceed the max within the defined window (in seconds), the server triggers a global throttle to mitigate distributed botnet attacks.
- $cloud_upload_blocked_exts: A critical array of file extensions (e.g., .php, .exe, .sh) explicitly forbidden from being written to the disk, halting reverse-shell uploads.
- $cloud_clamav_enabled and $cloud_clamav_path: Enables a synchronous hook into the ClamAV daemon. If enabled, every uploaded temporary file is piped through the binary located at $cloud_clamav_path before it is committed to the final directory.

Session and Networking Matrices

- $timeout_in_minutes: The absolute idle limit. This variable is injected directly into the frontend core.heartbeat.php daemon to establish the local JavaScript termination countdown.
- $cookie_name, $cookie_valid_duration, $cookie_secret: The parameters governing the HttpOnly, Secure, and SameSite parameters of the authentication cookie.
- $cookie_is_ip_bound: A boolean flag that mathematically binds a session ID to the client's IP address. If the client's IP changes mid-session, the session is instantly destroyed (preventing session hijacking, but potentially disrupting mobile users transitioning between Wi-Fi and Cellular networks).
- $allowed_domain, $cloud_only_domains, $cloud_only_paths: Arrays enforcing strict Host-header validation. The system immediately rejects requests from unmapped hostnames to prevent DNS rebinding attacks. The "Cloud Only" variables allow administrators to serve the frontend interface on a dedicated subdomain while isolating the backend processing.
- $enableGeoIP2 and $whitelist_admin_countries: Integrates MaxMind database checks to drop incoming connections originating from unauthorized geographical regions, specifically restricting access to the administrative backend.

Cloud Operations, Limits, and External Integrations

- $cloud_max_preview_size: An integer (in bytes) defining the threshold at which the frontend will interrupt a user with a "Large File Warning" modal prior to attempting to fetch a file into RAM for previewing.
- $zip_warn_limit: The maximum uncompressed byte size permitted before the server warns the user or blocks an on-the-fly ZIP archive generation request, protecting server memory and CPU threads.
- $cloud_icon_maxpixel and $cloud_preview_maxpixel: The exact pixel dimensions utilized by the cronjobs.php ImageMagick/GD pipeline to downscale original media into responsive web thumbnails.
- $cloud_recycle_bin_retention_days: The integer dictating the maximum lifespan of a deleted file. The background daemon reads this to perform automated garbage collection on .meta files.
- $cloud_share_url: The external-facing base URL concatenated with the generated GUIDs to formulate public sharing links.
- $cloud_public_quotas, $cloud_public_max_zip_size, $cloud_public_max_login_attempts: Dedicated, stricter limitation variables applied exclusively to unauthenticated external actors interacting with the Public Share module.
- $cloud_enable_admin_mode: The master kill-switch for the SFTP/SSH terminal capabilities. If false, the backend rejects all administrative daemon requests regardless of user RBAC.
- $cloud_onlyoffice_Secret, $cloud_onlyoffice_URL, $cloud_onlyoffice_ext_URL: The JSON Web Token (JWT) secret key and bidirectional routing addresses required to establish a secure, stateless bridge with an external ONLYOFFICE Document Server container.

F.2. The Workflow

The configuration variables participate in a rigid, two-phase lifecycle: ingestion during standard operation, and atomic mutation during administrative intervention.

During standard operation, config.php acts as the primary bootloader. Every backend script (whether main.php, main_login.php, or controller.server.php) begins execution by issuing a require_once command targeting the configuration file. This securely loads all defined variables directly into the PHP global namespace. Consequently, when a user requests a file

upload, the MyCloudServer class instantaneously accesses $cloud_upload_blocked_exts to validate the file extension without querying a database, ensuring near-zero latency in security evaluations.

The secondary lifecycle occurs when a system administrator alters these variables via the Administrative UI. Because the configuration is stored as raw PHP code rather than a static JSON object, standard string replacement is dangerous. The workflow proceeds as follows:

1. The frontend administrator modifies a setting (e.g., toggling $cloud_clamav_enabled to true) and transmits the JSON payload to the backend via the save_config action.
2. The backend PHP processor receives the payload and initializes a custom Abstract Syntax Tree (AST) parser utilizing token_get_all().
3. The parser iterates through the live config.php source code. It searches for the T_VARIABLE matching the requested key, isolates the assignment operator, and replaces the specific value segment up to the terminating semicolon (T_WHITESPACE and T_COMMENT are intelligently skipped to preserve file formatting).
4. The backend constructs a temporary PHP file with the new structural code and executes a command-line syntax verification (php -l).
5. Upon successful validation, the system acquires an exclusive filesystem lock (LOCK_EX) on the primary config.php file, truncates its contents, writes the newly parsed code string, and closes the file stream.
6. Crucially, the backend immediately invokes opcache_invalidate(). This forces the server's Zend Engine to flush the compiled bytecode of the old configuration from RAM.
7. The very next HTTP request hitting the server parses the new config.php file, applying the updated variables application-wide instantly.

F.3. Usability

The comprehensive nature of the config.php variables provides system administrators with an unparalleled degree of operational control, directly impacting the end-user experience. From an administrative perspective, the exposure of these variables through the Native Admin UI transforms complex server tuning into a highly usable, visual task. An administrator managing a low-resource Virtual Private Server (VPS) can quickly navigate to the

configuration dashboard and lower the $cloud_preview_maxpixel and $zip_warn_limit variables. This instantly prevents users from crashing the server by attempting to process 4K video files or compressing massive directory trees, ensuring platform stability.

For the end-user, the meticulous tuning of these variables defines the boundaries of their digital workspace. The $cookie_is_ip_bound variable perfectly illustrates the balance between usability and security. If an administrator is deploying the application for a highly secure financial firm, they will set this variable to true. The users benefit from absolute session security; if an attacker steals their session cookie, it is useless on a different IP address. However, this impacts usability for mobile users, whose IP addresses change constantly as they walk between cellular towers and office Wi-Fi, forcing them to log in repeatedly. The administrator has the explicit power to weigh this trade-off and configure the platform accordingly.

Furthermore, variables like $timeout_in_minutes and $cloud_recycle_bin_retention_days directly shape the psychological safety of the platform. By setting a generous recycle bin retention policy (e.g., 30 days), users feel empowered to organize their files aggressively, knowing that the system's housekeeper daemon will protect them from accidental deletions for an entire month. Conversely, by fine-tuning the brute-force variables ($brute_force_factor, $login_block_seconds), the administrator guarantees that legitimate users who simply forgot their passwords are only locked out for a few seconds, while automated botnets are mathematically penalized and locked out for days, maintaining a secure yet forgiving environment.

Appendix G: Application Programming Interface (API) and Stateless Endpoints

G.1 Technical Description

While the primary application relies heavily on tightly bound PHP sessions and CSRF tokens, specific infrastructural operations require the capability to communicate securely without a persistent browser session. The API

module (direct_api.php and specific routines within controller.server.php) handles these machine-to-machine integrations.

Stateless Webhooks and Callback Management The most prominent implementation of the stateless API is the OnlyOffice document server bridge. Because the OnlyOffice Docker container operates on an external network layer and does not possess the user's authentication cookies, it cannot execute standard POST requests to save edited documents.

To bridge this gap, the backend exposes the /myCloudOfficeFetch/ and /myCloudOfficeCallback endpoints. Security is enforced entirely via JSON Web Tokens (JWT). When the frontend initiates an OnlyOffice session, it signs a payload using the $cloud_onlyoffice_Secret via the HS256 algorithm. When OnlyOffice completes a document edit, it sends an asynchronous POST request to the callback URL. The PHP backend intercepts this request at the very beginning of the execution cycle. It parses the HTTP_AUTHORIZATION header, recalculates the cryptographic signature using the shared secret, and verifies the payload's integrity. If the signature is valid, the backend accepts the modified file buffer and updates the temporary file on the disk, entirely bypassing the need for a PHP session cookie.

Pre-Signed URLs and Cryptographic Verification For operations requiring direct, stateless access to files—such as downloading an encrypted asset for local client-side decryption—the API utilizes pre-signed, time-limited URLs. The backend generates a random token (bin2hex(random_bytes(20))) and maps it to a specific file path and expiration timestamp in the $_SESSION['myCloud_dl_tokens'] array. When the client's fetch API hits the server with ?myCloud_token=TOKEN, the backend validates the token's existence, verifies that the current Epoch time has not exceeded the expiration limit, and ensures the requesting user's identity hash matches the token's owner. It then streams the binary payload directly via readfile().

G.2 The Workflow

- **Stateless Token Generation:**
    - A user requests a preview or an OnlyOffice session.
    - The backend generates a random token or a signed JWT.
    - The token/JWT is returned to the client as a JSON payload.
- **External Execution:**

    - The client or external server (OnlyOffice) dispatches an HTTP request to the designated API endpoint, appending the token to the URL or Authorization header.

- **Stateless Interception and Validation:**
    - The backend parses the URI. If it matches a stateless pattern (e.g., /myCloudOfficeCallback), it bypasses session initialization.
    - The token/JWT is decoded. The cryptographic signature is validated against the server's private secret.
- **Payload Delivery:**
    - Upon successful validation, the backend processes the incoming webhook (saving the document) or streams the requested binary data.
    - Once the operation concludes, or if the timestamp expires, the token is permanently invalidated.

G.3 Usability

The API architecture allows the platform to seamlessly expand its capabilities by integrating with powerful external microservices. From the user's perspective, clicking a Word document instantly opens a fully-fledged word processor in their browser. They are entirely unaware that the application has securely negotiated a cryptographic token, handed the file off to a separate, stateless document server, and established a secure webhook tunnel for the document server to push the saved changes back into their encrypted vault. The API ensures that these complex, multi-server orchestrations occur instantly and securely, without ever exposing the user's core authentication credentials to the external services.

Appendix H: Installation, Deployment, and Server Requirements

(This part currently is in Beta)

H.1 Technical Description

MyDocpile is engineered to be a highly portable, zero-dependency PHP application, avoiding complex Docker containers or RDBMS setups for the core system. However, its advanced feature set—specifically its robust cryptography, media manipulation, and PDF parsing—demands specific OS-level binaries and PHP extensions.

Core PHP Requirements The application requires **PHP 8.0 or higher**, utilizing modern syntax structures (such as nullsafe assignment operators ??=) and strict typing. The following PHP extensions must be enabled in php.ini:

- mbstring: Required for UTF-8 character encoding and path manipulation.
- json: Required for all API communication and database parsing.
- session: Required for state management.
- pcntl and posix: Absolutely critical for the cronjobs.php housekeeper and the Admin Mode SFTP daemon, enabling true multi-process forking and background execution.
- gd or imagick: Required for image manipulation. Imagick is highly recommended for accurate color-space conversion (CMYK to sRGB) and EXIF auto-rotation.
- zip: Required for on-the-fly archive generation and extraction.

Operating System Binaries To power the advanced Office View and PDF Toolkit, the host Linux/Unix environment must have the following binaries installed and accessible via the system $PATH:

- qpdf: Required for structural PDF manipulation (stacking, splitting, encrypting, linearizing/repairing).
- ghostscript (gs): Used as a fallback rendering engine and for aggressive PDF compression/shrinking.
- pdftk: Required for flattening annotations and permanently injecting WYSIWYG form data into PDFs.

- ffmpeg: Crucial for extracting single-frame thumbnails from MP4, WebM, and MKV video files.
- poppler-utils (pdftoppm, pdftotext, pdfimages): Required for rendering PDFs to canvas, extracting raw text, and ripping embedded imagery.
- tesseract-ocr: Required to perform Optical Character Recognition on scanned, image-only PDFs.

Filesystem Architecture and Security Jails The application enforces a strict split-directory architecture. The cloud_dir (containing index.php and the frontend assets) must be mapped to the web-server's public document root (e.g., /var/www/html). However, the $work_dir (containing config.php, the SQLite/JSON databases, and the user's actual files) **must** reside outside the public root (e.g., /var/www/mycloud_data). Furthermore, the server's open_basedir configuration should be strictly defined to encompass only these specific directories, preventing PHP from ever accessing the underlying host operating system.

H.2 The Workflow

- **Server Preparation:**
    - The administrator provisions a Linux server (Ubuntu/Debian recommended).
    - PHP 8+ and the requisite extensions (Imagick, PCNTL, Zip) are installed.
    - CLI tools (ffmpeg, qpdf, tesseract-ocr, poppler-utils, pdftk) are installed via the OS package manager (e.g., apt-get install).
- **Codebase Deployment:**
    - The application files are cloned to the server.
    - The frontend files are moved to the public web directory.
    - The core engine and data directories are moved to a secure, non-public directory.
- **Configuration:**
    - The administrator modifies config.php, mapping the $work_dir and $cloud_dir variables to their absolute physical paths.

    - Directory permissions are configured (chown -R www-data:www-data), ensuring the PHP-FPM user has read/write access to the data and cache directories.

- **Automation Binding:**
    - The administrator adds the housekeeper script to the server's crontab (e.g., */5 * * * * sudo -u www-data php -d memory_limit=4G /path/to/cronjobs.php).
- **Finalization:**
    - The application is accessed via the web browser. The master administrator account is provisioned, and the deployment is complete.

H.3 Usability

The deployment process is designed to be straightforward for any system administrator familiar with standard LAMP/LEMP (Linux, Apache/Nginx, MySQL, PHP) stacks. Because there is no SQL database to configure, no complex migrations to run, and no Docker containers to orchestrate (unless utilizing the optional OnlyOffice integration), the application can be deployed and operational in under ten minutes.

The requirement for external binaries like ffmpeg and qpdf might appear demanding, but it represents a massive usability upgrade for the end-user. By offloading heavy processing tasks from PHP to optimized, C-compiled OS binaries, the server can crunch through massive PDF merges or video thumbnail extractions in milliseconds, rather than stalling the web server. For the administrator, managing the application post-deployment is incredibly simple: backing up the entire platform, including all user accounts, settings, and files, is as simple as copying a single folder on the hard drive.